Fuzzy Enhanced Private Set Union in Hamming and Minkowski Spaces

Qiang Liu*
Chung-Ang University
Seoul, Republic of Korea
liuqiang0321@gmail.com

JaeYoung Bae*
Chung-Ang University
Seoul, Republic of Korea
qo990102@gmail.com

Hyung Tae Lee†
Chung-Ang University
Seoul, Republic of Korea
hyungtaelee@cau.ac.kr

Joon-Woo Lee†
Chung-Ang University
Seoul, Republic of Korea
jwlee2815@cau.ac.kr

Abstract

Private Set Union (PSU) enables two parties holding private sets X and Y to compute their union $X \cup Y$ without revealing anything else. Enhanced PSU (ePSU) further eliminates during-execution leakage, but existing constructions are limited to exact matching. This restriction is inadequate for many real-world applications involving noisy data, approximate representations, or feature embeddings, where similarity is naturally defined via distance metric rather than strict equality.

In this work, we introduce fuzzy ePSU, a new cryptographic primitive that supports distance-based union while preserving the no during-execution leakage guarantee of ePSU. Given a distance metric $\mathrm{dist}(\cdot,\cdot)$ and a threshold δ, fuzzy ePSU allows the receiver to learn exactly those sender items whose distance to all receiver items exceeds δ, thereby computing a fuzzy union of the two sets.

We present two concrete fuzzy ePSU constructions instantiated over different metric spaces. For the Hamming space, we design a protocol based on a new primitive called Fuzzy Oblivious Non-Membership Conditional Randomness Generation (FOnMCRG), achieving linear complexity in the input set sizes and δ. For the Minkowski space, we introduce Fuzzy Permuted Non-Membership Conditional Randomness Generation (FpnMCRG), which combines fuzzy mapping with hashing-to-bin techniques and achieves (quasi-)linear complexity in the input sizes and dimension.

We implement our protocols and evaluate their performance in both metric spaces. For input sets of size 2^{12}, our Hamming-space protocol incurs about 79.990 MB of communication and 70.686 s of runtime with $\delta = 4$. In the Minkowski space with $\{1, 2, \infty\}$-norm, dimension $d = 10$, and $\delta = 30$, it incurs 388.137–689.889 MB of communication and 347.082–483.328 s of runtime.

Keywords

Fuzzy ePSU, Hamming distance, Minkowski distance

1 Introduction

Private Set Union (PSU) is a fundamental cryptographic primitive that enables two distrustful parties to securely compute the union of their sets without revealing anything else. Due to its central role in applications such as cyber risk assessment [1–3], private ID [4], and privacy-preserving data aggregation [5], extensive efforts have been devoted to designing scalable and efficient PSU protocols [3, 4, 6–11].

However, existing PSU protocols are tailored to exact-match aggregation, making them unsuitable for settings that require fuzzy-match, where similarity is determined by a distance metric rather than strict equality. A natural workaround is to enumerate or expand the receiver's input according to a chosen distance metric and then apply an existing PSU protocol. Unfortunately, this approach becomes impractical in real-world scenarios, since the expanded set can grow exponentially or to an otherwise prohibitive size, while the cost of PSU protocols scales directly with the cardinalities of the input sets. For example, in a d-dimensional space, the volume of a ball with radius δ is on the order of $O(\delta^d)$. As a result, even a moderate distance threshold can lead to an exponential blowup in the number of candidate points that must be included in the expanded set. This motivates the following question:

QUESTION 1: *Can we design a dedicated fuzzy PSU protocol whose complexity depends only on the sizes of the original input sets, rather than on the cardinality of the metric-based expansions?*

In addition, Jia et al. [8] observed that most state-of-the-art PSU protocols [3, 4, 6, 7, 10, 11] suffer from during-execution leakage. To address this issue, they strengthened the standard PSU functionality and proposed an enhanced-security variant, referred to as enhanced PSU (ePSU), together with a concrete protocol realization. The reason for this leakage is that the receiver learns the intersection size before the protocol completes, since that information is implicitly encoded in the oblivious transfer (OT) choice bits. Once the receiver obtains this information prematurely, it already gains partial knowledge about the intersection $X \cap Y$. This premature leakage enables several subtle but powerful attacks. For example, a corrupted receiver who learns the intersection size may abort the execution by claiming a network failure and request a restart. By adaptively modifying its input set across repeated executions and observing how the intersection size changes, the corrupted receiver can gradually infer additional items contained in the intersection.

A similar problem occurs in the fuzzy setting. If a fuzzy PSU protocol allows the receiver to obtain the fuzzy difference set $\mathrm{fuzzy}(X \setminus Y)$ through OT, then the receiver also learns the fuzzy-intersection size before the protocol terminates. This inevitably

*These authors contributed equally to this work.
†Corresponding author.

results in privacy leakage. These observations refine our earlier discussion and lead to a more fundamental question:

QUESTION 2: *Can we design a provably secure fuzzy enhanced PSU protocol that avoids leakage, and whose complexity depends on the sizes of the original input sets rather than on the cardinality of the metric-based expansions?*

Over the past decades, extensive research [12–21] has focused on Fuzzy Private Set Intersection (fuzzy PSI), which supports intersection outputs based on approximate matching. These protocols include in the output those items that satisfy similarity under a specific distance metric. Most existing works study distance measures such as the Hamming distance [12–15] and Minkowski distance [15–19]. Despite this extensive literature on fuzzy PSI, no dedicated solution for fuzzy PSU has been proposed. Nevertheless, the design of such protocols is essential. Consider a metric space such as Euclidean space, which corresponds to the Minkowski space of order 1. Given two sets of points, a fuzzy union can be defined as the union of all points held by one party together with those points of the other party that lie beyond a specified distance threshold from the first set. This notion is meaningful in many practical scenarios, where replacing an exact set union with a fuzzy union yields results that better reflect real-world behavior and tolerance to variability.

Below we highlight some important applications:

- **Biometric database curation.** In biometric systems, fuzzy PSU with Hamming or edit distance [22, 23] can be used to remove near-duplicate templates, including noisy variants of the same fingerprint or face embedding. This prevents dataset inflation and ensures that the curated database retains only genuinely distinct biometric entries.
- **Location and mobility data aggregation.** Many real-world systems collect high-dimensional numerical data such as GPS coordinates [24], Wi-Fi fingerprints [25], or mobility feature vectors [26]. Due to sensor noise and measurement inaccuracies, the same physical event is often recorded as multiple nearby but non-identical points. Fuzzy PSU under Minkowski distance enables privacy-preserving aggregation that eliminates such near-duplicate records. This aggregation reduces noise-induced redundancy, improves the accuracy of downstream analytics (e.g., hotspot detection or traffic estimation), and lowers the computational and storage overhead of subsequent processing.

1.1 Our Contributions

From the above discussion, we conclude that fuzzy PSU is of significant practical relevance and that no dedicated protocol currently exists to address this problem. In this work, we fill this gap. Our primary contributions are summarized below.

Definition of fuzzy (enhanced) PSU ideal functionalities. We formalize the ideal functionality $\mathcal{F}^{m,n}_{\text{fuzzy-PSU}}$ for fuzzy PSU, which captures fuzzy set union in the semi-honest model. We then show that this functionality is insufficient to prevent during-execution leakage when such leakage is not explicitly modeled. To overcome this limitation, we introduce an enhanced ideal functionality $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$ that is resilient to during-execution leakage. The enhanced functionality ensures that the receiver learns no intermediate information prior to the completion of the protocol, thereby providing strictly stronger security guarantees.

Realizing $\mathcal{F}^{m,n}_{\text{fuzzy-PSU}}$ via the mqFRPMT+OT framework and exposing its limitations. Gao et al. [15] introduced a cryptographic primitive called multi-query fuzzy reverse private membership test (mqFRPMT). In mqFRPMT, the sender holds a set $X = \{\boldsymbol{x}_i\}_{i\in[m]}$ and the receiver holds a set $Y = \{\boldsymbol{y}_j\}_{j\in[n]}$. Given a distance function $\mathsf{dist}(\cdot,\cdot)$ and a threshold δ, the protocol allows the receiver to learn, for each $\boldsymbol{x}_i \in X$, a bit e_i indicating whether there exists some $\boldsymbol{y} \in Y$ such that $\mathsf{dist}(\boldsymbol{x}_i, \boldsymbol{y}) \le \delta$, while revealing nothing else about X to the receiver and nothing about Y to the sender. They presented concrete mqFRPMT protocols for both the Hamming and Minkowski spaces by leveraging additively homomorphic encryption. We notice that by composing mqFRPMT with OT, one can directly realize the ideal functionality $\mathcal{F}^{m,n}_{\text{fuzzy-PSU}}$. However, this framework inherently materializes fuzzy membership decisions as explicit bits during execution. As a result, it inevitably leaks intermediate membership information. Consequently, it fundamentally fails to realize the enhanced ideal functionality $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$.

Realizing $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$ in the Hamming space. We introduce a new cryptographic primitive, termed fuzzy oblivious non-membership conditional randomness generation (FOnMCRG), which enables two parties to generate correlated randomness reflecting fuzzy non-membership relations. Informally, for each $\boldsymbol{x}_i \in X$, the parties obtain identical randomness if no $\boldsymbol{y} \in Y$ satisfies $\mathsf{dist}(\boldsymbol{x}_i, \boldsymbol{y}) \le \delta$, and distinct randomness otherwise.[1]

We construct an FOnMCRG protocol for the Hamming space by combining unique components fuzzy mapping (UniqC Fmap) [15], oblivious key-value stores (OKVS) [27, 28], and symmetric-key encryption (SKE) with single-message multi-ciphertext pseudorandomness [7]. A key technical component is a new sub-primitive, oblivious threshold vector decryption non-equality conditional randomness generation (OTVDnECRG), which we realize by composing threshold secret-shared vector oblivious decryption-then-matching (tssVODM) with non-equality conditional randomness generation (nECRG) [9]. The tssVODM functionality is directly implementable via generic two-party computation (2PC), for example through garbled circuits [29] or the GMW [30]..

By combining FOnMCRG with one-time pads, we obtain the first fuzzy ePSU protocol in the Hamming space that achieves strictly linear complexity in the input set sizes and the threshold δ.

Realizing $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$ in the Minkowski space. We introduce a new cryptographic primitive, called fuzzy permuted non-membership conditional randomness generation (FpnMCRG). In this primitive, the sender inputs a set $X = \{\boldsymbol{x}_i\}_{i\in[m]}$ together with a permutation π over $[m]$, and the receiver inputs m sets $\{Y_i\}_{i\in[m]}$. Given a distance function $\mathsf{dist}(\cdot,\cdot)$ and a threshold δ, for each $i \in [m]$, the two parties obtain identical random points if no $\boldsymbol{y} \in Y_i$ satisfies $\mathsf{dist}(\boldsymbol{x}_i, \boldsymbol{y}) \le \delta$, and distinct random points otherwise, with the output position determined by $\pi(i)$.

We construct FpnMCRG protocols in the Minkowski space with two instantiations, one for ∞-norm and one for norms in the range $[1, \infty)$. Our constructions combine 1-out-of-N shared oblivious

[1] In this work, we say that two points are identical if they coincide in all dimensions, and distinct if they differ in all dimensions.

Table 1: Asymptotic communication and computational complexities of our semi-honest secure fuzzy PSU and fuzzy ePSU protocols. The sender holds m points and the receiver holds n points in a d-dimensional space; δ is the threshold; p is the Minkowski order.

Metric	Protocol	Comm.	Comp. ($\mathcal{S}$)	Comp. ($\mathcal{R}$)
Hamming[a]	$\Pi^{\mathrm{dist}_H(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	$O(md^2+\delta dn)$	$O(md^2)$	$O(md^2+\delta dn)$
	$\Pi^{\mathrm{dist}_H(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	$O(md^2+\delta dn)$	$O(md^2)$	$O(md^2+\delta dn)$
∞-norm[b] Minkowski	$\Pi^{\mathrm{dist}_H(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	$O(\delta d(m+n))$	$O(\delta dm+n)$	$O(\delta dn+m)$
	$\Pi^{\mathrm{dist}_H(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	$O(\delta d(m+n)+md\log n\log d)$	$O(\delta dm+n+md\log n\log d)$	$O(\delta dn+md\log n\log d)$
$[1,\infty)$-norm[b] Minkowski	$\Pi^{\mathrm{dist}_H(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	$O(\delta d(m+n)+mp\log\delta)$	$O((\delta d+\delta^p)m+n)$	$O(p\log\delta\, m+\delta dn)$
	$\Pi^{\mathrm{dist}_H(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	$O(\delta d(m+n)+mp\log n(d\log d+\log\delta))$	$O((\delta d+\delta^p\log n+d\log n(\log d+p\log\delta))m+n)$	$O(m\log n(dp\log\delta+d\log d)+\delta dn)$

[a] **R. UniqC**: for each receiver point, there exist at least δ+1 dimensions such that on each of them this point's component differs from others.

[b] **R** $\wedge$ **S. dist. proj.**: for each the sender's (receiver's) point, there exists at least one dimension on which its component keeps a distance greater than 2δ from other the sender's (receiver's) points.

transfer ($\binom{1}{N}$-SOT) [31], distance-aware set augmentation (DA-set augmentation) [14], circuit-based PSI (cPSI) [32], and permuted equality conditional randomness generation (pECRG) [9]. By integrating spatial additive sharing fuzzy mapping (SAS-Fmap) [15], hash-to-bin, FpnMCRG, and one-time pads, we obtain fuzzy ePSU protocols in the Minkowski space with strictly (quasi-)linear complexity in the input set sizes and dimension.

Our fuzzy ePSU protocols in both the Hamming and Minkowski spaces conceal all indication bits throughout the entire execution. The receiver observes only one-time-pad-encrypted values and can recover valid plaintexts solely upon protocol completion, thereby provably eliminating during-execution leakage.

Performance Evaluation. We implement the protocols and conduct experiments in both the Hamming space and the Minkowski space for $\{\infty,1,2\}$-norm, covering a wide range of set sizes, dimensions, and distance thresholds. In the Hamming space, fuzzy ePSU achieves a 15.8–18.4× shrinking in communication and a 11.7–26.0× speedup in runtime compared to fuzzy PSU. In the Minkowski space, although fuzzy PSU demonstrates better performance in low-dimension, fuzzy ePSU achieves communication savings in high-dimension, realizing 1.1–4.4× communication shrinking.

The asymptotic communication and computational complexities of our protocols are given in Table 1. We also give a brief review to the related work in Appendix A.

1.2 Notation

We use κ and λ to denote the computational and statistical security parameters, respectively. For any positive integer n, $[n]$ denotes the set $\{1,\dots,n\}$. The symbol $\leftarrow$ denotes assignment, and $x \xleftarrow{\$} \{0,1\}^\kappa$ denotes uniform sampling from $\{0,1\}^\kappa$. For a value a, $\boldsymbol{a}^d$ denotes the d-dimensional point whose coordinates are all equal to a; in particular, $\mathbf{0}^d$ denotes the all-zero point. For a string x, $x[i:j]$ denotes the substring consisting of bits in positions i through j. We write $\overset{c}{\approx}$ for computational indistinguishability, and $\mathrm{negl}(\kappa)$ for a negligible function in κ. String concatenation is denoted by $\|$.

We use d to denote the dimension and δ the distance threshold. Let $\mathrm{dist}(\cdot,\cdot)$ be a generic distance function. We write $\mathrm{dist}_{\mathrm{H}}(\cdot,\cdot)$ for Hamming distance, $\mathrm{dist}_p(\cdot,\cdot)$ for the p-norm distance in the Minkowski space with $p\in[1,\infty]$, and use $\mathrm{dist}_\infty(\cdot,\cdot)$ when referring specifically to the ∞-norm. The domain is $\mathcal{D}=\mathbb{Z}_{2^\ell}^d$. For a set $X=\{\boldsymbol{x}_i\}_{i\in[m]}$, $\boldsymbol{x}_i$ denotes the i-th point and $x_{i,j}$ its j-th coordinate.

All protocols and functionalities in this work are analyzed in the semi-honest model.

2 Overview of Our Techniques

2.1 Introduction to Fuzzy (enhanced) PSU

Fuzzy (enhanced) PSU allows a sender and a receiver, holding sets $X\subset\mathcal{D}$ and $Y\subset\mathcal{D}$ respectively, to privately compute a fuzzy union under a given distance function $\mathrm{dist}(\cdot,\cdot)$ and a distance threshold δ. For each point $\boldsymbol{x}\in X$, the parties jointly determine whether there exists no point $\boldsymbol{y}\in Y$ such that $\mathrm{dist}(\boldsymbol{x},\boldsymbol{y})\le\delta$. If it holds, the receiver obtains $\boldsymbol{x}$; otherwise, it learns nothing. Finally, the receiver outputs the fuzzy union $Z=Y\cup\{\,\boldsymbol{x}\mid\boldsymbol{x}\in X,\ \forall\boldsymbol{y}\in Y,\ \mathrm{dist}(\boldsymbol{x},\boldsymbol{y})>\delta\,\}$.

2.2 Fuzzy ePSU in Hamming Space from FOnMCRG

In the Hamming setting, the distance function $\mathrm{dist}(\cdot,\cdot)$ is instantiated as the Hamming distance. For any two points $\boldsymbol{x},\boldsymbol{y}\in\mathcal{D}$, their Hamming distance is defined as $\mathrm{dist}_H(\boldsymbol{x},\boldsymbol{y})=\sum_{i=1}^d|x_i\neq y_i|$, which measures the number of coordinates on which the two points differ.

2.2.1 Fuzzy ePSU in the Hamming Space. Our fuzzy ePSU in the Hamming space is designed to eliminate during-execution leakage by avoiding the explicit materialization of fuzzy membership decisions as indicator bits, as is inherent in mqFRPMT+OT framework. Instead, all information related to fuzzy matching is absorbed into correlated randomness, which is consumed only once in the final one-time pad phase. Under this paradigm, the protocol leaks no intermediate information and reveals only the final fuzzy union.

To formalize this approach, we introduce FOnMCRG. Given input sets X and Y, a distance function, and a threshold δ, FOnMCRG generates for each $\boldsymbol{x}_i\in X$ a pair of uniformly random d-dimensional points $(\boldsymbol{u}_{\mathcal{S},i},\boldsymbol{u}_{\mathcal{R},i})$. These two points are identical if and only if $\boldsymbol{x}_i$ is at distance greater than δ from all points in Y; otherwise, they are distinct. The sender then encrypts each coordinate of $\boldsymbol{x}_i$ using $\boldsymbol{u}_{\mathcal{S},i}$ as a one-time pad and appends a authentication tag. The receiver attempts to decrypt using $\boldsymbol{u}_{\mathcal{R},i}$: successful tag verification and recovery of $\boldsymbol{x}_i$ occur only when $\boldsymbol{u}_{\mathcal{S},i}=\boldsymbol{u}_{\mathcal{R},i}$; otherwise, the decryption output is statistically indistinguishable from random and is rejected. As a result, the receiver learns exactly those $\boldsymbol{x}_i$ that do not fuzzy-match any point in Y, yielding the desired fuzzy union.

2.2.2 FOnMCRG in the Hamming Space. Our realization of FOnMCRG in the Hamming space adopts a two-layer structure. The first layer employs the UniqC Fmap to generate identifier sets, ensuring that any pair of points within Hamming distance δ induces at least one identifier collision, while allowing false positives for non-matching points. The second layer is a hidden exact verification layer that filters these false positives without revealing any membership indicators.

Concretely, the receiver samples a random indicator string s and generates a secret key sk. For each point $\boldsymbol{y}_j$, each identifier $\mathsf{id}_{\boldsymbol{y}_j,t} \in \mathsf{ID}(\boldsymbol{y}_j)$, and each $k \in [d]$, the receiver forms $(\mathsf{Enc}(sk,s) \oplus y_{j,1}, \ldots, \mathsf{Enc}(sk,s) \oplus y_{j,d})$ using a SKE satisfying single-message multi-ciphertext pseudorandomness, and encodes all key–value pairs $\big(\mathsf{id}_{\boldsymbol{y}_j,t}, (\mathsf{Enc}(sk,s) \oplus y_{j,1}, \ldots, \mathsf{Enc}(sk,s) \oplus y_{j,d})\big)$ into an OKVS, which is sent to the sender. Using its own identifiers $\mathsf{id}_{\boldsymbol{x}_i,t'} \in \mathsf{ID}(\boldsymbol{x}_i)$, the sender queries the OKVS, XORs the retrieved values with $\boldsymbol{x}_i$, and obtains a collection of verifiable ciphertexts. Whether the number of successful decryptions under (sk,s) exceeds a threshold exactly captures whether the Hamming distance between $\boldsymbol{x}_i$ and some $\boldsymbol{y}_j$ is at most δ. The parties evaluate this condition via a new sub-primitive, OTVDnECRG, which outputs the correlated randomness required by FOnMCRG without leaking the verification outcome.

2.2.3 OTVDnECRG. The OTVDnECRG is constructed by composing two components. The first component, tssVODM, performs threshold decryption checks and outputs secret-shared decision values; it can be instantiated via generic 2PC. The second component, nECRG, converts these secret-shared decisions into correlated randomness: identical randomness is generated for non-membership, while distinct randomness is produced otherwise.

2.3 Fuzzy ePSU in the Minkowski Space from SAS-Fmap and FpnMCRG

In the Minkowski space, the distance function is defined as $\mathsf{dist}_p(\boldsymbol{x},\boldsymbol{y}) = \left(\sum_{i=1}^{d} |x_i - y_i|^p\right)^{1/p}$, for $\boldsymbol{x},\boldsymbol{y} \in \mathcal{D}$ and $p \in [1,\infty)$. In the special case $p = \infty$, the distance reduces to $\mathsf{dist}_\infty(\boldsymbol{x},\boldsymbol{y}) = \max_{i\in[d]} |x_i - y_i|$.

2.3.1 Fuzzy ePSU in the Minkowski Space. Our construction of fuzzy ePSU in the Minkowski space follows the same design philosophy as in the Hamming space. However, directly realizing an FOnMCRG protocol in the Minkowski space is challenging. Minkowski distance depends on additive or max-based aggregation over numerical coordinates, which makes the non-membership condition difficult to compress into a single threshold test that can be fully hidden within correlated randomness. To address this, we adopt a decomposed design strategy, splitting the task into a coarse candidate localization phase and a fine-grained filtering phase.

In the first phase, we perform coarse candidate localization using SAS-Fmap combined with hash-to-bin. SAS-Fmap maps points to identifiers satisfying $\mathsf{dist}_\infty(\boldsymbol{x},\boldsymbol{y}) \le \delta \implies \mathsf{id}(\boldsymbol{x}) = \mathsf{id}(\boldsymbol{y})$, while allowing false positives. Then, the sender inserts its set X into a cuckoo hash table using $\mathsf{id}(\boldsymbol{x})$ as keys, obtaining β bins, each containing at most one point. The receiver inserts its set Y into a simple hash table, placing each $\boldsymbol{y}$ into the bins $\{h_1(\mathsf{id}(\boldsymbol{y})), \ldots, h_\gamma(\mathsf{id}(\boldsymbol{y}))\}$ and padding each bin to a fixed size B. Consequently, each sender bin $C_S[i]$ is associated with a candidate set Y_i that contains all potential δ-neighbors of the bin item, while keeping the size of each candidate set bounded.

In the second phase, we invoke our newly introduced primitive FpnMCRG within each bin to perform precise distance filtering. Concretely, the sender inputs $\{C_S[i]\}_{i\in[\beta]}$ and a permutation π over $[\beta]$, while the receiver inputs the candidate sets $\{Y_i\}_{i\in[\beta]}$. FpnMCRG outputs correlated randomness $\{(\boldsymbol{u}_{S,i}, \boldsymbol{u}_{\mathcal{R},i})\}_{i\in[\beta]}$ such that for each i, $\boldsymbol{u}_{S,i} = \boldsymbol{u}_{\mathcal{R},i} \iff \forall \boldsymbol{y} \in Y_{\pi^{-1}(i)} : \mathsf{dist}_p(C_S[\pi^{-1}(i)], \boldsymbol{y}) > \delta$. Finally, the sender encrypts the (permuted) bin items using $\boldsymbol{u}_{S,i}$ as a one-time pad and attaches authentication tags. The receiver attempts to decrypt using $\boldsymbol{u}_{\mathcal{R},i}$: only when the two randomness points coincide does the verification succeed and the sender's point is recovered; otherwise, the decoding output is statistically indistinguishable from random and is rejected. Consequently, the receiver learns exactly those sender points that admit no δ-neighbor in Y and outputs the fuzzy union.

Here, the permutation π is essential for preventing structural leakage. Without it, the receiver could align each fine-grained filtering instance with its own bins, thereby inferring information about where sender points are placed in the hashing structure. By breaking this correspondence, π ensures that the receiver observes only a collection of unlinkable randomized instances.

2.3.2 FpnMCRG in the Minkowski Space. FpnMCRG is designed to generate permuted correlated randomness that encodes non-membership under a metric threshold. The sender holds bucketed points $\{\boldsymbol{x}_i\}_{i\in[\beta]}$ and a permutation π over $[\beta]$, while the receiver holds β candidate sets $\{Y_i\}_{i\in[\beta]}$ (each of size B). For each index i, the goal is to output a pair of random d-dimensional points $(\boldsymbol{u}_{S,\pi(i)}, \boldsymbol{u}_{\mathcal{R},\pi(i)})$ such that: (i) if $\exists \boldsymbol{y} \in Y_i$ with $\mathsf{dist}_p(\boldsymbol{x}_i,\boldsymbol{y}) \le \delta$ then the two outputs are distinct; otherwise they are identical.

We construct two FpnMCRG protocols in the Minkowski space, corresponding to the cases $p = \infty$ and $p \in [1,\infty)$, respectively. These two protocols follow the same template:

(1) Privately compute a masked proximity predicate against each candidate point;
(2) Aggregate across the B candidates using a private membership test;
(3) Convert the (secret-shared) aggregate outcome into permuted correlated randomness via pECRG.

The two regimes differ in how Step (1) realizes the proximity predicate:

Case $p = \infty$. For $\mathsf{dist}_\infty(\boldsymbol{x},\boldsymbol{y}) \le \delta$, the predicate decomposes into d independent range checks: $|x_k - y_k| \le \delta$ for all $k \in [d]$. We realize each range check using $\binom{1}{N}$-SOT: the receiver encodes the interval membership function into a length-N vector and the sender selects position x_k, obtaining additive shares of a bit indicating whether x_k falls in the interval $[y_k - \delta, y_k + \delta]$. Summing these d shared bits yields a shared value that equals d if all coordinates pass. The sender masks the sum with a random mask and sends the masked value to the receiver, who reconstructs a masked witness $\widetilde{e}$ that equals $\mathsf{mask} + d$ in the match case and is strictly smaller otherwise. Therefore, for a fixed $\boldsymbol{x}$ and B candidates, determining whether any match exists reduces to checking whether the receiver's set of masked witnesses intersects the sender's set $\{e_j\}_{j\in[B]}$, where $e_j = \mathsf{mask}_j + d$. We perform this check via cPSI, which returns only secret-shared membership bit vectors, so no party learns which candidate matched (or whether a match exists) in the clear.

Case $p \in [1,\infty)$. For $p \in [1,\infty)$, proximity is determined by the aggregate constraint $\sum_{k=1}^{d} |x_k - y_k|^p \le \delta^p$. For a fixed receiver candidate point $\boldsymbol{y}$, we define the per-coordinate contribution

$$f^p(x_k) = \begin{cases} |x_k - y_k|^p, & \text{if } |x_k - y_k| \le \delta, \\ \delta^p + 1, & \text{otherwise,} \end{cases} \qquad k \in [d].$$

Using $\binom{1}{N}$-SOT, the parties obtain additive shares of $f^p(x_k)$ for each coordinate and sum them under a random mask. Concretely, the receiver obtains masked witness $\widetilde{e}_j = \mathsf{mask}_j + \sum_{k=1}^{d} f^p(x_k)$. The match condition becomes a bounded-difference test $|\widetilde{e}_j - \mathsf{mask}_j| \leq \delta^p$ rather than an equality test, so cPSI alone is insufficient. A naive expansion over all offsets in $[-\delta^p, \delta^p]$ would incur $\Theta(\delta^p)$ blow-up. Instead, we employ the DA-set augmentation algorithm, which maps each integer into a compact set of representative prefixes such that two values are within distance δ^p if their prefix sets intersect. This yields augmented sets of size $O(Bp \log \delta)$, enabling us to reuse cPSI to test proximity in a compressed manner while keeping the transcript independent of the underlying match relation.

In the above both cases, cPSI outputs secret-shared indicator bit vectors $e_{\mathcal{S},i}$ and $e_{\mathcal{R},i}$ for each bucket i, satisfying $e_{\mathcal{S},i} = e_{\mathcal{R},i}$ in the non-match case and $e_{\mathcal{S},i} \neq e_{\mathcal{R},i}$ otherwise. We hash them to fixed length and replicate them across d coordinates, obtaining $\widetilde{\boldsymbol{e}}_{\mathcal{S},i}$ and $\widetilde{\boldsymbol{e}}_{\mathcal{R},i}$. Finally, we invoke pECRG with permutation π: pECRG outputs $(\boldsymbol{u}_{\mathcal{S},j}, \boldsymbol{u}_{\mathcal{R},j})_{j\in[\beta]}$ such that $\boldsymbol{u}_{\mathcal{S},j} = \boldsymbol{u}_{\mathcal{R},j}$ if $\widetilde{\boldsymbol{e}}_{\mathcal{S},\pi^{-1}(j)} = \widetilde{\boldsymbol{e}}_{\mathcal{R},\pi^{-1}(j)}$. Consequently, each sender bucket obtains identical randomness exactly in the non-membership case, and distinct randomness otherwise, with indices permuted by π.

3 Preliminaries

In this section, we review the security model and cryptographic primitives employed in our constructions.

3.1 Security Model

In the semi-honest adversarial model, the adversaries follow the protocol specification exactly but may attempt to learn additional information by inspecting the transcript. We rely on the standard definition of semi-honest security for two party computation [33]. The formal definition is shown in Appendix B.1.

3.2 Building Blocks

Fuzzy Mapping. Fmap [15] is a two-party protocol that maps each input point to a set of identifiers. It guarantees that identifiers generated from distinct points within the same input set collide only with negligible probability. Moreover, if two points are within distance δ, their identifier sets intersect. However, the converse does not necessarily hold, and thus fuzzy mapping alone is insufficient to determine fuzzy matches, requiring an additional verification step. The formal definition is shown in Appendix B.2.

Oblivious Key-Value Store. OKVS [7, 27, 28] is a data structure that encodes a set of key-value pairs into a compact object from which individual values can be recovered, while revealing nothing about the keys used during encoding. The formal definition is shown in Appendix B.3.

Symmetric Key Encryption. A SKE scheme consists of four algorithms: $\big(\mathsf{Setup}(1^\kappa), \mathsf{KeyGen}(pp), \mathsf{Enc}(sk, m), \mathsf{Dec}(sk, c)\big)$. The detail is shown in Appendix B.4.

Conditional Randomness Generation. We rely on two correlated-randomness primitives from [9]. nECRG outputs identical randomness when the two inputs differ, and outputs distinct randomness when the two inputs are equal. pECRG instead outputs identical randomness when the permuted inputs are equal and outputs distinct randomness otherwise, while additionally supporting an index permutation chosen by the sender. Their ideal functionalities are given in Appendix B.5 and Appendix B.6, respectively.

1-out-of-N Shared Oblivious Transfer. OT [34] is a fundamental cryptographic primitive that allows a receiver to obtain exactly one of the sender's messages without revealing the selection. The $\binom{1}{N}$-OT [35] generalizes this notion to a choice among N messages. The $\binom{1}{N}$ SOT [31], which further extends $\binom{1}{N}$-OT by allowing the choice index to be additively secret-shared and producing additive shares of the selected message. The ideal functionalities for OT and for $\binom{1}{N}$-SOT are shown in Appendix B.7

Distance-Aware Set Augmentation. DA set augmentation [14] transforms each input into a compact set of representative prefixes that encode its local neighborhood under a given distance threshold, enabling efficient distance testing via set intersection. The formal description is provided in Appendix B.8.

Circuit-Based Private Set Intersection. cPSI [32] outputs secret-shared membership information between two sets. The ideal functionality $\mathcal{F}_{\mathsf{cPSI}}$ is described in Appendix B.9.

Multi-query Fuzzy Reverse Private Membership Test. The mqFRPMT functionality allows a receiver to learn, for each sender input, whether there exists a receiver point within distance δ, without revealing any additional information. The ideal functionality is defined in Appendix B.10.

Hash-to-Bin. We use cuckoo hashing and simple hashing [36] to organize inputs into hash tables with bounded bin sizes. The detailed constructions are given in Appendix B.11.

4 Defining Fuzzy (Enhanced) PSU Functionality

In this section, we formalize $\mathcal{F}^{m,n}_{\text{fuzzy-PSU}}$ and present a protocol framework based on the mqFRPMT+OT construction of Gao et al. [15]. We show that this framework inherently incurs during-execution leakage, and therefore that $\mathcal{F}^{m,n}_{\text{fuzzy-PSU}}$ does not capture security against such leakage. Motivated by the enhanced PSU paradigm of Jia et al. [8], we then introduce $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$, which provides provable protection against during-execution leakage.

4.1 Fuzzy PSU Functionality and a Protocol Framework

The fuzzy PSU functionality $\mathcal{F}^{m,n}_{\text{fuzzy-PSU}}$ is formally defined in Figure 1. The sender and the receiver provide their respective input sets $X = \{\boldsymbol{x_i}\}_{i\in[m]}$ and $Y = \{\boldsymbol{y_j}\}_{j\in[n]}$. Finally, the functionality outputs to the receiver the fuzzy union set $Z = Y \cup \mathsf{leakage}(X, Y)$, where $\mathsf{leakage}(X, Y) = \{\boldsymbol{x_i} \mid \forall \boldsymbol{y} \in Y,\ \mathsf{dist}(\boldsymbol{x_i}, \boldsymbol{y}) > \delta\}$.

A protocol that realizes $\mathcal{F}^{m,n}_{\text{fuzzy-PSU}}$ can be constructed using the mqFRPMT + OT framework. Gao et al. [15] presented concrete instantiations of mqFRPMT in both the Hamming and Minkowski metric spaces, which immediately yield realizations of fuzzy PSU in these settings. The concrete protocol is shown in Figure 2.

The correctness and security of $\Pi^{\mathsf{dist}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$ follow from the corresponding analyses in [15]. Thus, we do not repeat the arguments here. The complexity analysis is shown in Appendix C.1.

Parameter: Space dimension d. Distance function $\mathsf{dist}(\cdot,\cdot)$. Distance threshold δ; Leakage function $\mathsf{leakage}(\cdot,\cdot)$.
Functionality:

(1) Wait for input $X = \{\boldsymbol{x_i}\}_{i\in[m]} \subset \mathcal{D}$ from $\mathcal{S}$.
(2) Wait for input $Y = \{\boldsymbol{y_j}\}_{j\in[n]} \subset \mathcal{D}$ from $\mathcal{R}$.
(3) Compute and output $Z = Y \cup \mathsf{leakage}(X, Y)$ to $\mathcal{R}$.

Leakage Function:

$$\mathsf{leakage}(X, Y) = \{\boldsymbol{x_i} \mid \forall \boldsymbol{y} \in Y,\ \mathsf{dist}(\boldsymbol{x_i}, \boldsymbol{y}) > \delta\}.$$

Figure 1: Fuzzy private set union functionality: $\mathcal{F}^{m,n}_{\text{fuzzy-PSU}}$

Parameter setting: The sender $\mathcal{S}$ holds $X = \{\boldsymbol{x_i}\}_{i\in[m]} \subset \mathcal{D}$. The receiver $\mathcal{R}$ holds $Y = \{\boldsymbol{y_j}\}_{j\in[n]} \subset \mathcal{D}$. Space dimension d. Threshold δ. Distance function $\mathsf{dist}(\cdot,\cdot)$.
Protocol:

(1) $\mathcal{S}$ applies a random permutation to X.
(2) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}^{\mathsf{dist}(\cdot,\cdot)}_{\text{mqFRPMT}}$:
   - $\mathcal{S}$ acts as sender with input X, and $\mathcal{R}$ acts as receiver with input Y. As a result, $\mathcal{R}$ obtains an indicator vector $e \in \{0,1\}^m$.
(3) $\mathcal{R}$ initializes $Z \leftarrow Y$ and temporary set $\widetilde{Z} \leftarrow \emptyset$.
(4) For each $i \in [m]$, $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\text{OT}}$:
   - $\mathcal{S}$ acts as sender with input $(\boldsymbol{x_i}, \perp)$, and $\mathcal{R}$ acts as receiver with input e_i. As a result, $\mathcal{R}$ obtains $\boldsymbol{m}_{e_i} \in \{\boldsymbol{x_i}, \perp\}$. If $\boldsymbol{m}_{e_i} \neq \perp$, then $\mathcal{R}$ updates $\widetilde{Z} \leftarrow \widetilde{Z} \cup \{\boldsymbol{m}_{e_i}\}$.
(5) $\mathcal{R}$ outputs the fuzzy union $Z = Y \cup \widetilde{Z}$.

Figure 2: Fuzzy private set union protocol: $\Pi^{\mathsf{dist}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$

Vulnerability Analysis Under During-Execution Leakage. From the fuzzy PSU protocol in Figure 2, we observe that after Step (2), the receiver obtains a bit vector e, with each bit e_i indicating whether there exists $\boldsymbol{y_j} \in Y$ such that $\mathsf{dist}(\boldsymbol{x_i}, \boldsymbol{y_j}) \leq \delta$. Thus, the receiver learns the fuzzy intersection pattern before the protocol completes, which constitutes a during-execution leakage.

At first glance, this leakage may appear benign if the protocol guarantees that the receiver eventually obtains the final output. However, in realistic deployment settings, protocol executions may be interrupted due to network failures, system crashes, or deliberate aborts by a corrupted receiver. Once the receiver has obtained e, it can intentionally terminate the execution and exploit this prematurely revealed information. In particular, a corrupted receiver may mount an incremental attack by repeatedly initiating protocol executions and aborting them after learning the fuzzy intersection size. Although the honest party may enforce rate limits or basic restart controls, reconnections after failures are typically permitted in practice. By strategically interrupting and restarting executions, the receiver can gradually obtain more fuzzy intersection sizes to infer the fuzzy intersections.

Hence, during-execution leakage constitutes a realistic security threat that must be explicitly addressed. While the protocol in Figure 2 correctly realizes the functionality $\mathcal{F}^{m,n}_{\text{fuzzy-PSU}}$ defined in Figure 1, it nevertheless permits such leakage. This demonstrates that $\mathcal{F}^{m,n}_{\text{fuzzy-PSU}}$ fails to capture security against such leakage.

4.2 Fuzzy ePSU Functionality

We define a fuzzy ePSU functionality, denoted $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$, which strengthens the security guarantees of $\mathcal{F}^{m,n}_{\text{fuzzy-PSU}}$ by explicitly protecting against during-execution leakage. In particular, the functionality prevents the receiver from learning any information, including fuzzy intersection indicators, before the protocol completes. The formal definition of $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$ is given in Figure 3.

Parameter: The functionality interacts with the sender $\mathcal{S}$, the receiver $\mathcal{R}$, and the simulator Sim. Space dimension d. Distance function $\mathsf{dist}(\cdot,\cdot)$; Distance threshold δ. Leakage function $\mathsf{leakage}(\cdot,\cdot)$.
Functionality:

(1) Initialize an ideal state $\mathsf{state}_{\mathcal{U}} = \emptyset$ for each party $\mathcal{U} \in \{\mathcal{S}, \mathcal{R}\}$. If $\mathcal{U}$ is corrupt, Sim is allowed to access $\mathsf{state}_{\mathcal{U}}$.
(2) Wait for input $X = \{\boldsymbol{x_i}\}_{i\in[m]} \subset \mathcal{D}$ from $\mathcal{S}$; abort if $|X| \neq m$. Otherwise, update $\mathsf{state}_{\mathcal{S}} = \langle X \rangle$ and send $\langle \mathsf{Request}, \mathcal{S} \rangle$ to Sim.
(3) Wait for input $Y = \{\boldsymbol{y_j}\}_{j\in[n]} \subset \mathcal{D}$ from $\mathcal{R}$; abort if $|Y| \neq n$. Otherwise, update $\mathsf{state}_{\mathcal{R}} = \langle Y \rangle$ and send $\langle \mathsf{Request}, \mathcal{R} \rangle$ to Sim.
(4) Upon receiving $\langle \mathsf{Request}, \mathsf{OK} \rangle$ from Sim, compute $Z = Y \cup \mathsf{leakage}(X, Y)$.
(5) Update internal states: $\mathsf{state}_{\mathcal{S}} \leftarrow \mathsf{state}_{\mathcal{S}} \cup \langle \mathsf{Finished} \rangle$, $\mathsf{state}_{\mathcal{R}} \leftarrow \mathsf{state}_{\mathcal{R}} \cup \langle Z \rangle$.
(6) Output Z to $\mathcal{R}$ and $\langle \mathsf{Finished} \rangle$ to $\mathcal{S}$.

Leakage Function:

$$\mathsf{leakage}(X, Y) = \{\boldsymbol{x_i} \mid \forall \boldsymbol{y} \in Y,\ \mathsf{dist}(\boldsymbol{x_i}, \boldsymbol{y}) > \delta\}.$$

Figure 3: Fuzzy enhanced private set union functionality: $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$

The functionality $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$ shown in Figure 3 follows the design principles of the ePSU functionality [8]. The key difference between $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$ and $\mathcal{F}^{m,n}_{\text{fuzzy-PSU}}$ is that $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$ additionally delivers an output $\langle \mathsf{Finished} \rangle$ to the sender once the joint computation is complete. Returning $\langle \mathsf{Finished} \rangle$ to the sender is reasonable, since in any real-world execution the sender must know whether the computation has finished; therefore, the same behavior should be reflected in the ideal world as well. Furthermore, we explicitly specify the interactions between the functionality and the simulator. For example, whenever $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$ receives an input, the simulator must be notified, and whenever a party becomes corrupted, the simulator must be allowed to learn the corresponding internal state.

Analysis of Resistance to During Execution Leakage We use the fuzzy PSU protocol in Figure 2 to illustrate why the additional output $\langle \mathsf{Finished} \rangle$ is essential for preventing during-execution leakage. Consider an adversary that corrupts the receiver. At some time t, the mqFRPMT phase has completed, while the OT phase has not yet begun. At this point, the simulator must generate a string

$e \in \{0,1\}^m$ for the adversary. The simulator must decide whether to send the message $\langle \text{Response}, \text{OK} \rangle$ to $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$ immediately or to delay this response.

If the simulator immediately sends $\langle \text{Response}, \text{OK} \rangle$ to $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$, it can successfully simulate the string $e \in \{0,1\}^m$. However, the sender will then receive $\langle \text{Finished} \rangle$ from the functionality and forward it to the environment in the ideal world, even though the real world execution has not yet completed. The environment can therefore distinguish the ideal execution from the real one.

If the simulator delays its response, the environment will not receive $\langle \text{Finished} \rangle$ in either world. Nevertheless, in the real world, the length $|e|$ corresponds to the size of the fuzzy intersection of X and Y. Since both X and Y are chosen by the environment, it already knows the fuzzy intersection size. In contrast, in the ideal world, the simulator has only negligible probability of correctly guessing this size. Consequently, the environment can again distinguish between the two worlds.

5 Our Fuzzy ePSU Protocol in Hamming Space

In this section, we introduce two auxiliary primitives, tssVODM and OTVDnECRG, and use them to construct a protocol that realizes our core functionality, FOnMCRG. Building on FOnMCRG, we then realize a fuzzy ePSU protocol in the Hamming space.

5.1 New Building Blocks

Threshold Secret-Shared Vector Oblivious Decryption-then-Matching. The tssVODM is a threshold variant of the VODM [7]. Formally, the sender holds d ciphertext sets $\{C_i\}_{i\in[d]}$, each of size d', and a threshold value $t \in [d']$. The receiver holds a secret key-message pair (sk, s). The functionality outputs random values $e_{\mathcal{S}}$ and $e_{\mathcal{R}}$ to the sender and the receiver, respectively, such that $e_{\mathcal{S}} \oplus e_{\mathcal{R}} = 1 \iff \forall\, i \in [d] : \sum_{j=1}^{d} |\text{Dec}(sk, c_{i,j}) = s| < t$; otherwise, $e_{\mathcal{S}} \oplus e_{\mathcal{R}} = 0$. The ideal functionality $\mathcal{F}_{\text{tssVODM}}$ is shown in Figure 4.

Parameter: The functionality interacts with the sender $\mathcal{S}$, the receiver $\mathcal{R}$, and the simulator *Sim*. Set size d'. An encryption scheme $\epsilon = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$.
Functionality:

(1) Initialize an ideal state $\text{state}_{\mathcal{U}} = \emptyset$ for each party $\mathcal{U} \in \{\mathcal{S}, \mathcal{R}\}$. If $\mathcal{U}$ is corrupt, *Sim* is allowed to access $\text{state}_{\mathcal{U}}$.
(2) Wait for inputs $\{C_i\}_{i\in[d]}$ and t from $\mathcal{S}$, update $\text{state}_{\mathcal{S}} = \langle \{C_i\}_{i\in[d]}, t \rangle$ and send $\langle \text{Request}, \mathcal{S} \rangle$ to *Sim*.
(3) Wait for inputs sk and s from $\mathcal{R}$, update $\text{state}_{\mathcal{R}} = \langle sk, s \rangle$ and send $\langle \text{Request}, \mathcal{R} \rangle$ to *Sim*.
(4) Upon receiving $\langle \text{Request}, \text{OK} \rangle$ from *Sim*, generate two random values $e_{\mathcal{S}}$ and $e_{\mathcal{R}}$ such that $e_{\mathcal{S}} \oplus e_{\mathcal{R}} = 1 \iff \forall\, i \in [d] : \sum_{j=1}^{d} |\text{Dec}(sk, c_{i,j}) = s| < t$, and $e_{\mathcal{S}} \oplus e_{\mathcal{R}} = 0$ otherwise.
(5) Update internal states: $\text{state}_{\mathcal{S}} \leftarrow \text{state}_{\mathcal{S}} \cup \langle e_{\mathcal{S}} \rangle$ and $\text{state}_{\mathcal{R}} \leftarrow \text{state}_{\mathcal{R}} \cup \langle e_{\mathcal{R}} \rangle$.
(6) Output $\langle e_{\mathcal{S}} \rangle$ to $\mathcal{S}$ and $\langle e_{\mathcal{R}} \rangle$ to $\mathcal{R}$.

Figure 4: Threshold secret-shared vector oblivious decryption-then-matching functionality: $\mathcal{F}_{\text{tssVODM}}$

As noted in section 2.2, a generic 2PC is sufficient to implement $\mathcal{F}_{\text{tssVODM}}$, and both correctness and security follow directly from the guarantees of generic 2PC. The complexity analysis is shown in Appendix C.2

Oblivious Threshold Vector Decryption Non-Equality Conditional Randomness Generation. The OTVDnECRG takes as input from the sender m collections of ciphertext sets $\{C_{i,1}, \ldots, C_{i,d}\}_{i\in[m]}$ and a threshold $t \in [d]$, and from the receiver a secret key-message pair (sk, s). It outputs to the sender and the receiver two sets of uniformly random d-dimensional points $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ and $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$, respectively. For each $i \in [m]$, the outputs satisfy $\boldsymbol{u}_{\mathcal{S},i} = \boldsymbol{u}_{\mathcal{R},i} \iff \forall\, j \in [d] : |\{c \in C_{i,j} : \text{Dec}(sk, s) = s\}| < t$. The ideal functionality $\mathcal{F}_{\text{OTVDnECRG}}$ is presented in Figure 5.

Parameter: The functionality interacts with the sender $\mathcal{S}$, the receiver $\mathcal{R}$, and the simulator *Sim*. Set size parameter d'. An encryption scheme $\epsilon = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec})$.
Functionality:

(1) Initialize an ideal state $\text{state}_{\mathcal{U}} = \emptyset$ for each party $\mathcal{U} \in \{\mathcal{S}, \mathcal{R}\}$. If $\mathcal{U}$ is corrupt, *Sim* is allowed to access $\text{state}_{\mathcal{U}}$.
(2) Wait for inputs $\{C_{i,1}, \ldots, C_{i,d}\}_{i\in[m]}$ and t from $\mathcal{S}$, update $\text{state}_{\mathcal{S}} = \langle \{C_{i,j}\}_{i\in[m],j\in[d]}, t \rangle$, and send $\langle \text{Request}, \mathcal{S} \rangle$ to *Sim*.
(3) Wait for inputs sk and s from $\mathcal{R}$, update $\text{state}_{\mathcal{R}} = \langle sk, s \rangle$, and send $\langle \text{Request}, \mathcal{R} \rangle$ to *Sim*.
(4) Upon receiving $\langle \text{Request}, \text{OK} \rangle$ from *Sim*, generate two random sets $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ and $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$. For each $i \in [m]$, $\boldsymbol{u}_{\mathcal{S},i} = \boldsymbol{u}_{\mathcal{R},i} \iff \forall\, j \in [d] : |\{c \in C_{i,j} : \text{Dec}(sk, s) = s\}| < t$.
(5) Update internal states: $\text{state}_{\mathcal{S}} \leftarrow \text{state}_{\mathcal{S}} \cup \langle \{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]} \rangle$ and $\text{state}_{\mathcal{R}} \leftarrow \text{state}_{\mathcal{R}} \cup \langle \{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]} \rangle$.
(6) Output $\langle \{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]} \rangle$ to $\mathcal{S}$ and $\langle \{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]} \rangle$ to $\mathcal{R}$.

Figure 5: Oblivious threshold vector decryption non-equality conditional randomness generation functionality: $\mathcal{F}_{\text{OTVDnECRG}}$

We design the protocol $\Pi_{\text{OTVDnECRG}}$, presented in Figure 6, by composing tssVODM and nECRG. For each $i \in [m]$, the sender and the receiver jointly invoke tssVODM on inputs $\{C_{i,1}, \ldots, C_{i,d}\}$, a threshold t, and (sk, s), respectively, obtaining secret-shared outputs $e_{\mathcal{S},i}$ and $e_{\mathcal{R},i}$. Each share is then replicated across d coordinates and fed into nECRG, yielding correlated randomness $\boldsymbol{u}_{\mathcal{S},i}$, $\boldsymbol{u}_{\mathcal{R},i}$ such that $\boldsymbol{u}_{\mathcal{S},i} = \boldsymbol{u}_{\mathcal{R},i} \iff e_{\mathcal{S},i} = e_{\mathcal{R},i}$.

We prove the correctness and security of $\Pi_{\text{OTVDnECRG}}$ in Appendix D.1. The complexity analysis is shown in Appendix C.3.

5.2 Fuzzy Oblivious Non-Membership Conditional Randomness Generation

We introduce a new cryptographic primitive, FOnMCRG. The sender holds a point set $X = \{\boldsymbol{x}_i\}_{i\in[m]}$ and the receiver holds a point set $Y = \{\boldsymbol{y}_j\}_{j\in[n]}$. Given a distance function $\text{dist}(\cdot,\cdot)$ and a threshold δ,

Parameter setting: The sender $\mathcal{S}$ holds $\{C_{i,1}, \ldots, C_{i,d}\}_{i\in[m]}$ and t. The receiver $\mathcal{R}$ holds sk and s. Set size parameter d'. An encryption scheme $\epsilon = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$.
Protocol:

(1) For each $i \in [m]$, the parties invoke $\mathcal{F}_{\mathsf{tssVODM}}$:
- $\mathcal{S}$ acts as sender with inputs $\{C_{i,1}, \ldots, C_{i,d}\}$ and t, and $\mathcal{R}$ acts as receiver with inputs sk and s. As a result, They obtain random shares $e_{\mathcal{S},i}$ and $e_{\mathcal{R},i}$, respectively.

(2) For each $i \in [m]$, both parties set $\widetilde{\boldsymbol{e}}_{\mathcal{S},i} = \boldsymbol{e}^d_{\mathcal{S},i}$ and $\widetilde{\boldsymbol{e}}_{\mathcal{R},i} = \boldsymbol{e}^d_{\mathcal{R},i}$, respectively.

(3) The parties invoke $\mathcal{F}_{\mathsf{nECRG}}$:
- $\mathcal{S}$ acts as sender with input $\{\widetilde{e}_{\mathcal{S},i,1}, \ldots, \widetilde{e}_{\mathcal{S},i,d}\}_{i\in[m]}$, and $\mathcal{R}$ acts as receiver with input $\{\widetilde{e}_{\mathcal{R},i,1}, \ldots, \widetilde{e}_{\mathcal{R},i,d}\}_{i\in[m]}$. As a result, They obtain $\{u_{\mathcal{S},i,1}, \ldots, u_{\mathcal{S},i,d}\}_{i\in[m]}$ and $\{u_{\mathcal{R},i,1}, \ldots, u_{\mathcal{R},i,d}\}_{i\in[m]}$, respectively.

(4) $\mathcal{S}$ outputs $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ and $\mathcal{R}$ outputs $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$.

Figure 6: Oblivious threshold vector decryption non-equality conditional randomness generation protocol: $\Pi_{\mathsf{OTVDnECRG}}$

the functionality outputs to the sender and the receiver uniformaly random point sets $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ and $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$, respectively, such that for each $i \in [m]$, $\boldsymbol{u}_{\mathcal{S},i} = \boldsymbol{u}_{\mathcal{R},i} \iff \forall\, \boldsymbol{y}_j \in Y : \mathsf{dist}(\boldsymbol{x}_i, \boldsymbol{y}_j) > \delta$. We formally specify the ideal functionality of FOnMCRG, denoted by $\mathcal{F}_{\mathsf{FOnMCRG}}$, in Figure 7.

Parameter: The functionality interacts with the sender $\mathcal{S}$, the receiver $\mathcal{R}$, and the simulator Sim. Space dimension d. Distance function $\mathsf{dist}(\cdot,\cdot)$. Distance threshold δ.
Functionality:

(1) Initialize an ideal state $\mathsf{state}_{\mathcal{U}} = \emptyset$ for each party $\mathcal{U} \in \{\mathcal{S}, \mathcal{R}\}$. If $\mathcal{U}$ is corrupt, Sim is allowed to access $\mathsf{state}_{\mathcal{U}}$.

(2) Wait for input $X = \{\boldsymbol{x_i}\}_{i\in[m]} \subset \mathcal{D}$ from $\mathcal{S}$, update $\mathsf{state}_{\mathcal{S}} = \langle X\rangle$ and send $\langle \mathsf{Request}, \mathcal{S}\rangle$ to Sim.

(3) Wait for input $Y = \{\boldsymbol{y}_j\}_{j\in[n]} \subset \mathcal{D}$ from $\mathcal{R}$, update $\mathsf{state}_{\mathcal{R}} = \langle Y\rangle$ and send $\langle \mathsf{Request}, \mathcal{R}\rangle$ to Sim.

(4) Upon receiving $\langle \mathsf{Request}, \mathsf{OK}\rangle$ from Sim, generate two random point sets $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ and $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$ such that for each $i \in [m]$:
- If there exists no $\boldsymbol{y}_j \in Y$ satisfying $\mathsf{dist}(\boldsymbol{x_i}, \boldsymbol{y}_j) \le \delta$, then $\boldsymbol{u}_{\mathcal{S},i} = \boldsymbol{u}_{\mathcal{R},i}$; Otherwise set $\boldsymbol{u}_{\mathcal{S},i} \neq \boldsymbol{u}_{\mathcal{R},i}$.

(5) Update internal states: $\mathsf{state}_{\mathcal{S}} \leftarrow \mathsf{state}_{\mathcal{S}} \cup \langle \{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}\rangle$ and $\mathsf{state}_{\mathcal{R}} \leftarrow \mathsf{state}_{\mathcal{R}} \cup \langle \{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}\rangle$.

(6) Output $\langle \{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}\rangle$ to $\mathcal{S}$ and $\langle \{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}\rangle$ to $\mathcal{R}$.

Figure 7: Fuzzy oblivious non-membership conditional randomness generation functionality: $\mathcal{F}_{\mathsf{FOnMCRG}}$

Figure 8 gives the concrete realization of FOnMCRG in the Hamming space. Specifically, the parties invoke the UniqC Fmap to obtain identifier sets $\mathsf{ID}(X)$ and $\mathsf{ID}(Y)$, which ensure that $\mathsf{dist}_H(\boldsymbol{x}_i, \boldsymbol{y}_j) \le \delta$ implies at least one identifier collision, while allowing false positives for non-matching points. Then, the receiver samples a random string s and a secret key sk. For each identifier $\mathsf{id}_{\boldsymbol{y}_i,j} \in \mathsf{ID}(\boldsymbol{y}_i)$, where $i \in [n]$, the receiver encodes the masked vector $(\mathsf{Enc}(\mathsf{sk}, s) \oplus y_{i,1}, \ldots, \mathsf{Enc}(\mathsf{sk}, s) \oplus y_{i,d})$ into an OKVS indexed by $\mathsf{id}_{\boldsymbol{y}_i,j}$. Using its own identifiers $\mathsf{id}_{\boldsymbol{x}_i,j} \in \mathsf{ID}(\boldsymbol{x}_i)$, where $i \in [m]$, the sender decodes the OKVS and XORs the retrieved values with $\boldsymbol{x}_i$, obtaining collections of ciphertexts $\{C_{i,j}\}_{j\in|\mathsf{id}(\boldsymbol{x}_i)|}$. The parties then invoke the OTVDnECRG with threshold $t = d - \delta$, which privately checks whether the number of coordinates that decrypt to s reaches the threshold, and outputs correlated randomness $(\boldsymbol{u}_{\mathcal{S},i}, \boldsymbol{u}_{\mathcal{R},i})$.

Parameter setting: The sender $\mathcal{S}$ holds $X = \{\boldsymbol{x}_i\}_{i\in[m]} \subset \mathcal{D}$. The receiver $\mathcal{R}$ holds $Y = \{\boldsymbol{y}_j\}_{j\in[n]} \subset \mathcal{D}$. Space dimension d. Distance threshold δ. An OKVS scheme $(\mathsf{Encode}, \mathsf{Decode})$ and a randomness value r. A SKE $\epsilon = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec})$ that satisfies single-message multi-ciphertext pseudorandomness.
Protocol:

(1) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}^{\mathsf{UniqC}}_{\mathsf{Fmap}}$:
- $\mathcal{S}$ acts as sender with input X, and $\mathcal{R}$ acts as receiver with input Y. As a result, $\mathcal{S}$ receives $\mathsf{ID}(X)$ and $\mathcal{R}$ receives $\mathsf{ID}(Y)$.

(2) $\mathcal{R}$ selects a random indicator string s. $\mathcal{R}$ computes $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\kappa)$ and $sk \leftarrow \mathsf{KeyGen}(\mathsf{pp})$. For each $i \in [n]$, each $j \in [|\mathsf{id}(\boldsymbol{y}_i)|]$, and each $k \in [d]$, $\mathcal{R}$ computes $s_{i,j,k} = \mathsf{Enc}(sk, s)$.

(3) $\mathcal{R}$ initializes the set $\mathsf{M} \leftarrow \emptyset$. For each $i \in [n]$ and each $j \in [|\mathsf{id}(\boldsymbol{y}_i)|]$, $\mathcal{R}$ updates
$$\mathsf{M} \leftarrow \mathsf{M} \cup \left\{\left(\mathsf{id}_{\boldsymbol{y}_i,j},\ (s_{i,j,1} \oplus y_{i,1}, \ldots, s_{i,j,d} \oplus y_{i,d})\right)\right\}.$$
$\mathcal{R}$ encodes $\mathsf{D} \leftarrow \mathsf{Encode}(\mathsf{M}, r)$ and sends D to $\mathcal{S}$.

(4) For each $i \in [m]$ and each $j \in [|\mathsf{id}(\boldsymbol{x}_i)|]$, $\mathcal{S}$ computes
$$C_{i,j} \leftarrow \mathsf{Decode}(\mathsf{D}, \mathsf{id}_{\boldsymbol{x}_i,j}, r) \oplus (x_{i,1}, \ldots, x_{i,d}).$$
Thus $\mathcal{S}$ forms the collection $\{C_{i,1}, \ldots, C_{i,|\mathsf{id}(\boldsymbol{x}_i)|}\}_{i\in[m]}$.

(5) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\mathsf{OTVDnECRG}}$:
- $\mathcal{S}$ acts as sender with inputs threshold $t = d - \delta$ and $\{C_{i,1}, \ldots, C_{i,|\mathsf{id}(\boldsymbol{x}_i)|}\}_{i\in[m]}$, and $\mathcal{R}$ acts as receiver with inputs sk and s. As a result, $\mathcal{S}$ receives $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ and $\mathcal{R}$ receives $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$.

Figure 8: Fuzzy oblivious non-membership conditional randomness generation protocol in Hamming space: $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$

We prove the correctness and security of $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$ in Appendix D.2. The complexity analysis is shown in Appendix C.4.

5.3 Our Fuzzy ePSU Protocol for Hamming Distance

We give the concrete realization of the fuzzy ePSU in the Hamming space, as shown in Figure 9. Specifically, after invoking FOnMCRG, the sender and the receiver obtain correlated randomness $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ and $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$, respectively. For each i, the outputs coincide if and only if $\boldsymbol{x}_i$ has no δ-neighbor in Y. The sender then encrypts each coordinate of $\boldsymbol{x}_i$ using $\boldsymbol{u}_{\mathcal{S},i}$ as a one-time pad

Parameter setting: The sender $\mathcal{S}$ holds $X = \{\boldsymbol{x}_i\}_{i\in[m]} \subset \mathcal{D}$. The receiver $\mathcal{R}$ holds $Y = \{\boldsymbol{y}_j\}_{j\in[n]} \subset \mathcal{D}$. Space dimension d. Distance threshold δ. Distance function $\mathrm{dist}_H(\cdot,\cdot)$. A collusion resistant hash function $h : \{0,1\}^* \to \{0,1\}^{\ell_1}$ with $\ell_1 \geq \lambda + 2\log(md)$.

Protocol:

(1) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\mathrm{FOnMCRG}}^{\mathrm{dist_H}(\cdot,\cdot)}$:
- $\mathcal{S}$ acts as sender with input X and $\mathcal{R}$ acts as receiver with input Y. As a result, $\mathcal{S}$ receives $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ and $\mathcal{R}$ receives $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$.

(2) For each $i \in [m]$, $\mathcal{S}$ computes the vector $\boldsymbol{u}_i$ where for each $j \in [d]$, $u_{i,j} = u_{\mathcal{S},i,j} \oplus \big(x_{i,j} \,\|\, h(x_{i,j})\big)$. Then $\mathcal{S}$ sends $\{\boldsymbol{u}_i\}_{i\in[m]}$ to $\mathcal{R}$.

(3) $\mathcal{R}$ initializes $\mathsf{Z} \leftarrow \emptyset$ and $\widetilde{\mathsf{Z}} \leftarrow \emptyset$. For each $i \in [m]$, $\mathcal{R}$ computes the pair of points $\boldsymbol{v}_i \,\|\, \boldsymbol{v}'_i$ defined coordinatewise for each $j \in [d]$ as $v_{i,j} \,\|\, v'_{i,j} = u_{i,j} \oplus u_{\mathcal{R},i,j}$. If all $h(v_{i,j}) = v'_{i,j}$, then $\widetilde{\mathsf{Z}} \leftarrow \widetilde{\mathsf{Z}} \cup \{\boldsymbol{v}_i\}$.

(4) $\mathcal{R}$ outputs $\mathsf{Z} = Y \cup \widetilde{\mathsf{Z}}$, and $\mathcal{S}$ outputs Finished.

Figure 9: Fuzzy enhanced private set union protocol in Hamming space from FOnMCRG: $\Pi_{\text{fuzzy-ePSU}}^{\mathrm{dist_H}(\cdot,\cdot)}$

Parameter: The functionality interacts with the sender $\mathcal{S}$, the receiver $\mathcal{R}$, and the simulator *Sim*. Space dimension d. Distance function $\mathrm{dist}(\cdot,\cdot)$. Distance threshold δ.

Functionality:

(1) Initialize an ideal state $\mathrm{state}_{\mathcal{U}} = \emptyset$ for each party $\mathcal{U} \in \{\mathcal{S}, \mathcal{R}\}$. If $\mathcal{U}$ is corrupt, *Sim* is allowed to access $\mathrm{state}_{\mathcal{U}}$.

(2) Wait for inputs $X = \{\boldsymbol{x}_i\}_{i\in[m]} \subset \mathcal{D}$ and permutation π over $[m]$ from $\mathcal{S}$. Update $\mathrm{state}_{\mathcal{S}} = \langle X, \pi\rangle$, and send $\langle \mathrm{Request}, \mathcal{S}\rangle$ to *Sim*.

(3) Wait for inputs $\{Y_i\}_{i\in[m]}$, where $Y_i \subset \mathcal{D}$, from $\mathcal{R}$. Update $\mathrm{state}_{\mathcal{R}} = \langle\{Y_i\}_{i\in[m]}\rangle$, and send $\langle \mathrm{Request}, \mathcal{R}\rangle$ to *Sim*.

(4) Upon receiving $\langle \mathrm{Request}, \mathrm{OK}\rangle$ from *Sim*, generate two random point sets $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ and $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$, such that for each $i \in [m]$:
- If no point $Y_i[j] \in Y_i$ satisfies $\mathrm{dist}(\boldsymbol{x}_i, Y_i[j]) \leq \delta$, then $\boldsymbol{u}_{\mathcal{S},i} = \boldsymbol{u}_{\mathcal{R},i}$; otherwise, $\boldsymbol{u}_{\mathcal{S},i} \neq \boldsymbol{u}_{\mathcal{R},i}$.

(5) Update internal states: $\mathrm{state}_{\mathcal{S}} \leftarrow \mathrm{state}_{\mathcal{S}} \cup \langle\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}\rangle$ and $\mathrm{state}_{\mathcal{R}} \leftarrow \mathrm{state}_{\mathcal{R}} \cup \langle\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}\rangle$.

(6) Output $\langle\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}\rangle$ to $\mathcal{S}$ and $\langle\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}\rangle$ to $\mathcal{R}$.

Figure 10: Fuzzy permuted non-membership conditional randomness generation functionality: $\mathcal{F}_{\mathsf{FpnMCRG}}$

and appends a authentication tag by computing $u_{i,j} = u_{\mathcal{S},i,j} \oplus \big(x_{i,j} \,\|\, h(x_{i,j})\big)$, where $j \in [d]$, and sends $\boldsymbol{u}_i$ to the receiver. The receiver computes $(v_{i,j} \,\|\, v'_{i,j}) = u_{i,j} \oplus u_{\mathcal{R},i,j}$, and checks whether $h(v_{i,j}) = v'_{i,j}$ for all $j \in [d]$. If all checks succeed, $\boldsymbol{v}_i$ is accepted as a valid point and added to $\widetilde{Z}$; otherwise, $\boldsymbol{v}_i$ is discarded. By construction, $h(v_{i,j}) = v'_{i,j}$ holds for all $j \in [d]$ if and only if $\boldsymbol{u}_{\mathcal{S},i} = \boldsymbol{u}_{\mathcal{R},i}$. Otherwise, $\boldsymbol{u}_{\mathcal{S},i} \neq \boldsymbol{u}_{\mathcal{R},i}$ causing every decrypted coordinate to be uniformly random and the verification to fail. Finally, the receiver outputs $Z = Y \cup \widetilde{Z}$, while the sender outputs $\langle\mathsf{Finished}\rangle$.

We prove the correctness and security of $\Pi_{\text{fuzzy-ePSU}}^{\mathrm{dist_H}(\cdot,\cdot)}$ in Appendix D.3. The complexity analysis is shown in Appendix C.5.

6 Our Fuzzy ePSU Protocol in Minkowski Space

In this section, we define the FpnMCRG functionality and present its concrete instantiations for the ∞-norm and the $[1,\infty)$-norm Minkowski spaces. We then construct a fuzzy ePSU protocol in the Minkowski space by combining FpnMCRG with SAS-Fmap.

6.1 Fuzzy Permuted Non-Membership Conditional Randomness Generation Functionality

We introduce a new cryptographic primitive, FpnMCRG. The sender inputs a point set $X = \{\boldsymbol{x}_i\}_{i\in[m]}$ together with a permutation π over $[m]$, while the receiver inputs m point sets $\{Y_i\}_{i\in[m]}$. Given a distance function $\mathrm{dist}(\cdot,\cdot)$ and a threshold δ, the parties obtain the output point sets $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ and $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$, respectively, such that for each $i \in [m]$, $\boldsymbol{u}_{\mathcal{S},i} = \boldsymbol{u}_{\mathcal{R},i} \iff \forall\, \boldsymbol{y}_j \in Y : \mathrm{dist}(\boldsymbol{x}_{\pi^{-1}(i)}, \boldsymbol{y}_j) > \delta$. We formally define the ideal functionality of FpnMCRG, denoted by $\mathcal{F}_{\mathsf{FpnMCRG}}$, in Figure 10.

6.2 Our FpnMCRG Protcol in ∞-norm Minkowski Space

We design our FpnMCRG protocol in the ∞-norm Minkowski space, as shown in Figure 11. Specifically, the parties jointly evaluate the ∞-norm proximity predicate between each sender point $\boldsymbol{x}_i$ and every candidate point $Y_i[j] \in Y_i$ in a masked manner. For each triple $(i,j,k) \in [m] \times [B] \times [d]$, the parties invoke the $\binom{1}{N}$-SOT: the sender inputs $x_{i,k}$, and the receiver inputs $\boldsymbol{v}^p_{i,j,k} \in \mathbb{Z}_M^N$ defined in Eq. (1). As a result, they obtain additive secret shares $s_{i,j,k}$ and $\tilde{s}_{i,j,k}$, respectively, satisfying $s_{i,j,k} + \tilde{s}_{i,j,k} = v^p_{i,j,k,x_{i,k}}$. The sender then computes $e_{i,j} = \sum_{k=1}^{d} s_{i,j,k} + \mathsf{mask}_{i,j}$, and sends $\{e_{i,j}\}_{j\in[B]}$ to the receiver. The receiver locally reconstructs $\tilde{e}_{i,j} = \sum_{k=1}^{d} \tilde{s}_{i,j,k} + e_{i,j}$. By construction, $\tilde{e}_{i,j} = \mathsf{mask}_{i,j} + d$ if and only if $\mathrm{dist}_\infty(\boldsymbol{x}_i, Y_i[j]) \leq \delta$, and $\tilde{e}_{i,j} < \mathsf{mask}_{i,j} + d$ otherwise.

Following, the parties determine whether any candidate point in Y_i satisfies the distance constraint by invoking cPSI. The sender inputs the masked target set $\mathsf{MASK}_i = \{\mathsf{mask}_{i,j} + d\}_{j\in[B]}$, and the receiver inputs the reconstructed set $\tilde{E}_i = \{\tilde{e}_{i,j}\}_{j\in[B]}$. The cPSI returns only secret-shared membership bit vectors $(e_{\mathcal{S},i}, e_{\mathcal{R},i})$. Each bit vector $e_{\mathcal{S},i}$ and $e_{\mathcal{R},i}$ is hashed and expanded into a d-dimensional point, namely $\tilde{\boldsymbol{e}}_{\mathcal{S},i} = \boldsymbol{H}(\boldsymbol{e}_{\mathcal{S},i})^d$ and $\tilde{\boldsymbol{e}}_{\mathcal{R},i} = \boldsymbol{H}(\boldsymbol{e}_{\mathcal{R},i})^d$. The parties then invoke pECRG with permutation π, obtaining outputs $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ and $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$ such that $\boldsymbol{u}_{\mathcal{S},\pi(i)} = \boldsymbol{u}_{\mathcal{R},\pi(i)} \iff e_{\mathcal{S},i} = e_{\mathcal{R},i}$. Finally, the parties output the resulting permuted correlated randomness.

We prove the correctness and security of $\Pi_{\mathsf{FpnMCRG}}^{\mathrm{dist}_\infty(\cdot,\cdot)}$ in Appendix D.4. The complexity analysis is shown in Appendix C.6.

Parameter setting: The sender $\mathcal{S}$ holds $X = \{\boldsymbol{x}_i\}_{i\in[m]} \subset \mathcal{D}$ and a permutation π over $[m]$. The receiver $\mathcal{R}$ holds $\{Y_i\}_{i\in[m]} \subset \mathcal{D}$ with $|Y_i| = B$ for each $i \in [m]$. Space dimension d. Distance threshold δ. $N = 2^\ell$, $p = \infty$, $M = d + 1$. A collusion resistant hash function $H : \{0,1\}^* \to \{0,1\}^{\ell_2}$, where $\ell_2 = \lambda + 2\log m$.

Protocol:

(1) For each $i \in [m]$, each $j \in [B]$, and each $k \in [d]$, $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\mathsf{SOT}}$:
- $\mathcal{S}$ acts as sender with input $x_{i,k}$, and $\mathcal{R}$ acts as receiver with inputs $\mathbf{0}^d$ and $\boldsymbol{v}^p_{i,j,k} \in \mathbb{Z}_M^N$ (Eq. (1)). As a result, $\mathcal{S}$ obtains secret share $s_{i,j,k}$ and $\mathcal{R}$ obtains secret share $\widetilde{s}_{i,j,k}$ such that $s_{i,j,k} + \widetilde{s}_{i,j,k} = v^p_{i,j,k,x_{i,k}}$.

$$v^p_{i,j,k,q} = \begin{cases} 1, & \text{if } q \in [\, Y_i[j][k] - \delta,\ Y_i[j][k] + \delta\,], \\ 0, & \text{otherwise.} \end{cases} \quad \text{for each } q \in [N] \tag{1}$$

(2) For each $i \in [m]$ and $j \in [B]$, $\mathcal{S}$ computes $e_{i,j} = \sum_{k=1}^d s_{i,j,k} + \mathsf{mask}_{i,j}$ and $\widetilde{\mathsf{mask}}_{i,j} = \mathsf{mask}_{i,j} + d$, where $\mathsf{mask}_{i,j}$ is sampled uniformly at random. $\mathcal{S}$ sends $\{e_{i,1}, \ldots, e_{i,B}\}_{i\in[m]}$ to $\mathcal{R}$.

(3) For each $i \in [m]$ and $j \in [B]$, $\mathcal{R}$ computes $\widetilde{e}_{i,j} = \sum_{k=1}^d \widetilde{s}_{i,j,k} + e_{i,j}$.

(4) For each $i \in [m]$, $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\mathsf{cPSI}}$:
- $\mathcal{S}$ acts as receiver with input $\widetilde{\mathsf{MASK}}_i = \{\widetilde{\mathsf{mask}}_{i,j}\}_{j\in[B]}$, and $\mathcal{R}$ acts as sender with input $\widetilde{\mathsf{E}}_i = \{\widetilde{e}_{i,j}\}_{j\in[B]}$. As a result, $\mathcal{S}$ obtains $e_{\mathcal{S},i} \in \{0,1\}^{B'}$, and $\mathcal{R}$ obtains $e_{\mathcal{R},i} \in \{0,1\}^{B'}$ and a mapping $\mathsf{idx} : \widetilde{\mathsf{E}}_i \to [B']$, such that for each $j \in [B']$, if $\widetilde{e}_{i,*} \in \widetilde{\mathsf{MASK}}_i$ and $\mathsf{idx}(\widetilde{e}_{i,*}) = j$, then $e_{\mathcal{R},i,j} \oplus e_{\mathcal{S},i,j} = 1$; otherwise, $e_{\mathcal{R},i,j} \oplus e_{\mathcal{S},i,j} = 0$.

(5) For each $i \in [m]$, $\mathcal{S}$ sets $\widetilde{\boldsymbol{e}}_{\mathcal{S},i} = \boldsymbol{H}(\boldsymbol{e}_{\mathcal{S},i})^d$ and $\mathcal{R}$ sets $\widetilde{\boldsymbol{e}}_{\mathcal{R},i} = \boldsymbol{H}(\boldsymbol{e}_{\mathcal{R},i})^d$.

(6) For each $i \in [d]$, $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\mathsf{pECRG}}$:
- $\mathcal{S}$ acts as sender with input $\{\widetilde{e}_{\mathcal{S},1,i}, \ldots \widetilde{e}_{\mathcal{S},m,i}\}$ and π, and $\mathcal{R}$ acts as receiver with input $\{\widetilde{e}_{\mathcal{R},1,i}, \ldots, \widetilde{e}_{\mathcal{R},m,i}\}$. As a result, $\mathcal{S}$ obtains $\{u_{\mathcal{S},1,i}, \ldots, u_{\mathcal{S},m,i}\}$ and $\mathcal{R}$ obtains $\{u_{\mathcal{R},1,i}, \ldots, u_{\mathcal{R},m,i}\}$ satisfying that if $e_{\mathcal{S},\pi^{-1}(j),i} = e_{\mathcal{R},\pi^{-1}(j),i}$, $u_{\mathcal{S},j,i} = u_{\mathcal{R},j,i}$; otherwise $u_{\mathcal{S},j,i} \neq u_{\mathcal{R},j,i}$.

(7) $\mathcal{S}$ outputs $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ and $\mathcal{R}$ outputs $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$.

Figure 11: Fuzzy permuted non-membership conditional randomness generation protocol for ∞-norm Minkowski distance: $\Pi^{\mathsf{dist}_\infty(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$

6.3 Our FpnMCRG Protocol in $[1,\infty)$-norm Minkowski Space

We design our FpnMCRG protocol for Minkowski distance $p \in [1,\infty)$, as shown in 12. Specifically, for each sender point $\boldsymbol{x}_i$ and each candidate set $Y_i = \{Y_i[j]\}_{j\in[B]}$, the parties first privately evaluate the p-norm proximity predicate against every candidate point. They jointly invoke the $\binom{1}{N}$-SOT, where the sender inputs $x_{i,k}$, and the receiver inputs $v^p_{i,j,k}$ defined in Eq. (2). As a result, they obtain additive shares $s_{i,j,k}$ and $\tilde{s}_{i,j,k}$, respectively, satisfying $s_{i,j,k} + \tilde{s}_{i,j,k} = v^p_{i,j,k,x_{i,k}}$. The sender computes $e_{i,j} = \sum_{k=1}^d s_{i,j,k} + \mathsf{mask}_{i,j}$, and sends $\{e_{i,j}\}_{j\in[B]}$ to the receiver. The receiver reconstructs $\tilde{e}_{i,j} = \sum_{k=1}^d \tilde{s}_{i,j,k} + e_{i,j}$. By construction, $\tilde{e}_{i,j} - \mathsf{mask}_{i,j} \le \delta^p$ if and only if $\mathsf{dist}_p(\boldsymbol{x}_i, Y_i[j]) \le \delta$, and $\tilde{e}_{i,j} - \mathsf{mask}_{i,j} > \delta^p$ otherwise.

Following, the parties apply the DA-set augmentation algorithm with threshold δ^p. The sender (Alice) inputs $\{\mathsf{mask}_{i,j}\}_{j\in[B]}$ and obtains an augmented prefix set $\mathsf{PRE}_{\mathsf{MASK},i}$, while the receiver (Bob) inputs $\{\tilde{e}_{i,j}\}_{j\in[B]}$ and obtains $\mathsf{PRE}_{\tilde{E},i}$. This transformation guarantees that $\exists\, j \in [B] :\ |\tilde{e}_{i,j} - \mathsf{mask}_{i,j}| \le \delta^p \iff \mathsf{PRE}_{\mathsf{MASK},i} \cap \mathsf{PRE}_{\tilde{E},i} \neq \emptyset$. Then, the parties invoke cPSI with inputs $\mathsf{PRE}_{\mathsf{MASK},i}$ and $\mathsf{PRE}_{\tilde{E},i}$, obtaining secret-shared bit vectors $e_{\mathcal{S},i}$ and $e_{\mathcal{R},i}$, respectively. The parties then hash and expand the secret-shared bit vectors into d-dimensional points $\tilde{\boldsymbol{e}}_{\mathcal{S},i} = \boldsymbol{H}(\boldsymbol{e}_{\mathcal{S},i})^d$ and $\tilde{\boldsymbol{e}}_{\mathcal{R},i} = \boldsymbol{H}(\boldsymbol{e}_{\mathcal{R},i})^d$, and invoke pECRG with permutation π. The output points $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ and $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$ satisfy $\boldsymbol{u}_{\mathcal{S},\pi(i)} = \boldsymbol{u}_{\mathcal{R},\pi(i)} \iff e_{\mathcal{S},i} = e_{\mathcal{R},i}$. Finally, the parties output the resulting permuted correlated randomness.

We prove the correctness and security of $\Pi^{\mathsf{dist}_{p\in[1,\infty)}(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ in Appendix D.5. The complexity analysis is shown in Appendix C.7.

6.4 Our Fuzzy ePSU Protocol for Minkowski Distance

We present a fuzzy ePSU protocol in the Minkowski space built from SAS-Fmap and FpnMCRG. By instantiating FpnMCRG for both $\mathsf{dist}_\infty(\cdot,\cdot)$ and $\mathsf{dist}_{p\in[1,\infty)}(\cdot,\cdot)$, the framework supports fuzzy ePSU under a general Minkowski distance $\mathsf{dist}_p(\cdot,\cdot)$. The overall protocol structure is shown in Figure 13.

Specifically, the sender and the receiver invoke the SAS-Fmap on inputs X and Y, respectively, and obtain the corresponding identifier sets. The resulting identifiers provide a coarse locality guarantee: for any $\boldsymbol{x} \in X$ and $\boldsymbol{y} \in Y$, $\mathsf{dist}_\infty(\boldsymbol{x},\boldsymbol{y}) \le \delta \implies \mathsf{id}(\boldsymbol{x}) = \mathsf{id}(\boldsymbol{y})$, while collisions may still occur when $\mathsf{dist}_\infty(\boldsymbol{x},\boldsymbol{y}) > \delta$. Using these identifiers as keys, the sender inserts its points into a cuckoo hash table $C_{\mathcal{S}} = \{C_{\mathcal{S}}[i]\}_{i\in[\beta]}$, and the receiver inserts its points into a simple hash table, yielding for each bin $i \in [\beta]$ a candidate set Y_i of bounded size B. By construction, all δ-neighbors of a sender point (if any exist) are contained in the corresponding candidate set.

The parties then invoke the FpnMCRG, where the sender inputs $C_{\mathcal{S}}$ together with a permutation π over $[\beta]$, and the receiver inputs $\{Y_i\}_{i\in[\beta]}$. As a result, they obtain correlated randomness $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[\beta]}$ and $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[\beta]}$, respectively. The outputs satisfy that $\boldsymbol{u}_{\mathcal{S},i} = \boldsymbol{u}_{\mathcal{R},i}$ if and only if $C_{\mathcal{S}}[\pi^{-1}(i)]$ has no δ-neighbor in the

Parameter setting: The sender $\mathcal{S}$ holds $X = \{\boldsymbol{x}_i\}_{i\in[m]} \subset \mathcal{D}$ and a permutation π over $[m]$. The receiver $\mathcal{R}$ holds $\{Y_i\}_{i\in[m]} \subset \mathcal{D}$ with $|Y_i| = B$ for each $i \in [m]$. Space dimension d. Distance threshold δ. $N = 2^\ell$, $p \in [1,\infty)$, $M = d(\delta^p+1)+1$. A collusion resistant hash function $H : \{0,1\}^* \to \{0,1\}^{\ell_2}$, where $\ell_2 = \lambda + 2\log m$.

Protocol:

(1) For each $i \in [m]$, each $j \in [B]$, and each $k \in [d]$, the parties invoke $\mathcal{F}_{\mathsf{SOT}}$:
- $\mathcal{S}$ acts as sender with input $x_{i,k}$, and $\mathcal{R}$ acts as receiver with inputs $\mathbf{0}^d$ and $\boldsymbol{v}^p_{i,j,k} \in \mathbb{Z}_M^N$ (Eq (2)). As a result, $\mathcal{S}$ receives $s_{i,j,k}$ and $\mathcal{R}$ receives $\widetilde{s}_{i,j,k}$, satisfying $s_{i,j,k} + \widetilde{s}_{i,j,k} = v^p_{i,j,k,x_{i,k}}$.

$$v^p_{i,j,k,q} = \begin{cases} |q - Y_i[j][k]|^p, & \text{if } |q - Y_i[j][k]| \le \delta, \\ \delta^p + 1, & \text{otherwise.} \end{cases} \quad \text{for each } q \in [N] \tag{2}$$

(2) For each $i \in [m]$ and $j \in [B]$, $\mathcal{S}$ computes $e_{i,j} = \sum_{k=1}^d s_{i,j,k} + \mathsf{mask}_{i,j}$, where $\mathsf{mask}_{i,j}$ is chosen uniformly at random, and sends $\{e_{i,1},\dots,e_{i,B}\}$ to $\mathcal{R}$.
(3) For each $i \in [m]$ and $j \in [B]$, $\mathcal{R}$ computes $\widetilde{e}_{i,j} = \sum_{k=1}^d \widetilde{s}_{i,j,k} + e_{i,j}$.
(4) For each $i \in [m]$, the parties invoke $\mathsf{Algorithm}_{\mathsf{DA-SA}}$ with threshold δ^p:
- $\mathcal{S}$ (Alice) inputs $\{\mathsf{mask}_{i,1},\dots,\mathsf{mask}_{i,B}\}$ and obtains an augmented set $\mathsf{PRE}_{\mathsf{MASK},i}$ of size $\le 2B(\lfloor\log(2\delta^p-1)\rfloor+1)$. As a result, $\mathcal{R}$ (Bob) inputs $\{\widetilde{e}_{i,1},\dots,\widetilde{e}_{i,B}\}$ and obtains an augmented set $\mathsf{PRE}_{\widetilde{E},i}$ of size $B(\lfloor\log(2\delta^p-1)\rfloor+2)$.
(5) For each $i \in [m]$, the parties invoke $\mathcal{F}_{\mathsf{cPSI}}$:
- $\mathcal{S}$ acts as receiver with input $\mathsf{PRE}_{\mathsf{MASK},i}$, and $\mathcal{R}$ acts as sender with input $\mathsf{PRE}_{\widetilde{E},i}$. As a result, $\mathcal{S}$ obtains $e_{\mathcal{S},i} \in \{0,1\}^{B'}$, and $\mathcal{R}$ obtains $e_{\mathcal{R},i} \in \{0,1\}^{B'}$ and a mapping $\mathsf{idx} : \mathsf{PRE}_{\widetilde{E},i} \to [B']$, such that for each $j \in [B']$, if $\mathsf{PRE}_{\widetilde{E},i}[*] \in \mathsf{PRE}_{\mathsf{MASK},i}$ and $\mathsf{idx}(\mathsf{PRE}_{\widetilde{E},i}[*]) = j$, then $e_{\mathcal{R},i,j} \oplus e_{\mathcal{S},i,j} = 1$; otherwise, $e_{\mathcal{R},i,j} \oplus e_{\mathcal{S},i,j} = 0$.
(6) For each $i \in [m]$, $\mathcal{S}$ sets $\widetilde{\boldsymbol{e}}_{\mathcal{S},i} = \boldsymbol{H}(\boldsymbol{e}_{\mathcal{S},i})^d$ and $\mathcal{R}$ sets $\widetilde{\boldsymbol{e}}_{\mathcal{R},i} = \boldsymbol{H}(\boldsymbol{e}_{\mathcal{R},i})^d$.
(7) For each $i \in [d]$, $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}_{\mathsf{pECRG}}$:
- $\mathcal{S}$ acts as sender with input $\{\widetilde{e}_{\mathcal{S},1,i},\dots\widetilde{e}_{\mathcal{S},m,i}\}$ and π, and $\mathcal{R}$ acts as receiver with input $\{\widetilde{e}_{\mathcal{R},1,i},\dots,\widetilde{e}_{\mathcal{R},m,i}\}$. As a result, $\mathcal{S}$ obtains $\{u_{\mathcal{S},1,i},\dots,u_{\mathcal{S},m,i}\}$ and $\mathcal{R}$ obtains $\{u_{\mathcal{R},1,i},\dots,u_{\mathcal{R},m,i}\}$ satisfying that if $e_{\mathcal{S},\pi^{-1}(j),i} = e_{\mathcal{R},\pi^{-1}(j),i}$, $u_{\mathcal{S},j,i} = u_{\mathcal{R},j,i}$; otherwise $u_{\mathcal{S},j,i} \ne u_{\mathcal{R},j,i}$.
(8) $\mathcal{S}$ outputs $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ and $\mathcal{R}$ outputs $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$.

Figure 12: Fuzzy permuted non-membership conditional randomness generation protocol for Minkowski distance $p \in [1,\infty)$: $\Pi^{\mathsf{dist}_{p\in[1,\infty)}(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$

corresponding candidate set $Y_{\pi^{-1}(i)}$; otherwise, the two randomness points are distinct. Next, the sender encrypts each (permuted) bin content using the correlated randomness as a one-time pad. For each $i \in [\beta]$ and each $j \in [d]$, the sender computes $u_{i,j} = u_{\mathcal{S},i,j} \oplus \big(C_{\mathcal{S}}[\pi^{-1}(i)][j] \parallel h(C_{\mathcal{S}}[\pi^{-1}(i)][j])\big)$, and sends $\{\boldsymbol{u}_i\}_{i\in[\beta]}$ to the receiver. Upon receiving $\boldsymbol{u}_i$, the receiver computes $(v_{i,j} \parallel v'_{i,j}) = u_{i,j} \oplus u_{\mathcal{R},i,j}$ for each $j \in [d]$, and checks whether $h(v_{i,j}) = v'_{i,j}$ for all coordinates. If all checks succeed and $v_i \ne \perp$, the recovered point $\boldsymbol{v}_i$ is accepted and added to $\widetilde{Z}$; otherwise, $\boldsymbol{v}_i$ is discarded. By construction, the verification succeeds if and only if $\boldsymbol{u}_{\mathcal{S},i} = \boldsymbol{u}_{\mathcal{R},i}$; otherwise, the decrypted values are statistically indistinguishable from uniform and the verification fails. Finally, the receiver outputs the fuzzy union $Z = Y \cup \widetilde{Z}$, while the sender outputs $\langle\mathsf{Finished}\rangle$.

We prove the correctness and security of $\Pi^{\mathsf{dist}_p(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$ in Appendix D.6. The complexity analysis is shown in Appendix C.8.

7 Performance Evaluation

In this section, We report experimental results for our fuzzy PSU and ePSU protocols in both the Hamming and Minkowski spaces.

Environment. We run the experiments inside a Docker container built on Ubuntu, running on a single machine with Intel Xeon Gold 5415+ CPU and 256GB RAM. We measure the time in a local network setting with network latency of 0.02 ms and bandwidth of 10 Gbps.

Instantiations. We set the computational and statistical security parameters to $\kappa = 128$ and $\lambda = 40$, respectively. Our protocols are implemented in C++. For fuzzy PSU, UniqC Fmap, and SAS-Fmap, we follow the reference implementation of Gao et al. [15, 40]. We instantiate OKVS using the RB-OKVS construction of Bienstock et al. [28] (code in [40]). SKE is realized with LowMC [41] and the implementation scheme is shown in Figure 14[2]. The nECRG and pECRG are instantiated following Tu et al. [9] (code in [42]). We implement $\binom{1}{N}$-SOT using libOTe [43], DA-set augmentation as in Chakraborti et al. [14], and cPSI using the protocol of Raghuraman et al. [32] (code in [44]). We use $\gamma = 3$ hash functions for both cuckoo hashing and simple hashing. All auxiliary primitives, including collision-resistant hashing, PRNGs, and communication channels, are provided by cryptoTools [45].

7.1 Performance in Hamming Space

We evaluate the performance of our protocols in the Hamming space. The communication cost and runtime are reported in Table 2.

[2]Briefly, encryption samples a fresh random value r and outputs $(r, \mathsf{LowMC}(\mathsf{sk}, \mathsf{r}) \oplus \mathsf{m})$. Decryption follows by XORing the second component with $\mathsf{LowMC}(sk, r)$.

Parameter setting: The sender $\mathcal{S}$ holds $X = \{\boldsymbol{x}_i\}_{i\in[m]} \subset \mathcal{D}$. The receiver $\mathcal{R}$ holds $Y = \{\boldsymbol{y}_j\}_{j\in[n]} \subset \mathcal{D}$. Space dimension d. Distance threshold δ. Distance function $\mathrm{dist}_p(\cdot,\cdot)$. Hash functions $h_1, \ldots, h_\gamma : \{0,1\}^\ell \to [\beta]$. A cuckoo hash table based on $h_1, \ldots, h_\gamma$ with $\beta = \epsilon m$ bins and stash size $s = 0$. A simple hash table based on $h_1, \ldots, h_\gamma$ with $\beta = \epsilon m$ bins and bin size B, where $B = O(\log \gamma n)$. A collusion resistant hash function $h : \{0,1\}^* \to \{0,1\}^{\ell_1}$, where $\ell_1 = \lambda + 2\log(md)$.
Protocol:

(1) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}^{\mathsf{SAS}}_{\mathsf{Fmap}}$:
- $\mathcal{S}$ acts as sender with input X, and $\mathcal{R}$ acts as receiver with input Y. As a result, $\mathcal{S}$ obtains $\{\mathrm{id}(\boldsymbol{x_i})\}_{i\in[m]}$ and $\mathcal{R}$ obtains $\{\mathrm{id}(\boldsymbol{y_j})\}_{j\in[n]}$.

(2) $\mathcal{S}$ inserts X into a cuckoo hash table $\mathsf{C}_\mathcal{S} = \{\mathsf{C}_\mathcal{S}[i]\}_{i\in[\beta]}$, padding empty bins with dummy point $\perp$. Each item $\boldsymbol{x_i}$ is placed s.t. there exists $j \in [\gamma]$ satisfying $\mathsf{C}_\mathcal{S}[h_j(\mathrm{id}_{\boldsymbol{x_i}})] = \boldsymbol{x_i}$.

(3) $\mathcal{R}$ inserts Y into a simple hash table: each $\boldsymbol{y} \in Y$ is placed in bins $Y_{h_1(\mathrm{id}_{\boldsymbol{y}})}, \ldots, Y_{h_\gamma(\mathrm{id}_{\boldsymbol{y}})}$. Each bin Y_i is padded with $\perp$ to reach size B.

(4) $\mathcal{S}$ and $\mathcal{R}$ invoke $\mathcal{F}^{\mathrm{dist}_p(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$:
- $\mathcal{S}$ chooses a random permutation π over $[\beta]$, and acts as sender with inputs π and $\mathsf{C}_\mathcal{S}$. $\mathcal{R}$ acts as receiver with input $\{Y_1, \ldots, Y_\beta\}$. As a result, $\mathcal{S}$ obtains $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[\beta]}$ and $\mathcal{R}$ obtains $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[\beta]}$.

(5) For each $i \in [\beta]$, $\mathcal{S}$ computes $\boldsymbol{u}_i$ by setting, for each $j \in [d]$: $u_{i,j} = u_{\mathcal{S},i,j} \oplus (\mathsf{C}_\mathcal{S}[\pi^{-1}(i)][j] \,||\, h(\mathsf{C}_\mathcal{S}[\pi^{-1}(i)][j]))$. $\mathcal{S}$ sends $\{\boldsymbol{u}_i\}_{i\in[\beta]}$ to $\mathcal{R}$.

(6) $\mathcal{R}$ initializes $\mathsf{Z} \leftarrow \emptyset$ and $\widetilde{Z} \leftarrow \emptyset$. For each $i \in [\beta]$, it computes $\boldsymbol{v}_i || \boldsymbol{v}'_i$ where $v_{i,j} || v'_{i,j} = u_{i,j} \oplus u_{\mathcal{R},i,j}$ for all $j \in [d]$. If $\boldsymbol{v}_i \neq \perp$ and $h(v_{i,1}) = v'_{i,1}, \ldots, h(v_{i,d}) = v'_{i,d}$, then $\widetilde{Z} \leftarrow \widetilde{Z} \cup \{\boldsymbol{v}_i\}$.

(7) $\mathcal{R}$ outputs $\mathsf{Z} = Y \cup \widetilde{Z}$, and $\mathcal{S}$ outputs Finished.

Figure 13: Fuzzy enhanced private set union protocol in Minkowski space from SAS-Fmap and FpnMCRG: $\Pi^{\mathrm{dist}_p(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$

Following the experimental setup of [15], we set the input set sizes to $m = n \in \{256, 1024, 4096\}$, the domain to $\mathbb{Z}_2^{128}$, and the distance threshold to $\delta = 4$. In addition, UniqC Fmap packs $P = 24$ bits into a single super component.

Table 2: Communication cost (MB) and runtime (s) of fuzzy PSU and fuzzy ePSU in Hamming space.

$m = n$	Protocol	Comm.	Time
2^8	$\Pi^{\mathrm{dist_H}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	91.889	112.560
	$\Pi^{\mathrm{dist_H}(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	5.834	9.600
2^{10}	$\Pi^{\mathrm{dist_H}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	367.530	452.747
	$\Pi^{\mathrm{dist_H}(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	20.671	22.426
2^{12}	$\Pi^{\mathrm{dist_H}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	1470.084	1835.505
	$\Pi^{\mathrm{dist_H}(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	79.990	70.686

When $m = n = 2^8$, fuzzy PSU incurs 91.889 MB of communication and 112.560 s of runtime. In contrast, fuzzy ePSU reduces the communication to 5.834 MB and shortens the runtime to 9.600 s. When the input size increases to $m = n = 2^{12}$, the communication of fuzzy PSU rises to 1470.084 MB with a runtime of 1835.505 s, whereas fuzzy ePSU incurs 79.990 MB of communication and completes in 70.686 s. Across all evaluated input sizes, fuzzy ePSU consistently outperforms fuzzy PSU in both communication and computation, achieving a 15.8–18.4× communication shrinking, and achieving a 11.7–26.0× speedup. Thus, fuzzy ePSU eliminates the during-execution leakage inherent in fuzzy PSU, while achieving significant performance gains.

Table 3: Communication cost (MB) and runtime (s) of fuzzy PSU and fuzzy ePSU in Minkowski spaces under $p \in \{\infty, 1, 2\}$.

$m = n$	Protocol	(d,δ) (2,10) Comm.	(2,10) Time	(10,10) Comm.	(10,10) Time	(2,30) Comm.	(2,30) Time	(10,30) Comm.	(10,30) Time
$p = \infty$									
2^8	$\Pi^{\mathrm{dist}_\infty(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	7.518	1.892	36.750	8.713	21.440	4.803	106.38	20.822
	$\Pi^{\mathrm{dist}_\infty(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	2.637	2.277	10.839	9.957	5.387	4.841	24.590	21.529
2^{12}	$\Pi^{\mathrm{dist}_\infty(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	120.200	31.317	588.000	136.442	343.000	77.381	1702.000	358.754
	$\Pi^{\mathrm{dist}_\infty(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	36.8.00	36.574	168.137	147.842	80.800	75.900	388.137	347.082
$p = 1$									
2^8	$\Pi^{\mathrm{dist}_{p=1}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	7.502	2.380	36.380	9.012	21.280	5.245	105.200	21.534
	$\Pi^{\mathrm{dist}_{p=1}(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	10.714	6.012	18.706	13.128	15.359	9.094	34.350	25.987
2^{12}	$\Pi^{\mathrm{dist}_{p=1}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	120.000	32.417	589.200	139.790	340.300	72.916	1703.000	357.132
	$\Pi^{\mathrm{dist}_{p=1}(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	172.771	91.673	300.459	223.703	244.379	146.171	548.067	424.248
$p = 2$									
2^8	$\Pi^{\mathrm{dist}_{p=2}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	7.588	2.464	36.910	9.264	21.420	5.264	106.600	21.736
	$\Pi^{\mathrm{dist}_{p=2}(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	15.772	8.191	23.764	14.961	23.064	12.923	34.350	24.582
2^{12}	$\Pi^{\mathrm{dist}_{p=2}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	122.800	34.128	590.600	142.707	346.800	76.789	1706.000	360.595
	$\Pi^{\mathrm{dist}_{p=2}(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	257.643	135.884	385.331	263.151	385.200	218.342	688.889	483.328

7.2 Performance in the Minkowski Space

We evaluate our protocols under Minkowski distances with $p \in \{1, 2, \infty\}$. Following [15], our experiments are conducted on inputs with $m = n \in \{2^4, 2^8, 2^{12}, 2^{16}\}$, distance thresholds $\delta \in \{10, 30\}$, and dimensions ranging from low ($d = 2$) to high ($d \in \{6, 10\}$).

Table 3 reports the results for both low-dimensional ($d = 2$) and high-dimensional ($d = 10$) settings, with set sizes $m = n \in \{2^8, 2^{12}\}$.

For $p = \infty$, fuzzy ePSU achieves a 2.9–4.4× shringking in communication compared to fuzzy PSU, while maintaining comparable runtime. For $p \in \{1, 2\}$, fuzzy PSU demonstrates better performance in low-dimensional settings ($d = 2$) with small threshold ($\delta = 10$), achieving a 1.4–2.1× shrinking in communication and a 2.5–4.0× speedup in runtime compared to fuzzy ePSU. In contrast, in high-dimensional setting ($d = 10$), compared to fuzzy PSU, fuzzy ePSU incurs a 1.1–1.8× slowdown in runtime but achieves a 1.5–3.1× communication shrinking. Thus, fuzzy ePSU eliminates the during-execution leakage in fuzzy PSU with a moderate efficiency trade-off.

The complete evaluation and detailed analysis under the above parameter settings are provided in Appendix E.

References

[1] Lenstra, A., Voss, T.: Information Security Risk Assessment, Aggregation, and Mitigation. In: *ACISP 2004*. LNCS, vol 3108, pp. 391-401. Springer, Berlin, Heidelberg (2004).
[2] Hogan, K., Luther, N., Schear, N., Shen, E., Stott, D., Yakoubov, S., Yerukhimovich, A.: Secure multiparty computation for cooperative cyber risk assessment. In: *SecDev*, pp. 75-76. IEEE (2016).
[3] Kolesnikov, V., Rosulek, M., Trieu, N., Wang, X.: Scalable Private Set Union from Symmetric-Key Techniques. In:*ASIACRYPT 2019*. LNCS, vol 11922, pp. 636-666. Springer, Cham (2019).
[4] Garimella, G., Mohassel, P., Rosulek, M., Sadeghian, S., Singh, J.: Private Set Operations from Oblivious Switching. In: *PKC 2021*. LNCS, vol 12711, pp. 591-617. Springer, Cham (2021).
[5] Burkhart, M., Strasser, M., Many, D., Dimitropoulos, X.: SEPIA:Privacy-Preserving aggregation of Multi-Domain network events and statistics. In: *USENIX Security 2010*. USENIX Association (2010).
[6] Jia, Y., Sun, S. F., Zhou, H. S., Du, J., Gu, D.: Shuffle-based private set union: Faster and more secure. In: *USENIX Security 2022*, pp. 2947-2964. USENIX Association (2022).
[7] Zhang, C., Chen, Y., Liu, W., Zhang, M., Lin, D.: Linear private set union from Multi-Query reverse private membership test. In: *USENIX Security 2023*, pp. 337-354. USENIX Association (2023).
[8] Jia, Y., Sun, S. F., Zhou, H. S., Gu, D.: Scalable private set union, with stronger security. In: *USENIX Security 2024*, pp. 7471-6488. USENIX Association (2024).
[9] Tu, B., Bai, Y., Zhang, C., Cao, Y., Chen, Y.: Fast Enhanced Private Set Union in the Balanced and Unbalanced Scenarios. In: *USENIX Security 2025*. USENIX Association (2025).
[10] Tu, B., Chen, Y., Liu, Q., Zhang, C.: Fast unbalanced private set union from fully homomorphic encryption. In: *ACM CCS 2023*, Copenhagen, Denmark, 26-30 November 2023, pp. 2959-2973. ACM Press (2023).
[11] Zhang, C., Chen, Y., Liu, W., Peng, L., Hao, M., Wang, A., Wang, X.: Unbalanced private set union with reduced computation and communication. In: *ACM CCS 2024*, Salt Lake City, UT, USA, 14-18 October 2024, pp. 1434-1447. ACM Press (2024).
[12] Ye, Q., Steinfeld, R., Pieprzyk, J., Wang, H.: Efficient fuzzy matching and intersection on private datasets. In: *ICISC 2009*. pp. 211–228. Springer Berlin Heidelberg, Berlin, Heidelberg (2010).
[13] Uzun, E., Chung, S.P., Kolesnikov, V., Boldyreva, A., Lee, W.: Fuzzy labeled private set intersection with applications to private Real-Time biometric search. In: *USENIX Security 2021*, pp. 911–928. USENIX Association (2021)
[14] Chakraborti, A., Fanti, G., Reiter, M.K.: Distance-Aware private set intersection. In: *USENIX Security 2023*, pp. 319–336. USENIX Association (2023).
[15] Gao, Y., Qi, L., Liu, X., Luo, Y., Wang, L.: Efficient Fuzzy Private Set Intersection from Fuzzy Mapping. In: *ASIACRYPT 2024*. Part VI. LNCS, vol 15489, pp. 36-68. Springer, Singapore (2024).
[16] Garimella, G., Rosulek, M., Singh, J.: Structure-aware private set intersection, with applications to fuzzy matching. In: *CRYPTO 2022*. LNCS, Part I, vol. 13507, pp. 323–352. Springer, Cham (2022).
[17] van Baarsen, A., Pu, S.: Fuzzy Private Set Intersection with Large Hyperballs. In: *EUROCRYPT 2024*. LNCS, Part V, vol. 14655, pp. 340-369. Springer, Cham (2024).
[18] Garimella, G., Goff, B., Miao, P.: Computation Efficient Structure-Aware PSI from Incremental Function Secret Sharing. In: *CRYPTO 2024*. LNCS, Part VIII, vol. 14927, pp. 309-245. Springer, Cham (2024).
[19] Piske, L., Singh, J., Trieu, N., Kolesnikov, V., Zikas, V.: Distance-Aware OT with Application to Fuzzy PSI. In: *ACM CCS 2025*, Taipei Taiwan, October 13-17, 2025, pp. 4679-4691. ACM Press (2025).
[20] Dang, C., Zhou, X., Liang, B.: Efficient Fuzzy PSI Based on Prefix Representation. In: *ACM CCS 2025*, Taipei Taiwan, October 13-17, 2025, pp. 2204-2218. ACM Press (2025).
[21] van Baarsen, A., Pu, S.: Fuzzy private set intersection from VOLE. In *ASIACRYPT 2025*. LNCS, vol 16249, pp. 327–360. Springer, Singapore (2025).
[22] Wang, X. S., Huang, Y., Zhao, Y., Tang, H., Wang, X., Bu, D.: Efficient genomewide, privacy-preserving similar patient query based on private edit distance. In: *ACM CCS 2015*, Denver Colorado USA, 12-16 October 2015, pp. 492-503. ACM Press (2015).
[23] Mohammadi-Kambs, M., Hölz, K., Somoza, M. M., Ott, A.: Hamming distance as a concept in DNA molecular recognition. In: *ACS omega 2017*, vol, 2, issue 4, pp, 1302-1308 (2017).
[24] Riaz, Z., Dürr, F., Rothermel, K.: Location privacy and utility in geo-social networks: survey and research challenges. In *PST*, Belfast, Ireland, 2018, pp. 1-10. IEEE (2018).
[25] Youssef, M., Agrawala, A.: The Horus WLAN location determination system. In *MobiSys'05*, Seattle Washington, June 6-8, 2005, pp. 205-218. ACM Press (2005).
[26] Shokri, R., Theodorakopoulos, G., Troncoso, C., Hubaux, J. P., Le Boudec, J. Y.: Protecting location privacy: optimal strategy against localization attacks. In *ACM CCS 2012*, Raleigh North Carolina USA, October 16-18 2012, pp. 617-627. ACM Press (2012).
[27] Garimella, G., Pinkas, B., Rosulek, M., Trieu, N., Yanai, A.: Oblivious Key-Value Stores and Amplification for Private Set Intersection. In: *CRYPTO 2021*. LNCS, vol 12826, pp. 395–425. Springer, Cham (2021).
[28] Bienstock, A., Patel, S., Seo, J. Y., Yeo, K.: Near-Optimal Oblivious Key-Value Stores for Efficient PSI, PSU and Volume-Hiding Multi-Maps. In: *USENIX Security 2023*, pp. 301-318. USENIX Association (2023).
[29] Yao, A. C. C.: How to Generate and Exchange Secrets Extended Abstract. In: *FOCS 1986*, pp. 162-167. IEEE (1986).
[30] Goldreich, O., Micali, S., Wigderson, A.: How to Play Any Mental Game, or A Completeness Theorem for Protocols With Honest Majority. In: *STOC 1987*, pp. 218-229. ACM Press (1987).
[31] Piske, L., van de Graaf, J., Nascimento, A. C., Trieu, N.: Shared OT and Its Applications. In: *CIC 2025*.
[32] Raghuraman, S., Rindal, P.: Blazing fast PSI from improved OKVS and subfield VOLE. In: *ACM CCS 2022*. pp. 2505-2517. ACM Press (2022).
[33] Goldreich O.: Foundations of cryptography: volume 2, basic applications. Cambridge university press (2001).
[34] Rabin, M.O.: How to Exchange Secrets with Oblivious Transfer. In: Cryptology ePrint Archive (2005).
[35] Kolesnikov, V., Kumaresan, R.: Improved OT Extension for Transferring Short Secrets. In: *CRYPTO 2013*. LNCS, vol 8043, pp. 54-70. Springer, Berlin, Heidelberg (2013).
[36] Pinkas, B., Schneider, T., Segev, G., Zohner, M.: Phasing: Private set intersection using permutation-based hashing. In: *USENIX Security 2015*, pp. 515-530. USENIX Association (2015).
[37] Pinkas, B., Schneider, T., Zohner, M.: Scalable private set intersection based on OT extension. In: *ACM TOPS 2018*, vol 21, pp. 1-35. ACM Press (2018).
[38] Albrecht, M.R., Rechberger, C., Schneider, T., Tiessen, T., Zohner, M.: Ciphers for MPC and FHE. In: *EUROCRYPT 2015*. LNCS, vol 9056, pp. 430–454. Springer, Berlin, Heidelberg (2015).
[39] Beaver, D.: Efficient Multiparty Protocols Using Circuit Randomization. In: *CRYPTO 1991*. LNCS, vol 576, pp. 420–432. Springer, Berlin, Heidelberg (1991).
[40] https://github.com/ql70ql70/Fuzzy-Private-Set-Intersection-from-Fuzzy-Mapping.git
[41] https://github.com/LowMC/lowmc.git
[42] https://doi.org/10.5281/zenodo.14725816
[43] https://github.com/osu-crypto/libOTe.git
[44] https://github.com/Visa-Research/volepsi.git
[45] https://github.com/ladnir/cryptoTools.git

A Related Work

Existing work on PSU is restricted to the exact-match setting, and no prior construction addresses fuzzy matching. Moreover, due to asymmetric input sizes and computational roles, PSU protocols are commonly classified into two regimes: the balanced setting with $|X| = |Y| = n$, and the unbalanced setting with $|X| = m \ll |Y| = n$, where the sender holds X and the receiver holds Y.

Kolesnikov et al. [3] introduced the first scalable balanced PSU construction. Their design was centered around the reverse private membership test (RPMT), which enables the sender to privately determine, for each $x_i \in X$, whether it is contained in Y. The resulting indication bit is then used as the choice bit of an OT: if $x_i \in Y$, the receiver obtains a dummy value; otherwise, it learns x_i.

By aggregating all outcomes, the parties obtain the union $X \cup Y$. Several works subsequently improved this framework. Garimella et al. [4], Jia et al. [6], and Zhang et al. [7] proposed performance optimizations, and Zhang et al. achieved linear complexity.

However, Jia et al. [8] demonstrated that the RPMT followed by OT inherently suffers from during-execution leakage. To address this problem, they designed the first balanced ePSU protocol secure against during-execution leakage, although their protocol incurs superlinear complexity. More recently, Tu et al. [9] proposed an improved ePSU protocol that achieves linear complexity for the first time. Their construction is based on a new primitive called permuted non-membership conditional randomness generation (pnMCRG), complemented by the hash-to-bin. In pnMCRG, the sender inputs a set $X = \{x_i\}_{i\in[m]}$ and a permutation π over $[m]$, while the receiver inputs m sets $\{Y_i\}$. For each $i \in [m]$, if $x_i \in Y_i$, the two parties receive distinct random values; otherwise, they receive identical random values. Notably, in our construction of fuzzy ePSU in the Minkowski space, we introduce FpnMCRG, which can be viewed as a fuzzy generalization of pnMCRG.

Tu et al. [10] designed the first scalable unbalanced PSU protocol based on fully homomorphic encryption, achieving sublinear communication in the larger set size. Later, Zhang et al. [11] proposed an improved protocol that leverages sparse OKVS, which enhances the efficiency of the online phase compared to [10] while preserving sublinear communication in the larger set size. However, the setup phase in [11] incurs substantial overhead. Moreover, both protocols [10, 11] rely on the RPMT followed by OT framework, and therefore suffer from during execution leakage. Recently, Tu et al. [9] designed the first unbalanced ePSU protocol that employs pnMCRG together with the hash-to-bin, achieving sublinear communication in the larger set size while preventing during-execution leakage.

B Preliminary Details

B.1 Security Model

Let Π be a two party protocol for computing a function $f(X, Y)$, where the sender $\mathcal{S}$ holds input X and obtains output $f_{\mathcal{S}}(X, Y)$, and the receiver $\mathcal{R}$ holds input Y and obtains output $f_{\mathcal{R}}(X, Y)$. Let $\mathsf{view}^{\Pi}_{\mathcal{S}}(X, Y)$ and $\mathsf{view}^{\Pi}_{\mathcal{R}}(X, Y)$ denote the views of $\mathcal{S}$ and $\mathcal{R}$ during the execution of Π, respectively, including their inputs, random coins, and all messages received. Let $\mathsf{out}_{\Pi}(X, Y)$ denote the outputs of both parties in protocol Π.

Definition B.1 (Semi-Honest Security). A protocol Π securely computes functionality f in the semi-honest model if for every probabilistic polynomial time (PPT) adversary $\mathcal{A}$, there exist PPT simulators $\mathsf{Sim}_{\mathcal{S}}$ and $\mathsf{Sim}_{\mathcal{R}}$ such that for all inputs X and Y,

$$\big\{\mathsf{view}^{\Pi}_{\mathcal{S}}(X, Y),\ \mathsf{out}_{\Pi}(X, Y)\big\} \overset{c}{\approx} \big\{\mathsf{Sim}_{\mathcal{S}}(X,\ f_{\mathcal{S}}(X, Y)),\ f(X, Y)\big\},$$

$$\big\{\mathsf{view}^{\Pi}_{\mathcal{R}}(X, Y),\ \mathsf{out}_{\Pi}(X, Y)\big\} \overset{c}{\approx} \big\{\mathsf{Sim}_{\mathcal{R}}(Y,\ f_{\mathcal{R}}(X, Y)),\ f(X, Y)\big\}.$$

B.2 Fuzzy Mapping

Definition B.2 (Fuzzy Mapping). Π is a semi-honest secure fuzzy mapping party protocol, where the sender inputs $X = \{\boldsymbol{x}_i\}_{i\in[m]} \subset \mathcal{D}$ and obtains $\mathsf{ID}(X) = \{\mathsf{ID}(\boldsymbol{x}_i)\}_{i\in[m]}$, and the receiver inputs $Y = \{\boldsymbol{y}_j\}_{j\in[n]} \subset \mathcal{D}$ and obtains $\mathsf{ID}(Y) = \{\mathsf{ID}(\boldsymbol{y}_j)\}_{j\in[n]}$, with threshold δ under distance function $\mathsf{dist}(\cdot,\cdot)$ if and only if it satisfies the following properties.

- **Correctness.** For any $\boldsymbol{x}_i \in X$ and $\boldsymbol{y}_j \in Y$, $\mathsf{dist}(\boldsymbol{x}_i, \boldsymbol{y}_j) \le \delta \implies \mathsf{ID}(\boldsymbol{x}_i) \cap \mathsf{ID}(\boldsymbol{y}_j) \neq \emptyset$.
- **Distinctiveness.** For the receiver's output $\mathsf{ID}(Y)$, $\Pr\big[\exists\, i \neq i' \in [n]$ such that $\mathsf{ID}(\boldsymbol{y}_i) \cap \mathsf{ID}(\boldsymbol{y}_{i'}) \neq \emptyset\big] = \mathsf{negl}(\kappa)$.
- **Security.** For corrupted sender, for any $X \subset \mathcal{D}$ and any $Y, Y' \subset \mathcal{D}$, $\mathsf{view}^{\Pi}_{\mathcal{S}}(\kappa, \lambda; X, Y) \overset{c}{\approx} \mathsf{view}^{\Pi}_{\mathcal{S}}(\kappa, \lambda; X, Y')$.
  For corrupted receiver, for any $Y \subset \mathcal{D}$ and any $X, X' \subset \mathcal{D}$, $\mathsf{view}^{\Pi}_{\mathcal{R}}(\kappa, \lambda; X, Y) \overset{c}{\approx} \mathsf{view}^{\Pi}_{\mathcal{R}}(\kappa, \lambda; X', Y)$.

UniqC Fmap. The UniqC Fmap consists of two non interactive algorithms, executed independently by the sender and receiver.

Sender side UniqC Fmap(X). Given an input set $X = \{\boldsymbol{x}_i\}_{i\in[m]} \subset \mathcal{D}$, the sender computes $\mathsf{ID}(X) = \{\mathsf{ID}(\boldsymbol{x}_i)\}_{i\in[m]}$, where each $\mathsf{ID}(\boldsymbol{x}_i)$ is represented as $\{k \,\|\, x_{i,k}\}_{k\in[d]}$.

Receiver side UniqC Fmap(Y). Given a unique set $Y = \{\boldsymbol{y}_j\}_{j\in[n]} \subset \mathcal{D}$, the receiver computes $\mathsf{ID}(Y) = \{\mathsf{ID}(\boldsymbol{y}_j)\}_{j\in[n]}$, where each $\mathsf{ID}(\boldsymbol{y}_j)$ is represented as $\{u^j_k \,\|\, y_{j,u^j_k}\}_{k\in[\delta+1]}$, with indices $u^j_k \in [d]$ chosen such that $y_{j,u^j_k} \neq y_{j',u^{j'}_k}$ for all $j' \in [n] \setminus \{j\}$.

The set Y is called a unique set if it satisfies the uniqueness condition defined in Definition B.3. If the receiver's points are uniformly distributed, then by Lemma B.4, the R UniqC assumption holds with overwhelming probability in high dimensional settings.

Definition B.3 (Unique Set). For a point $\boldsymbol{y} \in Y$ in a d-dimensional space, $\boldsymbol{y}$ has a unique component y_k if and only if its component on dimension k differs from that of every other point in Y. A point $\boldsymbol{y}$ is said to be unique if and only if it has at least $\delta + 1$ unique components. A set Y is said to be unique if and only if all points in Y are unique.

Lemma B.4 (Uniform Distribution). *In a d-dimensional space, if points in set Y are uniformly distributed, then the probability that Y is unique is $1 - \mathsf{negl}(d)$, where $\mathsf{negl}(d) = d^{\delta+1}(\frac{n-1}{2^\ell})^{d-\delta}$.*

Remark 1. The uniqueness property in Lemma B.4 holds under the assumption that 2^ℓ is sufficiently large relative to n. If this condition does not hold, standard packing techniques can be applied, in which multiple components are grouped into supercomponents to ensure that the uniqueness guarantee is preserved.

SAS-Fap. Given two separated sets $X = \{\boldsymbol{x}_i\}_{i\in[m]} \subseteq \mathcal{D}$ and $Y = \{\boldsymbol{y}_j\}_{j\in[n]} \subseteq \mathcal{D}$, the protocol outputs identifiers $\mathsf{ID}(X) = \{\mathsf{id}(\boldsymbol{x}_i)\}_{i\in[m]}$ and $\mathsf{ID}(Y) = \{\mathsf{id}(\boldsymbol{y}_j)\}_{j\in[n]}$, with the property that $\mathsf{id}(\boldsymbol{x}_i) = \mathsf{id}(\boldsymbol{y}_j)$ whenever $\mathsf{dist}_\infty(\boldsymbol{x}_i, \boldsymbol{y}_j) \le \delta$. The notion of separated set is formally defined in Definition B.5, and Lemma B.6 shows that such set exists with overwhelming probability in high dimensional space. By Definition B.5, SAS-Fmap satisfies the $R \wedge S$.disj.proj. assumption, meaning that each point held by either party is guaranteed to be separated from all other points in the same set along at least one coordinate.

Definition B.5 (Separated Set). For two points x and x' in a d-dimensional space, x collides with x' on dimension k if and only if the distance between their components on dimension k is $\le 2\delta$; otherwise, x is separated from x' on dimension k. A set X is

separated if and only if, for each point in X, there exists a dimension such that this point is separated from any other points in X on it.

Lemma B.6 (Uniform Distribution). *In a d-dimensional space, if points in set X are uniformly distributed, then the probability that X is separated is $1 - \mathrm{negl}(d)$.*

B.3 Oblivious Key-Value Store

Definition B.7 (Oblivious Key-Value Store). An oblivious key-value store is parameterized by a key space $\mathcal{K}$, a value space $\mathcal{V}$, and randomness $r \in \{0,1\}^\kappa$. It consists of two algorithms:

- $\mathsf{Encode}(L, r)$: On input a set $L = \{(k_1, v_1), \ldots, (k_n, v_n)\} \subseteq \mathcal{K} \times \mathcal{V}$, where all keys are distinct, and randomness r, outputs an object $\mathsf{D} = (d_1, \ldots, d_m) \in \mathcal{V}^m$, or an error indicator $\perp$ with statistically small probability.
- $\mathsf{Decode}(\mathsf{D}, k)$: On input an object D and a key $k \in \mathcal{K}$, outputs a value $v \in \mathcal{V}$.

These algorithms satisfy the following properties:

- **Correctness.** For any set $L \subseteq (\mathcal{K} \times \mathcal{V})$ with distinct keys, for all $(k, v) \in L$, $\mathsf{Decode}(\mathsf{D}, k) = v$ if $\mathsf{D} \leftarrow \mathsf{Encode}(L, r) \neq \perp$.
- **Obliviousness.** Let $\{k_1^0, \ldots, k_n^0\}, \{k_1^1, \ldots, k_n^1\} \subseteq \mathcal{K}$ be two distinct sets of keys. If Encode does not output $\perp$ for either key set, then $\left\{\mathsf{Encode}(\{(k_1^0, v_1), \ldots, (k_n^0, v_n)\}, r) \;\middle|\; v_i \xleftarrow{\$} \mathcal{V}\right\} \overset{c}{\approx} \left\{\mathsf{Encode}(\{(k_1^1, v_1), \ldots, (k_n^1, v_n)\}, r) \;\middle|\; v_i \xleftarrow{\$} \mathcal{V}\right\}$.
- **Randomness.** For any set $A = \{(k_1, v_1), \ldots, (k_n, v_n)\}$ and any key $k^* \notin \{k_1, \ldots, k_n\}$, if $\mathsf{D} \leftarrow \mathsf{Encode}(A, r)$, then $\mathsf{Decode}(\mathsf{D}, k^*)$ is statistically indistinguishable from the uniform distribution over $\mathcal{V}$.

B.4 Symmetric Key Encryption

A SKE scheme is a tuple of four algorithms:

- $\mathsf{Setup}(1^\kappa)$: On input the security parameter κ, outputs public parameters pp, which include the description of the message space $\mathcal{M}$ and the ciphertext space C.
- $\mathsf{KeyGen}(pp)$: On input public parameters pp, outputs a secret key sk.
- $\mathsf{Enc}(sk, m)$: On input a secret key sk and a message $m \in \mathcal{M}$, outputs a ciphertext $c \in C$.
- $\mathsf{Dec}(sk, c)$: On input a secret key sk and a ciphertext $c \in C$, outputs a message $m \in \mathcal{M}$ or an error symbol $\perp$.

These algorithms satisfy the following properties:

- **Correctness.** For any $pp \leftarrow \mathsf{Setup}(1^\kappa)$, any $sk \leftarrow \mathsf{KeyGen}(pp)$, any $m \in \mathcal{M}$, and $c \leftarrow \mathsf{Enc}(sk, m)$, it holds that $\mathsf{Dec}(sk, c) = m$.
- **Security (Single Message Multi Ciphertext Pseudorandomness).** For our construction, we require a security notion tailored to a specific setting, referred to as single-message multi-ciphertext pseudorandomness. A SKE scheme provides single message multi ciphertext pseudorandomness if for every PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, its advantage in the following experiment is negligible in κ:

$$\mathrm{Adv}_{\mathcal{A}}(1^\kappa) = \Pr\left[\beta = \beta' : \begin{array}{l} pp \leftarrow \mathsf{Setup}(1^\kappa); \\ sk \leftarrow \mathsf{KeyGen}(pp); \\ (m, state) \leftarrow \mathcal{A}_1(pp); \\ \beta \xleftarrow{\$} \{0,1\}; \\ \text{for } i \in [n]: \; c_{i,0}^* \leftarrow \mathsf{Enc}(sk, m),\; c_{i,1}^* \xleftarrow{\$} C; \\ \beta' \leftarrow \mathcal{A}_2(pp, state, \{c_{i,\beta}^*\}_{i \in [n]}) \end{array}\right] - \frac{1}{2}.$$

This notion is a mild security requirement that is satisfied by most IND-CPA secure SKE schemes, including the classical pseudorandom function (PRF)-based SKE. We present a concrete PRF-based SKE scheme in Figure 12, whose security proof is provided in [7].

Parameter: PRF family: $F := \{f_k : \{0,1\}^\kappa \to \{0,1\}^\kappa\}_{\kappa \in \mathcal{K}}$.
Scheme:

(1) $\mathsf{Setup}(1^\kappa)$: On input the security parameter κ, outputs public parameters pp, which include the description of the message space $\mathcal{M}$ and the ciphertext space C.
(2) $\mathsf{KeyGen}(pp)$: On input public parameters pp, outputs a secret key $sk \xleftarrow{\$} \mathcal{K}$.
(3) $\mathsf{Enc}(sk, m)$: On input a secret key sk and a message $m \in \mathcal{M}$, chooses $r \xleftarrow{\$} \{0,1\}^\kappa$, outputs a ciphertext $c = (r, f_k(r) \oplus m)$.
(4) $\mathsf{Dec}(sk, c)$: On input a secret key sk and a ciphertext $c = (r, c')$, outputs $m = f_k(r) \oplus c'$.

Figure 14: Pseudorandom function based symmetric key encryption scheme.

B.5 Non-Equality Conditional Randomness Generation

The ideal functionality of nECRG is described in Figure 15.

B.6 Permuted Equality Conditional Randomness Generation

The ideal functionality for pECRG is given in Figure 16.

B.7 1-out-of-N Shared Oblivious Transfer

The ideal functionalities for OT and for $\binom{1}{N}$-SOT are given in Figures. 17 and 18.

B.8 Distance-aware Set Augmentation

Definition B.8. Let $A, B \subset \{0,1\}^\ell$ be finite sets of strings and let $\delta \in \mathbb{N}$ be a distance threshold. The DA-set augmentation algorithm outputs two augmented sets $\hat{A}$ and $\hat{B}$ satisfying

$$(a_i, b_j) \in A \times B,\ \mathsf{dist}(a_i, b_j) < \delta \iff \exists\, s \in \hat{A} \cap \hat{B}.$$

- **Alice: augmenting set A.** For A, DA-augmentation produces a collection of prefix patterns

$$\hat{A}_i = \left\{ a_i[t:\ell] \parallel *^{t-1} \;\middle|\; t \in [L_{A_i}] \right\},$$

where $L_{A_i} \leq 2 \cdot \left(\lfloor \log(2\delta - 1) \rfloor + 1\right)$ and the set of instantiated intervals $\bigcup_{t \in [L_{A_i}]} a_i[t:\ell] \parallel \{0,1\}^{t-1}$ exactly covers

Parameter: The functionality interacts with the sender $\mathcal{S}$, the receiver $\mathcal{R}$, and the simulator *Sim*.
Functionality:

(1) Initialize an internal state $\text{state}_{\mathcal{U}} = \emptyset$ for each party $\mathcal{U} \in \{\mathcal{S}, \mathcal{R}\}$. If $\mathcal{U}$ is corrupt, *Sim* is allowed to access $\text{state}_{\mathcal{U}}$.
(2) Wait for input $T = \{t_i\}_{i\in[n]}$ from $\mathcal{S}$. Update $\text{state}_{\mathcal{S}} \leftarrow \langle T \rangle$, and send $\langle \text{Request}, \mathcal{S} \rangle$ to *Sim*.
(3) Wait for input $\widetilde{T} = \{\widetilde{t_i}\}_{i\in[n]}$ from $\mathcal{R}$. Update $\text{state}_{\mathcal{R}} \leftarrow \langle \widetilde{T} \rangle$, and send $\langle \text{Request}, \mathcal{R} \rangle$ to *Sim*.
(4) Upon receiving $\langle \text{Request}, \text{OK} \rangle$ from *Sim*, sample $R = \{r_i\}_{i\in[n]}$ and $\widetilde{R} = \{\widetilde{r_i}\}_{i\in[n]}$, where for each $i \in [n]$:
$$\begin{cases} r_i = \widetilde{r_i}, & \text{if } t_i \neq \widetilde{t_i}, \\ r_i \neq \widetilde{r_i}, & \text{otherwise.} \end{cases}$$
(5) Update internal states: $\text{state}_{\mathcal{S}} \leftarrow \text{state}_{\mathcal{S}} \cup \langle R \rangle$, $\text{state}_{\mathcal{R}} \leftarrow \text{state}_{\mathcal{R}} \cup \langle \widetilde{R} \rangle$.
(6) Output $\langle R \rangle$ to $\mathcal{S}$ and $\langle \widetilde{R} \rangle$ to $\mathcal{R}$.

Figure 15: Non-equality conditional randomness generation functionality: $\mathcal{F}_{\text{nECRG}}$

Parameters: The functionality interacts with the sender $\mathcal{S}$, the receiver $\mathcal{R}$, and the simulator *Sim*.
Functionality:

(1) Initialize internal states $\text{state}_{\mathcal{U}} = \emptyset$ for each $\mathcal{U} \in \{\mathcal{S}, \mathcal{R}\}$. If $\mathcal{U}$ is corrupt, *Sim* is allowed to access $\text{state}_{\mathcal{U}}$.
(2) Wait for inputs $C = \{c_i\}_{i\in[n]}$ and permutation π over $[n]$ form $\mathcal{S}$. Update $\text{state}_{\mathcal{S}} \leftarrow \langle C, \pi \rangle$ and send $\langle \text{Request}, \mathcal{S} \rangle$ to *Sim*.
(3) Wait for input $\widetilde{C} = \{\widetilde{c_i}\}_{i\in[n]}$ from $\mathcal{R}$. Update $\text{state}_{\mathcal{R}} \leftarrow \langle \widetilde{C} \rangle$ and send $\langle \text{Request}, \mathcal{R} \rangle$ to *Sim*.
(4) Upon receiving $\langle \text{Request}, \text{OK} \rangle$ from *Sim*, sample $U = \{u_i\}_{i\in[n]}$ and $\widetilde{U} = \{\widetilde{u_i}\}_{i\in[n]}$, where for each $i \in [n]$:
$$\begin{cases} u_i = \widetilde{u_i}, & \text{if } c_{\pi^{-1}(i)} = \widetilde{c}_{\pi^{-1}(i)}, \\ u_i \neq \widetilde{u_i}, & \text{otherwise.} \end{cases}$$
(5) Update internal states: $\text{state}_{\mathcal{S}} \leftarrow \text{state}_{\mathcal{S}} \cup \langle U \rangle$, $\text{state}_{\mathcal{R}} \leftarrow \text{state}_{\mathcal{R}} \cup \langle \widetilde{U} \rangle$.
(6) Output $\langle U \rangle$ to $\mathcal{S}$ and $\langle \widetilde{U} \rangle$ to $\mathcal{R}$.

Figure 16: Permuted equality conditional randomness generation functionality: $\mathcal{F}_{\text{pECRG}}$

the neighborhood interval $[a_i - \delta,\ a_i + \delta]$. The resulting set $\hat{A} = \bigcup_{a_i \in A} \hat{A}_i$ forms a maximal enclosing complete prefix set. The number of prefixes satisfies $L_A \leq \sum_{i=1}^{|A|} L_{A_i}$.

- **Bob: augmenting set** B. For each $b_j \in B$, DA-augmentation produces the prefix set
$$\hat{B}_j = \left\{ b_j[t:\ell] \parallel *^{t-1} \;\middle|\; t \in [L_B] \right\},$$
where $L_B = \lfloor \log(2\delta - 1) \rfloor + 1$. The full augmented set is $\hat{B} = \bigcup_{b_j \in B} \hat{B}_j$.
- **Correctness and complexity.** The augmentation satisfies the intersection property:
$$\hat{A} \cap \hat{B} \neq \emptyset \iff \exists\, (a_i, b_j) \in A \times B : \text{dist}(a_i, b_j) < \delta.$$
Each string generates at most $O(\log \delta)$ representative prefixes, implying
$$|\hat{A}| = O(|A| \log \delta), \qquad |\hat{B}| = O(|B| \log \delta).$$

Parameters: The sender $\mathcal{S}$ and the receiver $\mathcal{R}$.
Functionality:

(1) Wait for input (m_0, m_1) from $\mathcal{S}$.
(2) Wait for input $b \in \{0, 1\}$ from $\mathcal{R}$.
(3) Output m_b to $\mathcal{R}$.

Figure 17: Oblivious transfer functionality: $\mathcal{F}_{\text{OT}}$

Parameter: Number of sender messages N. Modulus of additive shares M.
Functionality:

(1) Wait for input $(\text{choose}, c_{\mathcal{S}})$ from $\mathcal{S}$. If $c_{\mathcal{S}} \notin \mathbb{Z}_N$, send (invalid input) to both parties and halt. Store $c_{\mathcal{S}}$ and send the delayed public message (chosen) to $\mathcal{R}$. Ignore subsequent inputs of the form $(\text{choose}, \cdot)$.
(2) Wait for input (sample share) from $\mathcal{R}$. Sample $m_{\mathcal{R},c} \xleftarrow{\$} \mathbb{Z}_M$, store it, and send it to $\mathcal{R}$. Ignore subsequent (sample share) messages.
(3) Wait for input $(\text{propose}, c_{\mathcal{R}}, \{m_i\}_{i\in[N]})$ from $\mathcal{R}$. If $c_{\mathcal{R}} \notin \mathbb{Z}_N$ or $m_{\mathcal{R},c}$ is not stored or $\{m_i\}_{i\in[N]} \notin (\mathbb{Z}_M)^N$, send (invalid input) to both parties and halt; Otherwise compute $m_{\mathcal{S},c} = (m_c + m_{\mathcal{R},c}) \bmod M$, and send $m_{\mathcal{S},c}$ to $\mathcal{S}$. Ignore subsequent messages of this form.

Figure 18: 1-out-of-N shared oblivious transfer functionality: $\mathcal{F}_{\text{SOT}}$

B.9 Circuit-based Private Set Intersection

The ideal functionality $\mathcal{F}_{\text{cPSI}}$ is shown in Figure 19.

B.10 Multi-query Fuzzy Reverse Private Membership Test

The ideal functionality of mqFRPMT is shown in Figure 20.

B.11 Hash-to-Bin

Cuckoo hashing [36] employs γ hash functions $h_1, \ldots, h_\gamma : \{0,1\}^* \to [\beta]$ to map m items into a table of $\beta = \varepsilon n$ bins, where each bin stores exactly one item. For an item $\boldsymbol{x}$, it is hashed into the one of the candidate bins $B_{h_1(\text{id}(\boldsymbol{x}))}, \ldots, B_{h_\gamma(\text{id}(\boldsymbol{x}))}$ by its identifier $\text{id}(\boldsymbol{x})$. If all these bins are occupied, the algorithm selects one bin at random, evicts its current occupant, and attempts to reinsert the evicted item using another hash function. This eviction-and-reinsertion

Parameter: A randomized function Reorder that maps a set X of size n to an injective function $\mathsf{idx} : X \to [\theta]$.
Functionality:

(1) Initialize an internal state $\mathsf{state}_{\mathcal{U}} = \emptyset$ for each party $\mathcal{U} \in \{\mathcal{S}, \mathcal{R}\}$. If $\mathcal{U}$ is corrupt, the simulator Sim is allowed to access $\mathsf{state}_{\mathcal{U}}$.
(2) Wait for input $X = \{x_i\}_{i\in[n]}$ from $\mathcal{S}$. Update $\mathsf{state}_{\mathcal{S}} \leftarrow \langle X\rangle$, and send $\langle \mathsf{Request}, \mathcal{S}\rangle$ to Sim.
(3) Wait for input $Y = \{y_j\}_{j\in[m]}$ from $\mathcal{R}$. Update $\mathsf{state}_{\mathcal{R}} \leftarrow \langle Y\rangle$, and send $\langle \mathsf{Request}, \mathcal{R}\rangle$ to Sim.
(4) Upon receiving $\langle \mathsf{Request}, \mathsf{OK}\rangle$ from Sim, compute an injective function $\mathsf{idx} \leftarrow \mathsf{Recorder}(X)$. For each $x \in X$, define $b \in \{0,1\}^\theta$ as

$$b_i = \begin{cases} 1, & \text{if } (x \in Y) \text{ and } \mathsf{idx}(x) = i, \\ 0, & \text{otherwise.} \end{cases}$$

Generate Boolean shares of b: sample $b_{\mathcal{R}} \xleftarrow{\$} \{0,1\}^\theta$ and set $b_{\mathcal{S}} = b \oplus b_{\mathcal{R}}$.
(5) Update internal states: $\mathsf{state}_{\mathcal{S}} \leftarrow \mathsf{state}_{\mathcal{S}} \cup \langle b_{\mathcal{S}}\rangle$, $\mathsf{state}_{\mathcal{R}} \leftarrow \mathsf{state}_{\mathcal{R}} \cup \langle b_{\mathcal{R}}\rangle$.
(6) Output $\langle b_{\mathcal{S}}\rangle$ to $\mathcal{S}$ and $\langle b_{\mathcal{R}}\rangle$ to $\mathcal{R}$.

Figure 19: Circuit-based private set intersection functionality: $\mathcal{F}_{\mathsf{cPSI}}$

Parameter: Distance function $\mathsf{dist}(\cdot,\cdot)$. Distance threshold δ.
Functionalities:

(1) Wait for input $X = \{\boldsymbol{x_i}\}_{i\in[m]} \subset \mathcal{D}$ from $\mathcal{S}$.
(2) Wait for input $Y = \{\boldsymbol{y_j}\}_{j\in[n]} \subset \mathcal{D}$ from $\mathcal{R}$.
(3) Compute an indicator vector $e \in \{0,1\}^m$, where for each $i \in [m]$,

$$e_i = \begin{cases} 1, & \text{if } \exists\, \boldsymbol{y_j} \in Y \text{ such that } \mathsf{dist}(\boldsymbol{x_i}, \boldsymbol{y_j}) \le \delta, \\ 0, & \text{otherwise.} \end{cases}$$

(4) Output e to $\mathcal{R}$.

Figure 20: Multi-query fuzzy reverse private membership test functionality: $\mathcal{F}_{\mathsf{mqFRPMT}}$

process continues until a vacant bin is found or until a predefined eviction threshold is reached. When insertion fails, the remaining items are stored in a small auxiliary structure called "stash". With appropriate choices of γ and ε, the stash size becomes zero in almost all practical executions, and the failure probability is bounded by $2^{-\lambda}$ [37]. **Simple hashing** [36] uses the same collection of γ hash functions $h_1, \ldots, h_\gamma : \{0,1\}^* \to [\beta]$ to assign each receiver item $\boldsymbol{y_i}$ to multiple bins. Specifically, for each $i \in [n]$, the item $\boldsymbol{y_i}$ is inserted into the bins $Y_{h_j(\mathsf{id}(\boldsymbol{y_i}))}$ for all $j \in [\gamma]$. Unlike cuckoo hashing, where each bin stores only one item, simple hashing permits multiple items per bin. In the context of fuzzy matching, this procedure enables the receiver to construct sets $Y_1, \ldots, Y_\beta$ such that for any $\boldsymbol{x} \in X$, the sender only needs to compare $\boldsymbol{x}$ with the items in the designated candidate set $Y_{h_j(\mathsf{id}(\boldsymbol{x}))}$, rather than with all of Y.

C Complexity Analysis

C.1 $\Pi^{\mathsf{dist}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$ Complexity

Remark 2 (Complexity of $\Pi^{\mathsf{dist}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$). By instantiating the mqFRPMT functionality with the protocols of [15], we obtain the following asymptotic complexity bounds for the fuzzy PSU protocol described in Figure 2.

- **Hamming space.** The protocol incurs a communication cost of $O(d^2m+\delta dn)$, a sender-side computational cost of $O(d^2m)$, and a receiver-side computational cost of $O(d^2m + \delta dn)$.
- **Minkowski space ($p = \infty$).** The protocol incurs a communication cost of $O(\delta dm + \delta dn)$, a sender-side computational cost of $O(\delta dm + n)$, and a receiver-side computational cost of $O(\delta dn + m)$.
- **Minkowski space ($p \in [1,\infty)$).** The protocol incurs a communication cost of $O((\delta d + p\log\delta)m + \delta dn)$, a sender-side computational cost of $O((\delta d + p\log\delta)m + n)$, and a receiver-side computational cost of $O(p\log\delta\, m + \delta dn)$.

C.2 Π_{tssVODM} Complexity

Remark 3 (Π_{tssVODM} Complexity). We analyze the complexity of our instantiation of tssVODM. Similar to the VODM protocol of Zhang et al. [7], we use LowMC [38] as the underlying SKE and implement the generic 2PC through the GMW protocol. In addition to the LowMC decryption circuit, tssVODM requires d threshold comparison circuits in order to check whether the number of successful decryptions is greater than a given threshold t. Let A_{dec} denote the number of AND gates in a single LowMC decryption, and let σ be the bit length of the compared message. The tssVODM evaluates $\mathsf{Dec}(sk,\cdot)$ exactly dd' times. It also performs dd' comparisons of the form $\mathsf{Dec}(sk,\cdot) \stackrel{?}{=} s$, which are implemented by an σ-bit equality circuit with $A_{\mathsf{EQ}}(\sigma) = \sigma - 1$ AND gates (under the standard assumption that XOR and NOT gates are free). For each set C_i, the functionality computes the sum $\sum_{j=1}^{d'} |\mathsf{Dec}(sk, c_{i,j}) = s|$ using a CSA- or Wallace-tree-based bit-sum circuit. Such a circuit requires a number of AND gates linear in d', which we denote by $A_{\mathsf{SUM}}(d') = O(d')$. The resulting w-bit sum, where $w = \lceil \log(d'+1)\rceil$, is then compared with the threshold t using a standard w-bit comparison circuit, which requires $A_{\mathsf{CMP}}(w) = O(w)$ AND gates. Finally, the d per-set comparison results are ANDed together to enforce that the threshold condition holds for all sets, which incurs an additional $d-1$ AND gates. The overall AND gate count is therefore

$$A_{\mathsf{tssVODM}} = dd'(A_{\mathsf{dec}} + \sigma - 1) + d\big(A_{\mathsf{SUM}}(d') + A_{\mathsf{CMP}}(w)\big) + (d-1).$$

Each AND gate incurs a communication cost of exactly four bits and requires constant local computation under the Beaver triple technique [39]. Hence, the total asymptotic communication complexity of tssVODM is $O(dd')$, and both the sender and the receiver incur $O(dd')$ local asymptotic computational complexity.

C.3 $\Pi_{\mathsf{OTVDnECRG}}$ Complexity

Remark 4 ($\Pi_{\mathsf{OTVDnECRG}}$ Complexity). According to the analysis in [9], the nECRG can be implemented with linear communication

and computation complexity. Therefore, the overall asymptotic communication complexity is $O(mdd')$. Both the sender and the receiver incur an asymptotic computational complexity of $O(mdd')$.

C.4 $\Pi^{\mathsf{dist}_{\mathrm{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$ Complexity

Remark 5 ($\Pi^{\mathsf{dist}_{\mathrm{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$ Complexity). Assume that the OKVS employed in the protocol has constant rate, linear time encoding, and constant time decoding. Under these assumptions, the overall asymptotic communication complexity of the protocol $\Pi^{\mathsf{dist}_{\mathrm{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$ in Figure 8 is $O(md^2 + nd\delta)$. The sender incurs asymptotic computational complexity of $O(md^2)$, while the receiver incurs computational complexity of $O(md^2 + nd\delta)$.

C.5 $\Pi^{\mathsf{dist}_{\mathrm{H}}(\cdot,\cdot)}_{\mathsf{fuzzy\text{-}ePSU}}$ Complexity

Remark 6 ($\Pi^{\mathsf{dist}_{\mathrm{H}}(\cdot,\cdot)}_{\mathsf{fuzzy\text{-}ePSU}}$ Complexity). The asymptotic communication complexity of the fuzzy ePSU protocol in the Hamming space (as shown in Figure 9) is $O(md^2 + nd\delta)$. The sender's asymptotic computational complexity is $O(md^2)$, while the receiver's asymptotic computational complexity is $O(md^2 + nd\delta)$.

C.6 $\Pi^{\mathsf{dist}_{\infty}(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ Complexity

Remark 7 ($\Pi^{\mathsf{dist}_{\infty}(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ Complexity). We use the protocol of [34] to realize the functionality $\mathcal{F}_{\mathsf{SOT}}$, which incurs asymptotic communication complexity $O(N \log M)$ and asymptotic computational complexity $O(N \log M + \log N)$. The functionality $\mathcal{F}_{\mathsf{cPSI}}$ is instantiated using the protocol of [32], which achieves linear communication and computation. The functionality $\mathcal{F}_{\mathsf{pECRG}}$ is instantiated using the protocol of [9], which also achieves linear communication and computation. Combining these components, the asymptotic communication complexity of the FpnMCRG protocol for the ∞-norm Minkowski distance (as shown in Figure 11) is $O(mdBN \log M)$, and the asymptotic computational complexity of both the sender and receiver is $O(mdBN \log M + mdB \log N)$.

C.7 $\Pi^{\mathsf{dist}_{p\in[1,\infty)}(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ Complexity

Remark 8 ($\Pi^{\mathsf{dist}_{p\in[1,\infty)}(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ Complexity). The asymptotic communication complexity of the FpnMCRG protocol for $p \in [1,\infty)$ (as shown in Figure 12) is $O(mdBN \log M + mBp \log \delta)$, the sender's asymptotic computational complexity is $O(mdBN \log M + mdB \log N + mB\delta^p)$, and the receiver's asymptotic computational complexity is $O(mdBN \log M + mdB \log N + mBp \log \delta)$.

C.8 $\Pi^{\mathsf{dist}_{p}(\cdot,\cdot)}_{\mathsf{fuzzy\text{-}ePSU}}$ Complexity

Remark 9. We adopt the protocol from [15] to realize the SAS-Fmap, which achieves a asymptotic communication complexity of $O(\delta d(m+n))$, a asymptotic computational complexity of $O(\delta dm+n)$ for the sender, and $O(\delta dn + m)$ for the receiver.

- For the $\Pi^{\mathsf{dist}_{\infty}(\cdot,\cdot)}_{\mathsf{fuzzy\text{-}ePSU}}$ protocol, the asymptotic communication complexity is $O(\delta d(m+n) + mdBN \log M)$, the sender's asymptotic computation complexity is $O(\delta dm+n+mdBN \log M+mdB \log N)$, and the receiver's asymptotic computation complexity is $O(\delta dn + mdBN \log M + mdB \log N)$.
- For the $\Pi^{\mathsf{dist}_{p\in[1,\infty)}(\cdot,\cdot)}_{\mathsf{fuzzy\text{-}ePSU}}$ protocol, the asymptotic communication complexity is $O(\delta d(m+n) + mdBN \log M + mBp \log \delta)$, the sender's asymptotic computation complexity is $O(\delta dm + n + mdBN \log M + mdB \log N + mB\delta^p)$, and the receiver's asymptotic computation complexity is $O(\delta dn + mdBN \log M + mdB \log N + mBp \log \delta)$.

D Correctness and Security Proof

D.1 Proof of $\Pi_{\mathsf{OTVDnECRG}}$

Theorem D.1 ($\Pi_{\mathsf{OTVDnECRG}}$ Correctness). *The protocol $\Pi_{\mathsf{OTVDnECRG}}$ presented in Figure 6 correctly realizes the functionality $\mathcal{F}_{\mathsf{OTVDnECRG}}$ defined in Figure 5.*

Proof. For each $i \in [m]$. There are two possible cases.

- **Case 1.** Suppose there exists a set $C_{i,j}$ such that $\sum_{k=1}^{d'} |\mathrm{Dec}(sk, c_{i,j,k}) = s| \geq t$. By the correctness of $\mathcal{F}_{\mathsf{tssVODM}}$, this condition implies $e_{\mathcal{S},i} \oplus e_{\mathcal{R},i} = 0$, that is, $e_{\mathcal{S},i} = e_{\mathcal{R},i}$. Since $\widetilde{\boldsymbol{e}}_{\mathcal{S},i} = \boldsymbol{e}^d_{\mathcal{S},i}$ and $\widetilde{\boldsymbol{e}}_{\mathcal{R},i} = \boldsymbol{e}^d_{\mathcal{R},i}$, the correctness of $\mathcal{F}_{\mathsf{nECRG}}$ guarantees that equality of these input shares forces $u_{\mathcal{S},i,1} \neq u_{\mathcal{R},i,1}, \ldots, u_{\mathcal{S},i,d} \neq u_{\mathcal{R},i,d}$. Thus the output vectors $\boldsymbol{u}_{\mathcal{S},i}$ and $\boldsymbol{u}_{\mathcal{R},i}$ differ in every coordinate, exactly as required by the functionality.
- **Case 2.** Suppose that every set $C_{i,j}$ satisfies $\sum_{k=1}^{d'} |\mathrm{Dec}(sk, c_{i,j,k}) = s| < t$. By correctness of $\mathcal{F}_{\mathsf{tssVODM}}$, this condition implies $e_{\mathcal{S},i} \oplus e_{\mathcal{R},i} = 1$, that is, $e_{\mathcal{S},i} \neq e_{\mathcal{R},i}$. Again using $\widetilde{\boldsymbol{e}}_{\mathcal{S},i} = \boldsymbol{e}^d_{\mathcal{S},i}$ and $\widetilde{\boldsymbol{e}}_{\mathcal{R},i} = \boldsymbol{e}^d_{\mathcal{R},i}$, the correctness of $\mathcal{F}_{\mathsf{nECRG}}$ ensures that inequality of these input shares yields $u_{\mathcal{S},i,1} = u_{\mathcal{R},i,1}, \ldots, u_{\mathcal{S},i,d} = u_{\mathcal{R},i,d}$. Therefore the two output points coincide in every coordinate, again matching the specification of the ideal functionality.

Since these are exactly the two conditions specified by the ideal functionality $\mathcal{F}_{\mathsf{OTVDnECRG}}$, the protocol is correct.

□

Theorem D.2 ($\Pi_{\mathsf{OTVDnECRG}}$ security). *Assume the SKE scheme (*Setup, KeyGen, Enc, Dec*) satisfies single-message multi-ciphertext pseudorandomness. The protocol presented in Figure 6 securely computes the $\mathcal{F}_{\mathsf{OTVDnECRG}}$ functionality defined in Figure 5 against semi-honest adversaries in the ($\mathcal{F}_{\mathsf{tssVODM}}$, $\mathcal{F}_{\mathsf{nECRG}}$)-hybrid model.*

Proof. We will show that for any adversary $\mathcal{A}$, we can construct two simulators $\mathsf{Sim}_{\mathcal{S}}$ and $\mathsf{Sim}_{\mathcal{R}}$ for simulating the view of the corrupt $\mathcal{S}$ and $\mathcal{R}$ respectively, such that any PPT environment cannot distinguish the execution in the ideal world from that in the real world.

Corrupt Sender: $\mathsf{Sim}_{\mathcal{S}}$ simulates a real execution of the corrupt semi-honest sender. Since $\mathcal{A}$ is semi-honest, $\mathsf{Sim}_{\mathcal{S}}$ can obtain the input $\{C_{i,1}, \ldots, C_{i,d}\}_{i\in[m]}$ and a threshold t of $\mathcal{S}$ directly, and externally send them to $\mathcal{F}_{\mathsf{OTVDnECRG}}$ and then receives $\langle \mathsf{Request}, \mathcal{S}\rangle$. It then executes as follows:

(1) For $i \in [m]$, once obtaining $\{C_{i,1}, ..., C_{i,d}\}$ and t, the input of Π_{tssVODM}, from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{S}}$ randomly selects $e_{\mathcal{S},i}$ to simulate the execution of Π_{tssVODM}.
(2) Once obtaining $\{\widetilde{e}_{\mathcal{S},i,1}, \ldots, \widetilde{e}_{\mathcal{S},i,d}\}_{i\in[m]}$, the input of Π_{nECRG} from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{S}}$ sends $\langle \mathsf{Response}, \mathsf{OK}\rangle$ to $\mathcal{F}_{\mathsf{OTVDnECRG}}$, and obtains $\{u_{\mathcal{S},i,1}, \ldots, u_{\mathcal{S},i,d}\}_{i\in[m]}$. Finally, $\mathsf{Sim}_{\mathcal{S}}$ simulates the execution of Π_{nECRG} with $\{u_{\mathcal{S},i,1}, \ldots, u_{\mathcal{S},i,d}\}_{i\in[m]}$ as output.

We argue that the outputs of $\mathsf{Sim}_{\mathcal{S}}$ are indistinguishable from the real view of $\mathcal{S}$ by the following hybrids:

- Hybrid_0: $\mathcal{S}$'s view in the real protocol.
- Hybrid_1: Same as Hybrid_0 except that, for each $i \in [m]$, the output of Π_{tssVODM} is replaced by $e_{\mathcal{S},i}$ chosen by $\mathsf{Sim}_{\mathcal{S}}$, and $\mathsf{Sim}_{\mathcal{S}}$ runs the Π_{tssVODM} simulator to produce the simulated view for $\mathcal{S}$. The security of protocol Π_{tssVODM} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.
- Hybrid_2: Same as Hybrid_1 except that the output of Π_{nECRG} is replaced by $\{u_{\mathcal{S},i,1}, \ldots, u_{\mathcal{S},i,d}\}_{i\in[m]}$ output by $\mathcal{F}_{\mathsf{OTVDnECRG}}$ and $\mathsf{Sim}_{\mathcal{S}}$ runs the Π_{nECRG} simulator to produce the simulated view for $\mathsf{Sim}_{\mathcal{S}}$. The security of protocol Π_{nECRG} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.

Clearly, $\mathsf{Sim}_{\mathcal{S}}$'s simulated view is identical to $\mathcal{S}$'s view in the real execution.

Corrupt Receiver: $\mathsf{Sim}_{\mathcal{R}}$ simulates a real execution of the corrupt semi-honest receiver. Since $\mathcal{A}$ is semi-honest, $\mathsf{Sim}_{\mathcal{R}}$ can obtain the input sk and s of $\mathcal{R}$ directly, and externally send them to $\mathcal{F}_{\mathsf{OTVDnECRG}}$ and then receives $\langle \mathsf{Request}, \mathcal{R}\rangle$. It then executes as follows:

(1) For $i \in [m]$, once obtaining sk and s, the input of Π_{tssVODM}, from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{R}}$ randomly selects $e_{\mathcal{R},i}$ to simulate the execution of Π_{tssVODM}.
(2) Once obtaining $\{\widetilde{e}_{\mathcal{R},i,1}, \ldots, \widetilde{e}_{\mathcal{R},i,d}\}_{i\in[m]}$, the input of Π_{nECRG} from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{R}}$ sends $\langle \mathsf{Response}, \mathsf{OK}\rangle$ to $\mathcal{F}_{\mathsf{OTVDnECRG}}$, and obtains $\{u_{\mathcal{R},i,1}, \ldots, u_{\mathcal{R},i,d}\}_{i\in[m]}$. Finally, $\mathsf{Sim}_{\mathcal{R}}$ simulates the execution of Π_{nECRG} with $\{u_{\mathcal{R},i,1}, \ldots, u_{\mathcal{R},i,d}\}_{i\in[m]}$ as output.

We argue that the outputs of $\mathsf{Sim}_{\mathcal{R}}$ are indistinguishable from the real view of $\mathcal{R}$ by the following hybrids:

- Hybrid_0: $\mathcal{R}$'s view in the real protocol.
- Hybrid_1: Same as Hybrid_0 except that, for each $i \in [m]$, the output of Π_{tssVODM} is replaced by $e_{\mathcal{R},i}$ chosen by $\mathsf{Sim}_{\mathcal{R}}$, and $\mathsf{Sim}_{\mathcal{R}}$ runs the Π_{tssVODM} simulator to produce the simulated view for $\mathcal{R}$. The security of protocol Π_{tssVODM} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.
- Hybrid_2: Same as Hybrid_1 except that the output of Π_{nECRG} is replaced by $\{u_{\mathcal{R},i,1}, \ldots, u_{\mathcal{R},i,d}\}_{i\in[m]}$ output by $\mathcal{F}_{\mathsf{OTVDnECRG}}$ and $\mathsf{Sim}_{\mathcal{R}}$ runs the Π_{nECRG} simulator to produce the simulated view for $\mathsf{Sim}_{\mathcal{R}}$. The security of protocol Π_{nECRG} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.

Clearly, $\mathsf{Sim}_{\mathcal{R}}$'s simulated view is identical to $\mathcal{R}$'s view in the real execution.

□

D.2 Proof of $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$

Theorem D.3 ($\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$ Correctness). *The protocol $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$ presented in Figure 8 correctly realizes the functionality $\mathcal{F}_{\mathsf{FOnMCRG}}$ defined in Figure 7 for the Hamming distance $\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)$.*

Proof. The distinctiveness property of the UniqC Fmap guarantees that all keys inserted into the OKVS instance are unique. Hence the probability that the encoding procedure fails is negligible, and the protocol proceeds correctly with overwhelming probability.

For each $i \in [m]$. There are two possible cases.

- **Case 1.** There exists $\boldsymbol{y}_* \in Y$ such that $\mathsf{dist}_{\mathsf{H}}(\boldsymbol{x}_i, \boldsymbol{y}_*) \le \delta$. By correctness of the UniqC Fmap, there exist index $\ell_{\mathcal{S}} \in [|\mathsf{id}(\boldsymbol{x}_i)|]$ and $\ell_{\mathcal{R}} \in [|\mathsf{id}(\boldsymbol{y}_*)|]$ such that $\mathsf{id}_{\boldsymbol{x}_i,\ell_{\mathcal{S}}} = \mathsf{id}_{\boldsymbol{y}_*,\ell_{\mathcal{R}}}$. Therefore, $\mathsf{Decode}(\mathsf{D}, \mathsf{id}_{\boldsymbol{x}_i,\ell_{\mathcal{S}}}, r) = \big(\mathsf{Enc}(sk, s) \oplus y_{*,1}, \ldots, \mathsf{Enc}(sk, s) \oplus y_{*,d}\big)$. Thus $C_{i,\ell_{\mathcal{S}}} = \big(\mathsf{Enc}(sk, s) \oplus y_{*,1} \oplus x_{i,1}, \ldots, \mathsf{Enc}(sk, s) \oplus y_{*,d} \oplus x_{i,d}\big)$. Since $\mathsf{dist}_{\mathsf{H}}(\boldsymbol{x}_i, \boldsymbol{y}_*) \le \delta$, the number of coordinates for which $x_{i,k} \neq y_{*,k}$ is at most δ, and consequently the number of coordinates satisfying $x_{i,k} = y_{*,k}$ is at least $d-\delta$. For these coordinates, $\mathsf{Dec}(sk, \mathsf{Enc}(sk, s) \oplus y_{*,k} \oplus x_{i,k}) = \mathsf{Dec}(sk, \mathsf{Enc}(sk, s)) = s$. Hence, $\sum_{k=1}^{d} |\mathsf{Dec}\left(sk, c_{i,\ell_{\mathcal{S}},k}\right) = s| \ge d - \delta$. With threshold $t = d - \delta$, the correctness of OTVDnECRG implies that $\boldsymbol{u}_{\mathcal{S},i} \neq \boldsymbol{u}_{\mathcal{R},i}$, which matches the ideal behavior of $\mathcal{F}_{\mathsf{FOnMCRG}}$.
- **Case 2.** For all $\boldsymbol{y}_j \in Y$, $\mathsf{dist}_{\mathsf{H}}(\boldsymbol{x}_i, \boldsymbol{y}_j) > \delta$.
  - **Subcase 2a.** $\mathsf{ID}(\boldsymbol{x}_i)$ has no intersection with $\mathsf{ID}(Y)$. By the randomness property of OKVS, for each $k \in [|\mathsf{id}(\boldsymbol{x}_i)|]$ the value $\mathsf{Decode}(\mathsf{D}, \mathsf{id}_{\boldsymbol{x}_i,k}, r)$ is statistically indistinguishable from a uniformly random value. Consequently each ciphertext in $C_{i,k}$ is also statistically indistinguishable from uniform. Therefore, $\sum_{k=1}^{d} |\mathsf{Dec}(sk, c_{i,j,k}) = s| < d - \delta$ for all $j \in [d]$. By correctness of OTVDnECRG, $\boldsymbol{u}_{\mathcal{S},i} = \boldsymbol{u}_{\mathcal{R},i}$.
  - **Subcase 2b.** There exists $\boldsymbol{y}_* \in Y$ such that $\mathsf{ID}(\boldsymbol{x}_i) \cap \mathsf{ID}(\boldsymbol{y}_*) \neq \emptyset$. As in Case 1, $C_{i,\ell_{\mathcal{S}}}$ takes the form $\big(\mathsf{Enc}(sk, s) \oplus y_{*,1} \oplus x_{i,1}, \ldots, \mathsf{Enc}(sk, s) \oplus y_{*,d} \oplus x_{i,d}\big)$. However, since $\mathsf{dist}_{\mathsf{H}}(\boldsymbol{x}_i, \boldsymbol{y}_*) > \delta$, the number of coordinates satisfying $x_{i,k} = y_{*,k}$ is strictly less than $d - \delta$. Hence, $\sum_{k=1}^{d} |\mathsf{Dec}(sk, c_{i,\ell_{\mathcal{S}},k}) = s| < d - \delta$. The same bound holds for all other $C_{i,j}$. Thus correctness of OTVDnECRG yields $\boldsymbol{u}_{\mathcal{S},i} = \boldsymbol{u}_{\mathcal{R},i}$.

  Therefore, in all situations covered by Case 2, $\boldsymbol{u}_{\mathcal{S},i} = \boldsymbol{u}_{\mathcal{R},i}$.

Since these are exactly the two conditions specified by the ideal functionality $\mathcal{F}_{\mathsf{FOnMCRG}}$, the protocol is correct.

□

Theorem D.4 ($\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$ security). *Assume the SKE scheme (*Setup, KeyGen, Enc, Dec*) satisfies single-message multi-ciphertext pseudorandomness, and OKVS satisfies the obliviousness and randomness. The protocol presented in Figure 8 securely computes the $\mathcal{F}_{\mathsf{FOnMCRG}}$ functionality defined in Figure 7 for Hamming distance $\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)$ against semi-honest adversaries in the $(\mathcal{F}^{\mathsf{UniqC}}_{\mathsf{Fmap}}, \mathcal{F}_{\mathsf{OTVDnECRG}})$-hybrid model.*

Proof. We will show that for any adversary $\mathcal{A}$, we can construct two simulators $\mathsf{Sim}_{\mathcal{S}}$ and $\mathsf{Sim}_{\mathcal{R}}$ for simulating the view of the corrupt $\mathcal{S}$ and $\mathcal{R}$ respectively, such that any PPT environment cannot distinguish the execution in the ideal world from that in the real world.

Corrupt Sender: $\mathsf{Sim}_{\mathcal{S}}$ simulates a real execution of the corrupt semi-honest sender. Since $\mathcal{A}$ is semi-honest, $\mathsf{Sim}_{\mathcal{S}}$ can obtain the input $X = \{\boldsymbol{x}_i\}_{i\in[m]}$ of $\mathcal{S}$ directly, and externally send them to $\mathcal{F}_{\mathsf{FOnMCRG}}$ and then receives $\langle \mathsf{Request}, \mathcal{S}\rangle$. It then executes as follows:

(1) Once obtaining X, the input of $\Pi^{\mathsf{UniqC}}_{\mathsf{Fmap}}$, from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{S}}$ randomly samples $\mathsf{ID}(X)$ to simulate the execution of $\Pi^{\mathsf{UniqC}}_{\mathsf{Fmap}}$.

(2) $\mathsf{Sim}_{\mathcal{S}}$ computes a random OKVS D by selecting random key-value pairs.
(3) Once obtaining $\{C_{i,1}, \ldots, C_{i,|\mathsf{id}(\boldsymbol{x}_i)|}\}_{i\in[m]}$ and a threshold t, the input of $\Pi_{\mathsf{OTVDnECRG}}$ from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{S}}$ sends $\langle \mathsf{Response}, \mathsf{OK}\rangle$ to $\mathcal{F}_{\mathsf{FOnMCRG}}$, and obtains $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$. Finally, $\mathsf{Sim}_{\mathcal{S}}$ simulates the execution of $\Pi_{\mathsf{OTVDnECRG}}$ with $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ as output.

We argue that the outputs of $\mathsf{Sim}_{\mathcal{S}}$ are indistinguishable from the real view of $\mathcal{S}$ by the following hybrids:

- Hybrid_0: $\mathcal{S}$'s view in the real protocol.
- Hybrid_1: Same as Hybrid_0 except that the output of $\Pi^{\mathsf{UniqC}}_{\mathsf{Fmap}}$ is replaced by $\mathsf{ID}(X)$ chosen by $\mathsf{Sim}_{\mathcal{S}}$, and $\mathsf{Sim}_{\mathcal{S}}$ runs the $\Pi^{\mathsf{UniqC}}_{\mathsf{Fmap}}$ simulator to produce the simulated view for $\mathcal{S}$. The security of protocol $\Pi^{\mathsf{UniqC}}_{\mathsf{Fmap}}$ guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.
- Hybrid_2: Same as Hybrid_1 except that $\mathsf{Sim}_{\mathcal{S}}$ computes a random OKVS D. Briefly, this simulation is indistinguishable from the following reasons: The IND-CPA security of the SKE scheme with single-message multi-ciphertext pseudo-randomness property ensures the ciphertext value is indistinguishable from random, and then by the obliviousness and randomness of OKVS, D is distributed uniformly.
- Hybrid_3: Same as Hybrid_2 except that the output of $\Pi_{\mathsf{OTVDnECRG}}$ is replaced by $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ output by $\mathcal{F}_{\mathsf{FOnMCRG}}$ and $\mathsf{Sim}_{\mathcal{S}}$ runs the Π_{OTVnECRG} simulator to produce the simulated view for $\mathsf{Sim}_{\mathcal{S}}$. The security of protocol $\Pi_{\mathsf{OTVDnECRG}}$ guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.

Clearly, $\mathsf{Sim}_{\mathcal{S}}$'s simulated view is identical to $\mathcal{S}$'s view in the real execution.

Corrupt Receiver: $\mathsf{Sim}_{\mathcal{R}}$ simulates a real execution of the corrupt semi-honest receiver. Since $\mathcal{A}$ is semi-honest, $\mathsf{Sim}_{\mathcal{R}}$ can obtain the input $Y = \{\boldsymbol{y}_j\}_{j\in[n]}$ of $\mathcal{R}$ directly, and externally send them to $\mathcal{F}_{\mathsf{FOnMCRG}}$ and then receives $\langle \mathsf{Request}, \mathcal{R}\rangle$. It then executes as follows:

(1) Once obtaining Y, the input of $\Pi^{\mathsf{UniqC}}_{\mathsf{Fmap}}$, from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{R}}$ randomly selects $\mathsf{ID}(Y)$ to simulate the execution of $\Pi^{\mathsf{UniqC}}_{\mathsf{Fmap}}$.
(2) Once obtaining sk and s, the input of $\Pi_{\mathsf{OTVDnECRG}}$ from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{R}}$ sends $\langle \mathsf{Response}, \mathsf{OK}\rangle$ to $\mathcal{F}_{\mathsf{FOnMCRG}}$, and obtains $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$. Finally, $\mathsf{Sim}_{\mathcal{R}}$ simulates the execution of $\Pi_{\mathsf{OTVDnECRG}}$ with $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$ as output.

We argue that the outputs of $\mathsf{Sim}_{\mathcal{R}}$ are indistinguishable from the real view of $\mathcal{R}$ by the following hybrids:

- Hybrid_0: $\mathcal{R}$'s view in the real protocol.
- Hybrid_1: Same as Hybrid_0 except that the output of $\Pi^{\mathsf{UniqC}}_{\mathsf{Fmap}}$ is replaced by $\mathsf{ID}(Y)$ chosen by $\mathsf{Sim}_{\mathcal{R}}$, and $\mathsf{Sim}_{\mathcal{R}}$ runs the $\Pi^{\mathsf{UniqC}}_{\mathsf{Fmap}}$ simulator to produce the simulated view for $\mathcal{R}$. The security of protocol $\Pi^{\mathsf{UniqC}}_{\mathsf{Fmap}}$ guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.
- Hybrid_2: Same as Hybrid_1 except that the output of $\Pi_{\mathsf{OTVDnECRG}}$ is replaced by $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$ output by $\mathcal{F}_{\mathsf{FOnMCEG}}$ and $\mathsf{Sim}_{\mathcal{R}}$ runs the $\Pi_{\mathsf{OTVDnECRG}}$ simulator to produce the simulated view for $\mathsf{Sim}_{\mathcal{R}}$. The security of protocol $\Pi_{\mathsf{OTVDnECRG}}$ guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.

Clearly, $\mathsf{Sim}_{\mathcal{R}}$'s simulated view is identical to $\mathcal{R}$'s view in the real execution. □

D.3 Proof of $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{fuzzy\text{-}ePSU}}$

Theorem D.5 ($\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{fuzzy\text{-}ePSU}}$ Correctness). *The protocol* $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{fuzzyePSU}}$ *presented in Figure 9 correctly realizes the functionality* $\mathcal{F}^{m,n}_{\mathsf{fuzzyePSU}}$ *defined in Figure 3 for the Hamming distance* $\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)$.

Proof. For each $i \in [m]$. There are two possible cases.

- **Case 1.** For every $\boldsymbol{y}_j \in Y$, we have $\mathsf{dist}_{\mathsf{H}}(\boldsymbol{x}_i, \boldsymbol{y}_j) > \delta$. By the correctness of $\mathcal{F}_{\mathsf{FOnMCRG}}$, this implies $\boldsymbol{u}_{\mathcal{S},i} = \boldsymbol{u}_{\mathcal{R},i}$. Therefore, for each $j \in [d]$ the receiver computes $v_{i,j} \,\|\, v'_{i,j} = u_{i,j} \oplus u_{\mathcal{R},i,j} = (x_{i,j} \,\|\, h(x_{i,j}))$. Hence, $h(v_{i,j}) = v'_{i,j} = h(x_{i,j})$ for all $j \in [d]$. Thus the receiver inserts $\boldsymbol{x}_i$ into $\widetilde{\mathsf{Z}}$, and the final output $\mathsf{Z} = Y \cup \widetilde{\mathsf{Z}}$ contains $\boldsymbol{x}_i$, which matches the behavior required by $\mathcal{F}^{m,n}_{\mathsf{fuzzy\text{-}ePSU}}$.
- **Case 2.** There exists $\boldsymbol{y}_* \in Y$ such that $\mathsf{dist}_{\mathsf{H}}(\boldsymbol{x}_i, \boldsymbol{y}_*) \le \delta$. By correctness of $\mathcal{F}_{\mathsf{FOnMCRG}}$, this implies $\boldsymbol{u}_{\mathcal{S},i} \neq \boldsymbol{u}_{\mathcal{R},i}$, and in particular every coordinate differs. For each $j \in [d]$, $v_{i,j} \,\|\, v'_{i,j} = u_{i,j} \oplus u_{\mathcal{R},i,j}$, which yields a pair that is independent of $(x_{i,j}, h(x_{i,j}))$ because $u_{i,j}$ and $u_{\mathcal{R},i,j}$ differ in every coordinate. Thus, $h(v_{i,j}) = v'_{i,j}$ fails for all $j \in [d]$. Consequently, $\boldsymbol{x}_i \notin \widetilde{\mathsf{Z}}$ and therefore $\boldsymbol{x}_i \notin \mathsf{Z}$. This agrees exactly with the specification of $\mathcal{F}^{m,n}_{\mathsf{fuzzy\text{-}ePSU}}$, which excludes any $\boldsymbol{x}_i$ that is within distance δ of some point of Y.

Since these are exactly the two conditions specified by the ideal functionality $\mathcal{F}^{m,n}_{\mathsf{fuzzy\text{-}ePSU}}$, the protocol is correct.

□

Theorem D.6 ($\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{fuzzy\text{-}ePSU}}$ security). *The protocol presented in Figure 9 securely computes the* $\mathcal{F}^{m,n}_{\mathsf{fuzzy\text{-}ePSU}}$ *functionality defined in Figure 3 for Hamming distance* $\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)$ *against semi-honest adversaries in the* ($\mathcal{F}_{\mathsf{FOnMCRG}}$)*-hybrid model.*

Proof. We will show that for any adversary $\mathcal{A}$, we can construct two simulators $\mathsf{Sim}_{\mathcal{S}}$ and $\mathsf{Sim}_{\mathcal{R}}$ for simulating the view of the corrupt $\mathcal{S}$ and $\mathcal{R}$ respectively, such that any PPT environment cannot distinguish the execution in the ideal world from that in the real world.

Corrupt Sender: $\mathsf{Sim}_{\mathcal{S}}$ simulates a real execution of the corrupt semi-honest sender. Since $\mathcal{A}$ is semi-honest, $\mathsf{Sim}_{\mathcal{S}}$ can obtain the input $X = \{\boldsymbol{x}_i\}_{i\in[m]}$ of $\mathcal{S}$ directly, and externally send them to $\mathcal{F}^{m,n}_{\mathsf{fuzzy\text{-}ePSU}}$ and then receives $\langle \mathsf{Request}, \mathcal{S}\rangle$. It then executes as follows:

(1) Once obtaining X, the input of $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$, from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{S}}$ randomly samples $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ to simulate the execution of $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$.
(2) Once obtaining $\{\boldsymbol{u}_i\}_{i\in[m]}$, $\mathsf{Sim}_{\mathcal{S}}$ sends $\langle \mathsf{Response}, \mathsf{OK}\rangle$ to $\mathcal{F}^{m,n}_{\mathsf{fuzzy\text{-}ePSU}}$.

We argue that the outputs of $\mathsf{Sim}_{\mathcal{S}}$ are indistinguishable from the real view of $\mathcal{S}$ by the following hybrids:

- Hybrid_0: $\mathcal{S}$'s view in the real protocol.
- Hybrid_1: Same as Hybrid_0 except that the output of $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$ is replaced by $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$ chosen by $\mathsf{Sim}_{\mathcal{S}}$, and $\mathsf{Sim}_{\mathcal{S}}$ runs the $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$ simulator to produce the simulated view for $\mathcal{S}$. The security of protocol $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$ guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.

Clearly, $\mathsf{Sim}_{\mathcal{S}}$'s simulated view is identical to $\mathcal{S}$'s view in the real execution.

Corrupt Receiver: $\mathsf{Sim}_{\mathcal{R}}$ simulates a real execution of the corrupt semi-honest receiver. Since $\mathcal{A}$ is semi-honest, $\mathsf{Sim}_{\mathcal{R}}$ can obtain the input $Y = \{\boldsymbol{y}_j\}_{j\in[n]}$ of $\mathcal{R}$ directly, and externally send them to $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$ and then receives $\langle \text{Request}, \mathcal{R}\rangle$. It then executes as follows:

(1) Once obtaining Y, the input of $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$, from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{R}}$ randomly selects $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$ to simulate the execution of $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$.
(2) $\mathsf{Sim}_{\mathcal{R}}$ sends $\langle \text{Response}, \mathsf{OK}\rangle$ to $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$. After obtaining Z, $\mathsf{Sim}_{\mathcal{R}}$ calculates $\widetilde{Z}$ and randomly selects a subset $\widetilde{U}$ from $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$, where $|\widetilde{U}| = |\widetilde{Z}|$. For each items in $\widetilde{U}$, $\mathsf{Sim}_{\mathcal{R}}$ sets $\boldsymbol{u}_i$, where for each $j \in [d]$, $u_{i,j} = \widetilde{u}_{i,j} \oplus (\widetilde{z}_{i,j} \| h(\widetilde{z}_{i,j}))$. Finally, $\mathsf{Sim}_{\mathcal{R}}$ chooses random items for $i \in \{|\widetilde{U}|+1, \ldots, m\}$ and sends all shuffled ciphertexts to $\mathcal{A}$.

We argue that the outputs of $\mathsf{Sim}_{\mathcal{R}}$ are indistinguishable from the real view of $\mathcal{R}$ by the following hybrids:

- Hybrid_0: $\mathcal{R}$'s view in the real protocol.
- Hybrid_1: Same as Hybrid_0 except that the output of $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$ is replaced by $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$ chosen by $\mathsf{Sim}_{\mathcal{R}}$, and $\mathsf{Sim}_{\mathcal{R}}$ runs the $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$ simulator to produce the simulated view for $\mathcal{R}$. The security of protocol $\Pi^{\mathsf{dist}_{\mathsf{H}}(\cdot,\cdot)}_{\mathsf{FOnMCRG}}$ guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.

Clearly, $\mathsf{Sim}_{\mathcal{R}}$'s simulated view is identical to $\mathcal{R}$'s view in the real execution. □

D.4 Proof of $\Pi^{\mathsf{dist}_\infty(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$

Theorem D.7 ($\Pi^{\mathsf{dist}_\infty(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ Correctness). *The protocol $\Pi^{\mathsf{dist}_\infty(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ presented in Figure 11 correctly realizes the $\mathcal{F}_{\mathsf{FpnMCRG}}$ functionality in Figure 10 under the ∞-norm Minkowski distance $\mathsf{dist}_\infty(\cdot,\cdot)$.*

Proof. For each $i \in [m]$. There are two possible cases.

- **Case 1:** There exists point $Y_i[*] \in Y_i$ satisfying $\mathsf{dist}_\infty(\boldsymbol{x}_i, Y_i[*]) \le \delta$. By definition of the ∞-norm, this means that for every $k \in [d]$, $\left|x_{i,k} - Y_i[*][k]\right| \le \delta$. Hence $x_{i,k} \in [Y_i[*][k] - \delta,\ Y_i[*][k] + \delta]$ holds for every k. By correctness of the $\binom{1}{N}$-SOT, this implies that $s_{i,*,k} + \widetilde{s}_{i,*,k} = v^p_{i,*,k,x_{i,k}} = 1$ for all $k \in [d]$. Thus $\sum_{k=1}^d s_{i,*,k} + \sum_{k=1}^d \widetilde{s}_{i,*,k} = d$, and therefore $\widetilde{e}_{i,*} = \sum_{k=1}^d \widetilde{s}_{i,*,k} + e_{i,*} = \mathsf{mask}_{i,*} + d = \widetilde{\mathsf{mask}}_{i,*}$. By correctness of cPSI, whenever $\widetilde{e}_{i,\Delta} \in \widetilde{\mathsf{MASK}}_i$ and $\mathsf{idx}(\widetilde{e}_{i,\Delta}) = j$ holds, the output satisfies $e_{\mathcal{S},i,j} \oplus e_{\mathcal{R},i,j} = 1$, so $e_{\mathcal{S},i} \ne e_{\mathcal{R},i}$. Since each coordinate is computed as $\widetilde{e}_{\mathcal{S},i,k} = H(e_{\mathcal{S},i})$ and $\widetilde{e}_{\mathcal{R},i,k} = H(e_{\mathcal{R},i})$, the inequality $e_{\mathcal{S},i} \ne e_{\mathcal{R},i}$ implies $\widetilde{e}_{\mathcal{S},i,k} \ne \widetilde{e}_{\mathcal{R},i,k}$ for all $k \in [d]$. By correctness of pECRG, for each k we obtain $u_{\mathcal{S},\pi(i),k} \ne u_{\mathcal{R},\pi(i),k}$. Thus the output points differ in every coordinate: $\boldsymbol{u}_{\mathcal{S},\pi(i)} \ne \boldsymbol{u}_{\mathcal{R},\pi(i)}$.
- **Case 2:** For every $Y_i[j] \in Y_i$, $\mathsf{dist}_\infty(\boldsymbol{x}_i, Y_i[j]) > \delta$. Thus there exists $k' \in [d]$ such that $\left|x_{i,k'} - Y_i[j][k']\right| > \delta$, which means $x_{i,k'} \notin [Y_i[j][k'] - \delta,\ Y_i[j][k'] + \delta]$. By correctness of the $\binom{1}{N}$-SOT, $s_{i,j,k'} + \widetilde{s}_{i,j,k'} = v^p_{i,j,k',x_{i,k'}} = 0$. Hence $\sum_{k=1}^d s_{i,*,k} + \sum_{k=1}^d \widetilde{s}_{i,*,k} < d$, and therefore $\widetilde{e}_{i,j} < \mathsf{mask}_{i,j} + d = \widetilde{\mathsf{mask}}_{i,j}$. Thus $\widetilde{e}_{i,j} \ne \widetilde{\mathsf{mask}}_{i,j}$. By correctness of cPSI, this implies $e_{\mathcal{S},i} = e_{\mathcal{R},i}$. Consequently, $\widetilde{e}_{\mathcal{S},i,k} = \widetilde{e}_{\mathcal{R},i,k}$ for all $k \in [d]$. By correctness of pECRG, this implies $u_{\mathcal{S},\pi(i),k} = u_{\mathcal{R},\pi(i),k}$ for all $k \in [d]$, and therefore $\boldsymbol{u}_{\mathcal{S},\pi(i)} = \boldsymbol{u}_{\mathcal{R},\boldsymbol{\pi}(\boldsymbol{i})}$.

Since these are exactly the two conditions specified by the ideal functionality $\mathcal{F}_{\mathsf{FpnMCRG}}$, the protocol is correct. □

Theorem D.8 ($\Pi^{\mathsf{dist}_\infty(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ security). *The protocol presented in Figure 11 securely computes the $\mathcal{F}_{\mathsf{FpnMCRG}}$ functionality defined in Figure 10 for $p = \infty$ Minkowski distance $\mathsf{dist}_\infty(\cdot,\cdot)$ against semi-honest adversaries in the ($\mathcal{F}_{\mathsf{SOT}}$, $\mathcal{F}_{\mathsf{cPSI}}$,$\mathcal{F}_{\mathsf{pECRG}}$)-hybrid model.*

Proof. We will show that for any adversary $\mathcal{A}$, we can construct two simulators $\mathsf{Sim}_{\mathcal{S}}$ and $\mathsf{Sim}_{\mathcal{R}}$ for simulating the view of the corrupt $\mathcal{S}$ and $\mathcal{R}$ respectively, such that any PPT environment cannot distinguish the execution in the ideal world from that in the real world.

Corrupt Sender: $\mathsf{Sim}_{\mathcal{S}}$ simulates a real execution of the corrupt semi-honest sender. Since $\mathcal{A}$ is semi-honest, $\mathsf{Sim}_{\mathcal{S}}$ can obtain the input $X = \{\boldsymbol{x}_i\}_{i\in[m]}$ of $\mathcal{S}$ directly, and externally send them to $\mathcal{F}_{\mathsf{FpnMCRG}}$ and then receives $\langle \text{Request}, \mathcal{S}\rangle$. It then executes as follows:

(1) For each $i \in [m]$, each $j \in [B]$, and each $k \in [d]$, once obtaining $x_{i,k}$, the input of Π_{SOT}, from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{S}}$ randomly samples $s_{i,j,k}$ to simulate the execution of Π_{SOT}.
(2) For each $i \in [m]$, once obtaining $\{\widetilde{\mathsf{mask}}_{i,1}, \ldots, \widetilde{\mathsf{mask}}_{i,B}\}$, the input of Π_{cPSI} from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{S}}$ randomly samples $e_{\mathcal{S},i}$ to simulate the execution of Π_{cPSI}.
(3) Once obtaining $\{\widetilde{e}_{\mathcal{S},1,i}, \ldots, \widetilde{e}_{\mathcal{S},m,i}\}_{i\in[d]}$ and a permutation π, the input of Π_{pECRG} from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{S}}$ sends $\langle \text{Response}, \mathsf{OK}\rangle$ to $\mathcal{F}_{\mathsf{FpnMCRG}}$, and obtains $\{\boldsymbol{u}_{\mathcal{S},\boldsymbol{i}}\}_{i\in[m]}$. Finally, for each $i \in [d]$, $\mathsf{Sim}_{\mathcal{S}}$ simulates the execution of Π_{pECRG} with $\{u_{\mathcal{S},1,i}, \ldots, u_{\mathcal{S},m,i}\}$ as output.

We argue that the outputs of $\mathsf{Sim}_{\mathcal{S}}$ are indistinguishable from the real view of $\mathcal{S}$ by the following hybrids:

- Hybrid_0: $\mathcal{S}$'s view in the real protocol.
- Hybrid_1: Same as Hybrid_0 except that, for each $i \in [m]$, each $j \in [B]$, and each $k \in [d]$, the output of Π_{SOT} is replaced by $s_{i,j,k}$ chosen by $\mathsf{Sim}_{\mathcal{S}}$, and $\mathsf{Sim}_{\mathcal{S}}$ runs the Π_{SOT} simulator to produce the simulated view for $\mathcal{S}$. The security of protocol Π_{SOT} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.
- Hybrid_2: Same as Hybrid_1 except that, for each $i \in [m]$, the output of Π_{cPSI} is replaced by $e_{\mathcal{S},i}$ chosen by $\mathsf{Sim}_{\mathcal{S}}$, and $\mathsf{Sim}_{\mathcal{S}}$ runs the Π_{cPSI} simulator to produce the simulated view for

$\mathcal{S}$. The security of protocol Π_{cPSI} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.

- Hybrid_3: Same as Hybrid_2 except that, for each $i \in [d]$, the output of Π_{pECRG} is replaced by $\{u_{\mathcal{S},1,i}, \ldots, u_{\mathcal{S},m,i}\}$ output by $\mathcal{F}_{\mathsf{FpnMCRG}}$ and $\mathsf{Sim}_{\mathcal{S}}$ runs the Π_{pECRG} simulator to produce the simulated view for $\mathsf{Sim}_{\mathcal{S}}$. The security of protocol Π_{pECRG} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.

Clearly, $\mathsf{Sim}_{\mathcal{S}}$'s simulated view is identical to $\mathcal{S}$'s view in the real execution.

Corrupt Receiver: $\mathsf{Sim}_{\mathcal{R}}$ simulates a real execution of the corrupt semi-honest receiver. Since $\mathcal{A}$ is semi-honest, $\mathsf{Sim}_{\mathcal{R}}$ can obtain the input $\{Y_i\}_{i\in[m]}$ of $\mathcal{R}$ directly, and externally send them to $\mathcal{F}_{\mathsf{FOnMCRG}}$ and then receives $\langle \mathsf{Request}, \mathcal{R}\rangle$. It then executes as follows:

(1) For each $i \in [m]$, each $j \in [B]$, and each $k \in [d]$, once obtaining $\mathbf{0}^d$ and $\boldsymbol{v}^p_{i,j,k}$, the input of Π_{SOT}, from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{R}}$ randomly samples $\widetilde{s}_{i,j,k}$ to simulate the execution of Π_{SOT}.
(2) For each $i \in [m]$, once obtaining $\{\widetilde{e}_{i,1}, \ldots, \widetilde{e}_{i,B}\}$, the input of Π_{cPSI} from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{R}}$ randomly samples $e_{\mathcal{R},i}$ and idx to simulate the execution of Π_{cPSI}.
(3) Once obtaining $\{\widetilde{e}_{\mathcal{R},1,i}, \ldots, \widetilde{e}_{\mathcal{R},m,i}\}_{i\in[d]}$, the input of Π_{pECRG} from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{R}}$ sends $\langle \mathsf{Response}, \mathsf{OK}\rangle$ to $\mathcal{F}_{\mathsf{FpnMCRG}}$, and obtains $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$. Finally, for each $i \in [d]$, $\mathsf{Sim}_{\mathcal{R}}$ simulates the execution of Π_{pECRG} with $\{u_{\mathcal{R},1,i}, \ldots, u_{\mathcal{R},m,i}\}$ as output.

We argue that the outputs of $\mathsf{Sim}_{\mathcal{R}}$ are indistinguishable from the real view of $\mathcal{R}$ by the following hybrids:

- Hybrid_0: $\mathcal{R}$'s view in the real protocol.
- Hybrid_1: Same as Hybrid_0 except that, for each $i \in [m]$, each $j \in [B]$, and each $k \in [d]$, the output of Π_{SOT} is replaced by $\widetilde{s}_{i,j,k}$ chosen by $\mathsf{Sim}_{\mathcal{R}}$, and $\mathsf{Sim}_{\mathcal{R}}$ runs the Π_{SOT} simulator to produce the simulated view for $\mathcal{R}$. The security of protocol Π_{SOT} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.
- Hybrid_2: Same as Hybrid_1 except that, for each $i \in [m]$, the outputs of Π_{cPSI} are replaced by $e_{\mathcal{R},i}$ and idx chosen by $\mathsf{Sim}_{\mathcal{R}}$, and $\mathsf{Sim}_{\mathcal{R}}$ runs the Π_{cPSI} simulator to produce the simulated view for $\mathcal{R}$. The security of protocol Π_{cPSI} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.
- Hybrid_3: Same as Hybrid_2 except that, for each $i \in [d]$, the output of Π_{pECRG} is replaced by $\{u_{\mathcal{R},1,i}, \ldots, u_{\mathcal{R},m,i}\}$ output by $\mathcal{F}_{\mathsf{FpnMCRG}}$ and $\mathsf{Sim}_{\mathcal{R}}$ runs the Π_{pECRG} simulator to produce the simulated view for $\mathsf{Sim}_{\mathcal{R}}$. The security of protocol Π_{pECRG} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.

Clearly, $\mathsf{Sim}_{\mathcal{R}}$'s simulated view is identical to $\mathcal{R}$'s view in the real execution. □

D.5 Proof of $\Pi^{\mathsf{dist}_{p\in[1,\infty)}(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$

Theorem D.9 ($\Pi^{\mathsf{dist}_{p\in[1,\infty)}(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ Correctness). *The protocol $\Pi^{\mathsf{dist}_{p\in[1,\infty)}(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ in Figure 12 correctly realizes the functionality $\mathcal{F}_{\mathsf{FpnMCRG}}$ in Figure 10 for all $p \in [1,\infty)$ under the Minkowski distance $\mathsf{dist}_p(\cdot,\cdot)$.*

Proof. For each $i \in [m]$. There are two possible cases.

- **Case 1:** There exists point $Y_i[*] \in Y_i$ such that $\mathsf{dist}_p(\boldsymbol{x}_i, Y_i[*]) \le \delta$. By definition of the p norm, we have for every $k \in [d]$, $|x_{i,k} - Y_i[*][k]|^p \le \delta^p$. Thus, in the $\binom{1}{N}$-SOT, $s_{i,*,k} + \widetilde{s}_{i,*,k} = |x_{i,k} - Y_i[*][k]|^p < \delta^p$, and also $\left|\widetilde{e}_{i,*} - \mathsf{mask}_{i,*}\right| < \delta^p$. By correctness of the DA-set augmentation algorithm, there exists an index $*'$ such that $\mathsf{PRE}_{\widetilde{E},i}[*'] \in \mathsf{PRE}_{\mathsf{MASK},i}$. Therefore, correctness of cPSI implies $e_{\mathcal{S},i,\mathsf{idx}(\mathsf{PRE}_{\widetilde{E},i}[*'])} \oplus e_{\mathcal{R},i,\mathsf{idx}(\mathsf{PRE}_{\widetilde{E},i}[*'])} = 1$, so $e_{\mathcal{S},i} \neq e_{\mathcal{R},i}$. Since all coordinates satisfy $\widetilde{e}_{\mathcal{S},i,k} = H(e_{\mathcal{S},i})$ and $\widetilde{e}_{\mathcal{R},i,k} = H(e_{\mathcal{R},i})$, the inequality $e_{\mathcal{S},i} \neq e_{\mathcal{R},i}$ implies $\widetilde{e}_{\mathcal{S},i,k} \neq \widetilde{e}_{\mathcal{R},i,k}$ for all k. Correctness of pECRG then ensures $u_{\mathcal{S},\pi(i),k} \neq u_{\mathcal{R},\pi(i),k}$ for all $k \in [d]$, and thus the final outputs differ: $\boldsymbol{u}_{\mathcal{S},\pi(i)} \neq \boldsymbol{u}_{\mathcal{R},\boldsymbol{\pi}(i)}$.
- **Case 2:** For every $Y_i[j] \in Y_i$, $\mathsf{dist}_p(\boldsymbol{x}_i, Y_i[j]) > \delta$. Thus for each j there exists $k' \in [d]$ such that $|x_{i,k'} - Y_i[j][k']|^p > \delta^p$, so that $1\binom{1}{N}$-SOT outputs $s_{i,j,k'} + \widetilde{s}_{i,j,k'} = \delta^p + 1$. Hence, $\left|\widetilde{e}_{i,j} - \mathsf{mask}_{i,j}\right| > \delta^p$. Correctness of DA-set augmentation guarantees that no value of $\mathsf{PRE}_{\widetilde{E},i}$ matches any value in $\mathsf{PRE}_{\mathsf{MASK},i}$. By correctness of cPSI, for all such j', $e_{\mathcal{S},i,j'} \oplus e_{\mathcal{R},i,j'} = 0$, so $e_{\mathcal{S},i} = e_{\mathcal{R},i}$. Consequently, $\widetilde{e}_{\mathcal{S},i,k} = \widetilde{e}_{\mathcal{R},i,k}$ for all k, and correctness of pECRG yields $u_{\mathcal{S},\pi(i),k} = u_{\mathcal{R},\pi(i),k}$, so the outputs coincide: $\boldsymbol{u}_{\mathcal{S},\boldsymbol{\pi}(i)} = \boldsymbol{u}_{\mathcal{R},\boldsymbol{\pi}(i)}$.

Since these are exactly the two conditions specified by the ideal functionality $\mathcal{F}_{\mathsf{FpnMCRG}}$, the protocol is correct.

□

Theorem D.10 ($\Pi^{\mathsf{dist}_{p\in[1,\infty)}(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ security). *Assume that the DA-set augmentation algorithm is correct, i.e., it always outputs the augmented distance-aware sets that faithfully preserve the pairwise δ^p distances between the original items. The protocol presented in Figure 12 securely computes the $\mathcal{F}_{\mathsf{FpnMCRG}}$ functionality defined in Figure 10 for $p \in [1,\infty)$ Minkowski distance $\mathsf{dist}_{p\in[1,\infty)}(\cdot,\cdot)$ against semi-honest adversaries in the ($\mathcal{F}_{\mathsf{SOT}}$, $\mathcal{F}_{\mathsf{cPSI}}$,$\mathcal{F}_{\mathsf{pECRG}}$)-hybrid model.*

Proof. We will show that for any adversary $\mathcal{A}$, we can construct two simulators $\mathsf{Sim}_{\mathcal{S}}$ and $\mathsf{Sim}_{\mathcal{R}}$ for simulating the view of the corrupt $\mathcal{S}$ and $\mathcal{R}$ respectively, such that any PPT environment cannot distinguish the execution in the ideal world from that in the real world.

Corrupt Sender: $\mathsf{Sim}_{\mathcal{S}}$ simulates a real execution of the corrupt semi-honest sender. Since $\mathcal{A}$ is semi-honest, $\mathsf{Sim}_{\mathcal{S}}$ can obtain the input $X = \{\boldsymbol{x}_i\}_{i\in[m]}$ of $\mathcal{S}$ directly, and externally send them to $\mathcal{F}_{\mathsf{FpnMCRG}}$ and then receives $\langle \mathsf{Request}, \mathcal{S}\rangle$. It then executes as follows:

(1) For each $i \in [m]$, each $j \in [B]$, and each $k \in [d]$, once obtaining $x_{i,k}$, the input of Π_{SOT}, from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{S}}$ randomly samples $s_{i,j,k}$ to simulate the execution of Π_{SOT}.
(2) For each $i \in [m]$, once obtaining $\mathsf{PRE}_{\mathsf{MASK},i}$, the input of Π_{cPSI} from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{S}}$ randomly samples $e_{\mathcal{S},i}$ to simulate the execution of Π_{cPSI}.
(3) Once obtaining $\{\widetilde{e}_{\mathcal{S},1,i}, \ldots, \widetilde{e}_{\mathcal{S},m,i}\}_{i\in[d]}$ and a permutation π, the input of Π_{pECRG} from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{S}}$ sends $\langle \mathsf{Response}, \mathsf{OK}\rangle$ to $\mathcal{F}_{\mathsf{FpnMCRG}}$, and obtains $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[m]}$. Finally, for each $i \in [d]$, $\mathsf{Sim}_{\mathcal{S}}$ simulates the execution of Π_{pECRG} with $\{u_{\mathcal{S},1,i}, \ldots, u_{\mathcal{S},m,i}\}$ as output.

We argue that the outputs of $\mathsf{Sim}_{\mathcal{S}}$ are indistinguishable from the real view of $\mathcal{S}$ by the following hybrids:

- Hybrid_0: $\mathcal{S}$'s view in the real protocol.
- Hybrid_1: Same as Hybrid_0 except that, for each $i \in [m]$, each $j \in [B]$, and each $k \in [d]$, the output of Π_{SOT} is replaced by $s_{i,j,k}$ chosen by $\mathsf{Sim}_{\mathcal{S}}$, and $\mathsf{Sim}_{\mathcal{S}}$ runs the Π_{SOT} simulator to produce the simulated view for $\mathcal{S}$. The security of protocol Π_{SOT} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.
- Hybrid_2: Same as Hybrid_1 except that, for each $i \in [m]$, the output of Π_{cPSI} is replaced by $e_{\mathcal{S},i}$ chosen by $\mathsf{Sim}_{\mathcal{S}}$, and $\mathsf{Sim}_{\mathcal{S}}$ runs the Π_{cPSI} simulator to produce the simulated view for $\mathcal{S}$. The security of protocol Π_{cPSI} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.
- Hybrid_3: Same as Hybrid_2 except that, for each $i \in [d]$, the output of Π_{pECRG} is replaced by $\{u_{\mathcal{S},1,i}, \ldots, u_{\mathcal{S},m,i}\}$ output by $\mathcal{F}_{\mathsf{FpnMCRG}}$ and $\mathsf{Sim}_{\mathcal{S}}$ runs the Π_{pECRG} simulator to produce the simulated view for $\mathsf{Sim}_{\mathcal{S}}$. The security of protocol Π_{pECRG} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.

Clearly, $\mathsf{Sim}_{\mathcal{S}}$'s simulated view is identical to $\mathcal{S}$'s view in the real execution.

Corrupt Receiver: $\mathsf{Sim}_{\mathcal{R}}$ simulates a real execution of the corrupt semi-honest receiver. Since $\mathcal{A}$ is semi-honest, $\mathsf{Sim}_{\mathcal{R}}$ can obtain the input $\{Y_i\}_{i\in[m]}$ of $\mathcal{R}$ directly, and externally send them to $\mathcal{F}_{\mathsf{FOnMCRG}}$ and then receives $\langle \mathsf{Request}, \mathcal{R}\rangle$. It then executes as follows:

(1) For each $i \in [m]$, each $j \in [B]$, and each $k \in [d]$, once obtaining $\mathbf{0}^d$ and $\boldsymbol{v}^p_{i,j,k}$, the input of Π_{SOT}, from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{R}}$ randomly samples $\widetilde{s}_{i,j,k}$ to simulate the execution of Π_{SOT}.
(2) For each $i \in [m]$, once obtaining $\mathsf{PRE}_{\widetilde{E},i}$, the input of Π_{cPSI} from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{R}}$ randomly samples $e_{\mathcal{R},i}$ and idx to simulate the execution of Π_{cPSI}.
(3) Once obtaining $\{\widetilde{e}_{\mathcal{R},1,i}, \ldots, \widetilde{e}_{\mathcal{R},m,i}\}_{i\in[d]}$, the input of Π_{pECRG} from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{R}}$ sends $\langle \mathsf{Response}, \mathsf{OK}\rangle$ to $\mathcal{F}_{\mathsf{FpnMCRG}}$, and obtains $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[m]}$. Finally, for each $i \in [d]$, $\mathsf{Sim}_{\mathcal{R}}$ simulates the execution of Π_{pECRG} with $\{u_{\mathcal{R},1,i}, \ldots, u_{\mathcal{R},m,i}\}$ as output.

We argue that the outputs of $\mathsf{Sim}_{\mathcal{R}}$ are indistinguishable from the real view of $\mathcal{R}$ by the following hybrids:

- Hybrid_0: $\mathcal{R}$'s view in the real protocol.
- Hybrid_1: Same as Hybrid_0 except that, for each $i \in [m]$, each $j \in [B]$, and each $k \in [d]$, the output of Π_{SOT} is replaced by $\widetilde{s}_{i,j,k}$ chosen by $\mathsf{Sim}_{\mathcal{R}}$, and $\mathsf{Sim}_{\mathcal{R}}$ runs the Π_{SOT} simulator to produce the simulated view for $\mathcal{R}$. The security of protocol Π_{SOT} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.
- Hybrid_2: Same as Hybrid_1 except that, for each $i \in [m]$, the outputs of Π_{cPSI} are replaced by $e_{\mathcal{R},i}$ and idx chosen by $\mathsf{Sim}_{\mathcal{R}}$, and $\mathsf{Sim}_{\mathcal{R}}$ runs the Π_{cPSI} simulator to produce the simulated view for $\mathcal{R}$. The security of protocol Π_{cPSI} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.
- Hybrid_3: Same as Hybrid_2 except that, for each $i \in [d]$, the output of Π_{pECRG} is replaced by $\{u_{\mathcal{R},1,i}, \ldots, u_{\mathcal{R},m,i}\}$ output by $\mathcal{F}_{\mathsf{FpnMCRG}}$ and $\mathsf{Sim}_{\mathcal{R}}$ runs the Π_{pECRG} simulator to produce the simulated view for $\mathsf{Sim}_{\mathcal{R}}$. The security of protocol Π_{pECRG} guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.

Clearly, $\mathsf{Sim}_{\mathcal{R}}$'s simulated view is identical to $\mathcal{R}$'s view in the real execution. □

D.6 Proof of $\Pi^{\mathsf{dist}_p(\cdot,\cdot)}_{\mathsf{fuzzy\text{-}ePSU}}$

Theorem D.11 ($\Pi^{\mathsf{dist}_p(\cdot,\cdot)}_{\mathsf{fuzzy\text{-}ePSU}}$ Correctness). *The protocol $\Pi^{\mathsf{dist}_p(\cdot,\cdot)}_{\mathsf{fuzzy\text{-}ePSU}}$ presented in Figure 13 correctly realizes the functionality $\mathcal{F}^{m,n}_{\mathsf{fuzzy\text{-}ePSU}}$ defined in Figure 3 for the Minkowski distance $\mathsf{dist}_p(\cdot,\cdot)$.*

Proof. For each $i \in [m]$. There are two possible cases.

- **Case 1:** There exists $\boldsymbol{y}_* \in Y$ such that $\mathsf{dist}_p(\boldsymbol{x}_i, \boldsymbol{y}_*) \le \delta$. By the correctness of SAS-Fmap, we have $\mathsf{id}_{\boldsymbol{x}_i} = \mathsf{id}_{\boldsymbol{y}_*}$. Therefore, there exists index $j \in [\beta]$ such that $\mathsf{C}_{\mathcal{S}}[j] = \boldsymbol{x}_i$ and $\boldsymbol{y}_* \in Y_j$. Since $\mathsf{dist}_p(\boldsymbol{x}_i, \boldsymbol{y}_*) \le \delta$, the correctness of FpnMCRG implies $\boldsymbol{u}_{\mathcal{S},\pi(j)} \neq \boldsymbol{u}_{\mathcal{R},\pi(j)}$. Hence, for every $k \in [d]$, $h(v_{\pi(j),k}) \neq v'_{\pi(j),k} \neq h(x_{i,k})$, meaning that $\mathcal{R}$ will not insert $\boldsymbol{x}_i$ into $\widetilde{Z}$ nor into Z. Consequently, $\boldsymbol{x}_i \notin Z$.
- **Case 2:** For every $\boldsymbol{y}_j \in Y$ it holds that $\mathsf{dist}_p(\boldsymbol{x}_i, \boldsymbol{y}_j) > \delta$.
  - **Case 2(a):** $\mathsf{id}_{\boldsymbol{x}_i}$ is different from all $\mathsf{id}_{\boldsymbol{y}_j}$. Then there exists a bin index $j' \in [\beta]$ such that $\mathsf{C}_{\mathcal{S}}[j'] = \boldsymbol{x}_i$, and for all $Y_{j'}[k] \in Y_{j'}$ we have $\mathsf{dist}_p(\boldsymbol{x}_i, Y_{j'}[k]) > \delta$. By the correctness of FpnMCRG, $\boldsymbol{u}_{\mathcal{S},\pi(j')} = \boldsymbol{u}_{\mathcal{R},\pi(j')}$. Therefore, for each $k \in [d]$, $h(v_{\pi(j'),k}) = v'_{\pi(j'),k} = h(x_{i,k})$, which implies $\boldsymbol{x}_i$ is inserted into $\widetilde{Z}$, and thus $\boldsymbol{x}_i \in Z$.
  - **Case 2(b):** There exists $\boldsymbol{y}_* \in Y$ such that $\mathsf{id}_{\boldsymbol{x}_i} = \mathsf{id}_{\boldsymbol{y}_*}$, but $\mathsf{dist}_p(\boldsymbol{x}_i, \boldsymbol{y}_*) > \delta$ still holds. As in Case 1, let j^* satisfy $\mathsf{C}_{\mathcal{S}}[j^*] = \boldsymbol{x}_i$ and $\boldsymbol{y}_* \in Y_{j^*}$. Since $\mathsf{dist}_p(\boldsymbol{x}_i, \boldsymbol{y}_*) > \delta$, and all other points in Y_{j^*} are also at a distance larger than δ, the correctness of FpnMCRG again yields $\boldsymbol{u}_{\mathcal{S},\pi(j^*)} = \boldsymbol{u}_{\mathcal{R},\pi(j^*)}$. Thus, for all $k \in [d]$, $h(v_{\pi(j^*),k}) = v'_{\pi(j^*),k} = h(x_{i,k})$, which implies $\boldsymbol{x}_i \in Z$.

  Hence, in Case 2 we always have $\boldsymbol{x}_i \in Z$.

Since these are exactly the two conditions specified by the ideal functionality $\mathcal{F}^{m,n}_{\mathsf{fuzzy\text{-}ePSU}}$, the protocol is correct. □

Theorem D.12 ($\Pi^{\mathsf{dist}_p(\cdot,\cdot)}_{\mathsf{fuzzy\text{-}ePSU}}$ security). *The protocol presented in Figure 13 securely computes the $\mathcal{F}^{m,n}_{\mathsf{fuzzy\text{-}ePSU}}$ functionality defined in Figure 3 for Minkowski distance $\mathsf{dist}_{\mathsf{p}}(\cdot,\cdot)$ against semi-honest adversaries in the $(\mathcal{F}^{\mathsf{SAS}}_{\mathsf{Fmap}}, \mathcal{F}_{\mathsf{FOnMCRG}})$-hybrid model.*

Proof. We will show that for any adversary $\mathcal{A}$, we can construct two simulators $\mathsf{Sim}_{\mathcal{S}}$ and $\mathsf{Sim}_{\mathcal{R}}$ for simulating the view of the corrupt $\mathcal{S}$ and $\mathcal{R}$ respectively, such that any PPT environment cannot distinguish the execution in the ideal world from that in the real world.

Corrupt Sender: $\mathsf{Sim}_{\mathcal{S}}$ simulates a real execution of the corrupt semi-honest sender. Since $\mathcal{A}$ is semi-honest, $\mathsf{Sim}_{\mathcal{S}}$ can obtain the input $X = \{\boldsymbol{x}_i\}_{i\in[m]}$ of $\mathcal{S}$ directly, and externally send them to $\mathcal{F}^{m,n}_{\mathsf{fuzzy\text{-}ePSU}}$ and then receives $\langle \mathsf{Request}, \mathcal{S}\rangle$. It then executes as follows:

(1) Once obtaining X, the input of $\Pi^{\mathsf{SAS}}_{\mathsf{Fmap}}$, from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{S}}$ randomly samples $\{\mathsf{id}(\boldsymbol{x}_i)\}_{i\in[m]}$ to simulate the execution of $\Pi^{\mathsf{SAS}}_{\mathsf{Fmap}}$.

(2) Once obtaining $\mathsf{C}_{\mathcal{S}}$ and π, the input of $\Pi^{\mathsf{dist}_p(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$, from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{S}}$ randomly selects $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[\beta]}$ to simulate the execution of $\Pi^{\mathsf{dist}_p(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$.
(3) After obtaining $\{\boldsymbol{u}_i\}_{i\in[\beta]}$, $\mathsf{Sim}_{\mathcal{S}}$ sends $\langle\mathsf{Response},\mathsf{OK}\rangle$ to $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$.

We argue that the outputs of $\mathsf{Sim}_{\mathcal{S}}$ are indistinguishable from the real view of $\mathcal{S}$ by the following hybrids:

- Hybrid_0: $\mathcal{S}$'s view in the real protocol.
- Hybrid_1: Same as Hybrid_0 except that the output of $\Pi^{\mathsf{SAS}}_{\mathsf{Fmap}}$ is replaced by $\{\mathsf{id}(\boldsymbol{x}_i)\}_{i\in[m]}$ chosen by $\mathsf{Sim}_{\mathcal{S}}$, and $\mathsf{Sim}_{\mathcal{S}}$ runs the $\Pi^{\mathsf{SAS}}_{\mathsf{Fmap}}$ simulator to produce the simulated view for $\mathcal{S}$. The security of protocol $\Pi^{\mathsf{SAS}}_{\mathsf{Fmap}}$ guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.
- Hybrid_2: Same as Hybrid_1 except that the output of $\Pi^{\mathsf{dist}_p(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ is replaced by $\{\boldsymbol{u}_{\mathcal{S},i}\}_{i\in[\beta]}$ chosen by $\mathsf{Sim}_{\mathcal{S}}$, and $\mathsf{Sim}_{\mathcal{S}}$ runs the $\Pi^{\mathsf{dist}_p(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ simulator to produce the simulated view for $\mathcal{S}$. The security of protocol $\Pi^{\mathsf{dist}_p(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.

Clearly, $\mathsf{Sim}_{\mathcal{S}}$'s simulated view is identical to $\mathcal{S}$'s view in the real execution.

Corrupt Receiver: $\mathsf{Sim}_{\mathcal{R}}$ simulates a real execution of the corrupt semi-honest receiver. Since $\mathcal{A}$ is semi-honest, $\mathsf{Sim}_{\mathcal{R}}$ can obtain the input $Y = \{\boldsymbol{y}_j\}_{j\in[n]}$ of $\mathcal{R}$ directly, and externally send them to $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$ and then receives $\langle\mathsf{Request},\mathcal{R}\rangle$. It then executes as follows:

(1) Once obtaining Y, the input of $\Pi^{\mathsf{SAS}}_{\mathsf{Fmap}}$, from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{R}}$ randomly samples $\{\mathsf{id}(\boldsymbol{y}_j)\}_{j\in[n]}$ to simulate the execution of $\Pi^{\mathsf{SAS}}_{\mathsf{Fmap}}$.
(2) Once obtaining $Y_1,\ldots,Y_\beta$, the input of $\Pi^{\mathsf{dist}_p(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$, from $\mathcal{A}$, $\mathsf{Sim}_{\mathcal{R}}$ randomly selects $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[\beta]}$ to simulate the execution of $\Pi^{\mathsf{dist}_p(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$.
(3) $\mathsf{Sim}_{\mathcal{R}}$ sends $\langle\mathsf{Response},\mathsf{OK}\rangle$ to $\mathcal{F}^{m,n}_{\text{fuzzy-ePSU}}$. After obtaining Z, $\mathsf{Sim}_{\mathcal{R}}$ calculates $\widetilde{Z}$ and randomly selects a subset $\widetilde{U}$ from $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[\beta]}$, where $|\widetilde{U}| = |\widetilde{Z}|$. For each items in $\widetilde{U}$, $\mathsf{Sim}_{\mathcal{R}}$ sets $\boldsymbol{u}_i$, where for each $j\in[d]$, $u_{i,j} = \widetilde{u}_{i,j} \oplus (\widetilde{z}_{i,j}\|h(\widetilde{z}_{i,j}))$. Finally, $\mathsf{Sim}_{\mathcal{R}}$ chooses random items for $i \in \{|\widetilde{U}|+1,\ldots,\beta\}$ and sends all shuffled ciphertexts to $\mathcal{A}$.

We argue that the outputs of $\mathsf{Sim}_{\mathcal{R}}$ are indistinguishable from the real view of $\mathcal{R}$ by the following hybrids:

- Hybrid_0: $\mathcal{R}$'s view in the real protocol.
- Hybrid_1: Same as Hybrid_0 except that the output of $\Pi^{\mathsf{SAS}}_{\mathsf{Fmap}}$ is replaced by $\{\mathsf{id}(\boldsymbol{y}_j)\}_{j\in[n]}$ chosen by $\mathsf{Sim}_{\mathcal{R}}$, and $\mathsf{Sim}_{\mathcal{R}}$ runs the $\Pi^{\mathsf{SAS}}_{\mathsf{Fmap}}$ simulator to produce the simulated view for $\mathcal{R}$. The security of protocol $\Pi^{\mathsf{SAS}}_{\mathsf{Fmap}}$ guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.
- Hybrid_2: Same as Hybrid_1 except that the output of $\Pi^{\mathsf{dist}_p(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ is replaced by $\{\boldsymbol{u}_{\mathcal{R},i}\}_{i\in[\beta]}$ chosen by $\mathsf{Sim}_{\mathcal{R}}$, and $\mathsf{Sim}_{\mathcal{R}}$ runs the $\Pi^{\mathsf{dist}_p(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ simulator to produce the simulated view for $\mathcal{R}$. The security of protocol $\Pi^{\mathsf{dist}_p(\cdot,\cdot)}_{\mathsf{FpnMCRG}}$ guarantees the view in simulation is computationally indistinguishable from the view in the real protocol.

Clearly, $\mathsf{Sim}_{\mathcal{R}}$'s simulated view is identical to $\mathcal{R}$'s view in the real execution. □

E Detailed Performance Analysis in the Minkowski Space

Case $p = \infty$. The experimental results for the ∞-norm Minkowski distance is reported in Table 4. As the input set size increases from 2^4 to 2^{16}, fuzzy PSU and fuzzy ePSU exhibit comparable runtime across all tested dimensions and thresholds. In contrast, fuzzy ePSU consistently achieves lower communication cost.

More specifically, When $m = n = 2^4$, fuzzy ePSU achieves a 1.6–3.3× shrinking in communication compared to fuzzy PSU. This advantage becomes more pronounced at larger scales: when $m = n = 2^{16}$, the communication reduction increases to 3.3–4.4× across different values of (d,δ). These results indicate that, for $p = \infty$, fuzzy ePSU attains substantial communication savings while maintaining runtime performance comparable to fuzzy PSU, in addition to eliminating during-execution leakage.

Cases $p \in \{1,2\}$. The experimental results for the 1-norm and 2-norm Minkowski distances are reported in Table 5. In low-dimension ($d = 2$) setting with small threshold ($\delta = 10$), fuzzy PSU shows clear advantages over fuzzy ePSU. For set sizes ranging from 2^4 to 2^{16}, fuzzy PSU achieves a 1.4–3.0× shrinking in communication and a 2.5–3.3× speedup in runtime for $p = 1$, and a 2.1–3.6× shrinking in communication and a 3.3–4.4× speedup in runtime for $p = 2$.

In high-dimensional settings with $d \in \{6, 10\}$, fuzzy PSU still attains faster runtime, yielding a 1.1–2.2× speedup for $p = 1$ and a 1.1–2.5× speedup for $p = 2$. However, fuzzy ePSU becomes more communication efficient in these configurations: for $p = 1$, it achieves a 1.2–3.1× communication shrinking relative to fuzzy PSU, and for $p = 2$, it achieves a 1.1–3.1× communication shrinking.

Moreover, as both the dimension and the threshold increase, the runtime of fuzzy PSU and fuzzy ePSU gradually becomes comparable in both cases. For example, when $m = n = 2^{16}$, the runtime gap decreases from a 3.3× speedup of fuzzy PSU for $p = 1$ (resp., a 4.4× speedup for $p = 2$) in the configuration $(d = 2, \delta = 10)$ to only about a 1.1× difference for $p = 1$ (resp., a 1.2× difference for $p = 2$) in the configuration $(d = 10, \delta = 30)$.

In summary, for $p \in \{1,2\}$, fuzzy PSU is computationally more efficient in low-dimensional regimes, whereas fuzzy ePSU achieves substantial communication savings in high-dimensional regimes, while additionally eliminating the during-execution leakage inherent in fuzzy PSU at the cost of a moderate increase in computation.

Table 4: Communication cost (MB) and runtime (s) of fuzzy PSU and fuzzy ePSU in Minkowski space under $p = \infty$.

$m = n$	Protocol	(d, δ)											
		(2,10)		(6,10)		(10,10)		(2,30)		(6,30)		(10,30)	
		Comm.	Time	Comm.	Time	Comm.	Time	Comm.	Time	Comm.	Time	Comm.	Time
2^4	$\Pi^{\text{dist}_\infty(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	0.470	0.193	1.384	0.514	2.298	0.743	1.340	0.404	3.994	1.055	6.648	1.493
	$\Pi^{\text{dist}_\infty(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	0.537	0.236	0.862	0.547	1.168	0.804	0.709	0.458	1.377	1.192	2.028	1.515
2^8	$\Pi^{\text{dist}_\infty(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	7.518	1.892	22.140	6.148	36.750	8.713	21.440	4.803	63.900	13.590	106.380	20.822
	$\Pi^{\text{dist}_\infty(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	2.637	2.277	6.760	6.688	10.839	9.957	5.387	4.841	15.009	14.199	24.590	21.529
2^{12}	$\Pi^{\text{dist}_\infty(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	120.200	31.317	354.100	82.442	588.000	136.442	343.000	77.381	1022.000	209.350	1702.000	358.754
	$\Pi^{\text{dist}_\infty(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	36.800	36.574	102.349	90.548	168.137	147.842	80.800	75.900	234.347	208.556	388.137	347.082
2^{16}	$\Pi^{\text{dist}_\infty(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	1924.000	504.030	5665.000	1383.238	9408.000	2358.993	5488.000	1154.985	16358.000	3498.689	27228.000	5745.399
	$\Pi^{\text{dist}_\infty(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	590.500	543.674	1661.357	1508.766	2726.753	2464.272	1294.050	1140.704	3733.365	3382.989	6246.588	5689.891

Table 5: Communication cost (MB) and runtime (s) of fuzzy PSU and fuzzy ePSU in Minkowski spaces under $p \in \{1, 2\}$.

$m = n$	Protocol	(d, δ)											
		(2,10)		(6,10)		(10,10)		(2,30)		(6,30)		(10,30)	
		Comm.	Time	Comm.	Time	Comm.	Time	Comm.	Time	Comm.	Time	Comm.	Time
$p = 1$													
2^4	$\Pi^{\text{dist}_{p=1}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	0.469	0.215	1.371	0.526	2.274	0.765	1.330	0.433	3.951	1.144	6.570	1.771
	$\Pi^{\text{dist}_{p=1}(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	1.393	0.678	1.691	1.156	1.989	1.300	1.651	0.872	2.293	1.553	2.934	2.338
2^8	$\Pi^{\text{dist}_{p=1}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	7.502	2.380	21.840	5.948	36.380	9.012	21.280	5.245	63.210	13.440	105.200	21.534
	$\Pi^{\text{dist}_{p=1}(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	10.714	6.012	14.710	9.442	18.706	13.128	15.359	9.094	24.854	16.738	34.350	25.987
2^{12}	$\Pi^{\text{dist}_{p=1}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	120.000	32.417	351.000	86.566	589.200	139.790	340.300	72.916	1024.000	215.054	1703.000	357.132
	$\Pi^{\text{dist}_{p=1}(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	172.771	91.673	236.615	153.409	300.459	223.703	244.379	146.171	396.223	294.879	548.067	424.248
2^{16}	$\Pi^{\text{dist}_{p=1}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	1919.000	529.660	5685.000	1436.554	9427.000	2349.376	5513.000	1220.915	16382.000	3536.664	27253.000	6004.146
	$\Pi^{\text{dist}_{p=1}(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	3068.437	1767.682	4104.034	2672.875	5139.632	3620.884	4298.300	1930.568	7841.897	4231.041	9185.494	6642.534
$p = 2$													
2^4	$\Pi^{\text{dist}_{p=2}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	0.475	0.235	1.377	0.539	2.279	0.807	1.339	0.487	3.96	1.156	6.581	1.816
	$\Pi^{\text{dist}_{p=2}(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	1.709	0.933	2.007	1.353	2.305	1.750	2.739	1.503	3.381	1.993	4.023	2.880
2^8	$\Pi^{\text{dist}_{p=2}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	7.588	2.464	22.030	5.976	36.910	9.264	21.420	5.264	63.360	13.553	106.600	21.736
	$\Pi^{\text{dist}_{p=2}(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	15.772	8.191	19.768	11.393	23.764	14.961	23.064	12.923	33.100	20.821	34.350	24.582
2^{12}	$\Pi^{\text{dist}_{p=2}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	122.800	34.128	356.700	89.288	590.600	142.707	346.800	76.789	1026.000	219.589	1706.000	360.595
	$\Pi^{\text{dist}_{p=2}(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	257.643	135.884	321.487	197.044	385.331	263.151	385.200	218.342	537.045	341.974	688.889	483.328
2^{16}	$\Pi^{\text{dist}_{p=2}(\cdot,\cdot)}_{\text{fuzzy-PSU}}$	1964.000	566.289	5707.000	1477.500	9449.000	2397.706	5549.000	1270.606	16419.000	3566.706	27289.000	6135.924
	$\Pi^{\text{dist}_{p=2}(\cdot,\cdot)}_{\text{fuzzy-ePSU}}$	4610.683	2518.389	5646.28	3472.523	6681.878	4351.129	6835.064	2724.863	9278.662	4981.261	11722.259	7281.729